FINAL YEAR PROJECT

MULTI-SENSOR SETUP FOR FAULT DETECTION IN CIRCUIT BREAKER

By

Ritesh Patil (211020014)

Grishma Balgi (211021009)

Omkar Ahir (221020911)



Prof. V.B. Suryawanshi

Guide

Mechanical Department

Veermata Jijabai Technological Institute

Mumbai 400 019

2024-25

CERTIFICATE

This is to certify that Ritesh Patil (211020014), Grishma Balgi (211021009) and Omkar Ahir (221020911), students of Bachelor of Technology in Mechanical Engineering, have completed the report titled, "**MULTI- SENSOR SETUP FOR FAULT DETECTION IN CIRCUIT BREAKER**" to our satisfaction.

Prof. V.B. Suryawanshi

Guide

Prof. W.S. Rathod

Head of Department

Director

Date: 2nd May, 2025

Place: VJTI, Mumbai

CERTIFICATE

The report "**MULTI - SENSOR SETUP FOR FAULT DETECTION IN CIRCUIT BREAKER**" submitted by Ritesh Patil (211020014), Grishma Balgi (211021009)) and Omkar Ahir (221020911)) is found to be satisfactory and is approved for the Degree of Final Year Bachelor of Technology in Mechanical Engineering.

Prof. V.B. Suryawanshi **Examiner**

Guide

Date: 2nd May, 2025 Place: VJTI, Mumbai

ACKNOWLEDGMENT

We would like to express our sincere gratitude to the following individuals who have played a pivotal role in the successful completion of our project and the preparation of this report:

We are deeply thankful to **Mr. Avinash Talreja** (HR, Schneider Electric) for his unwavering support at Schneider Electric. His insights into the organization's culture and processes were invaluable in helping us navigate and contribute effectively.

We extend our heartfelt thanks to **Mr. Mahesh Ranade** (Senior General Manager, Schneider Electric) for providing us with the opportunity to intern at Schneider Electric. His mentorship and leadership have been a source of inspiration throughout our project.

We are grateful to **Mr. Amol Kale** (Deputy General Manager, Schneider Electric) for his continuous encouragement and for entrusting us with challenging tasks that enhanced our learning experience during the project.

We are fortunate to have had the opportunity to work with such a dedicated and supportive team at Schneider Electric. Their guidance and mentorship have been instrumental in shaping our careers.

A special thanks to **Mr. V.B. Suryawanshi**, Professor at VJTI, for his unwavering belief in our potential. Mr. Suryawanshi's guidance and support have been instrumental in shaping our academic and professional journey.

We would like to thank the Director of VJTI, **Dr. Sachin Kore**, the Head of the Mechanical Department, **Dr. W.S. Rathod**, for providing us with the facilities that have been so helpful to us throughout this entire process.

We would also like to express our gratitude to every one of our fellow classmates who supported and encouraged us while we worked to complete this project. We would like to thank our parents for their support and assistance.

Lastly, we would like to thank all the teaching and non-teaching staff of the Mechanical Department who provided us with the necessary resources for the completion of our project.

Thank you once again for your unwavering support and encouragement.

DECLARATION

We declare that this written submission represents our ideas in our own words and where other's ideas or words have been included, I have adequately cited and referenced the original sources.

We also declare that weI have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea / data / fact / source in my submission.

We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Ritesh Patil (211020014)	Grishma Balgi (211021009)	Omkar Ahir (221020911)

Date: 2nd May, 2025

ABSTRACT

This project introduces a comprehensive digital twin-based framework for fault detection in CBs using a multi-sensor setup. The system focuses on detecting anomalies in the CB's internal springs using vibration and sound data. A total of four fault states are considered—ranging from zero to three springs removed—representing increasing levels of degradation.

The core hardware comprises an Arduino Nano 33 IoT microcontroller, chosen for its compact design and wireless communication capabilities. It is integrated with a 6-axis accelerometer and an analog sound level sensor to capture real-time data. The data acquisition system supports both Wi-Fi transmission and microSD card backup, ensuring continuous and secure data logging. Collected sensor data is pre-processed, normalized, and stored as CSV files.

Machine learning techniques are central to fault detection. Supervised learning algorithms—including Support Vector Machine (SVM), Decision Tree, XGBoost, Naive Bayes, and K-Nearest Neighbours (KNN)—are trained on the multi-modal dataset to classify the CB's health condition accurately. The models are evaluated based on classification accuracy, confusion matrices, and performance metrics.

To enhance usability, a Python-based Graphical User Interface (GUI) using Tkinter was developed. The GUI allows users to input sensor CSV files, initiate predictions across all five trained models, and view the results instantly. The modular system architecture supports future scalability and integration with industrial predictive maintenance systems. It aligns with modern asset management strategies by offering proactive fault detection capabilities. The fusion of hardware, software, and intelligent analytics makes this project a practical and low-cost solution for real-time circuit breaker monitoring.

Ultimately, the project successfully demonstrates that integrating multi-sensor data acquisition with machine learning and GUI-based fault diagnosis enables accurate, scalable, and real-time condition monitoring for circuit breakers. The work lays a foundation for advancing predictive maintenance in industrial electrical systems, with potential applications in smart grids, substations, and automated monitoring frameworks.

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION 2

1.1 Objectives 3

1.2 Scope and Limitations 4

1.3 Structure of Report 5

CHAPTER 2: LITERATURE REVIEW 7

2.1 Overview of Air Circuit Breakers and Moulded Case Circuit Breakers[1] 7

2.1.1 Air Circuit Breakers (ACBs) 7

2.1.2 Moulded Case Circuit Breakers (MCCBs) 9

2.2 Predictive Maintenance[3] 12

2.3 Digital Twins for ACBs and MCCBs 13

2.3.1 The Concept of a Digital Twin 13

2.3.2 Digital Twin for Air Circuit Breakers 14

2.3.3 Digital Twin for Moulded Case Circuit Breakers 14

2.3.4 Benefits of Digital Twins for ACBs and MCCBs 15

2.3.5 Integration with Predictive Maintenance 15

2.4 Machine Learning[4] 15

2.4.1 Decision Tree 16

2.4.2 Support Vector Machine 17

2.4.3 K-Nearest Neighbors Algorithm 18

2.4.4 XGBoost Algorithm[5] 20

2.4.5 Naive Bayes 21

2.4.6 Random Forest 23

2.5 Graphical User Interfaces (GUI) 25

2.5.1 Principles of Good GUI Design 26

2.5.2 Key Components and Concepts 27

2.5.3 User Interaction and Applications 28

2.6 Previously Published Work 28

2.7 Summary of Literature Review 36
CHAPTER 3: MODEL DEVELOPMENT 39
3.1 System Design 39
3.1.1 Design 41
3.1.2 Sensor Selection Matrix 42
3.1.2.1 Matrix 44
3.1.3 Casing And Sensor Mounting 44
3.1.3.1 Casing 44
3.1.3.2 Sensor Mounting 51
3.1.4 Data Acquisition System 54
3.1.4.1 Acquisition of Sound Data from the DFRobot Gravity 55
3.1.4.2 Acquisition of Vibration Data from the DFRobot Serial 6-Axis Accelerometer 56
3.1.4.3 Synchronization of Multi-Modal Data Acquisition 57
3.1.4.4 Handling Potential Electrical Interference 58
3.1.5 Power Supply and Circuit 58
3.1.5.1 Power Source and Input 58
3.1.5.2 Voltage Regulation and Distribution 60
3.1.7 All Components 62
3.1.7.1 Arduino Nano 33 IoT Microcontroller (with Built-in Wi-Fi Module) 62
3.1.7.2 DFRobot Serial 6-Axis Accelerometer for Arduino (SKU:SEN0386) 63
3.1.7.3 DFRobot Gravity: Analog Sound Level Meter 63
3.1.7.4 DFRobot Fermion: MicroSD Card Module for Arduino 64
3.1.7.5 Custom-Designed and 3D-Printed PLA Cuboidal Case/Enclosure 64
3.1.7.6 DC Power Supply Unit 65
3.1.7.7 Micro USB Cable 65
3.1.7.8 Pin Quick Connectors with Water Rings/Seals 66
3.1.7.9 Multi-Core Wires 66
3.1.7.10 Jumper Wires 67
3.1.7.11 Physical Push Button 67

3.1.7.12 LED Indicator 67
3.1.7.13 Micro SD Card 67
3.2 Data Recording 68
3.2.1 Specific Points of Data Recording 68
3.2.2 Commencement of Data Recording 68
3.2.3 Generation of New Data Logs for Every Recording Session 69
3.2.4 Arduino Code 70
3.3 Machine Learning Model Creation 74
3.3.1 Preprocessing 74
3.3.1.1 Importing Required Libraries 76
3.3.1.2 Defining File Patterns and Parameters 77
3.3.1.3 Reading and Processing Sensor Data 77
3.3.1.4 Smoothing Using Rolling Average 78
3.3.1.5 Combining and Normalizing the Data 78
3.3.2 Model Training 79
3.3.2.1 Data Loading and Labeling 82
3.3.2.2 Feature-Label Separation and Encoding 83
3.3.2.3 Train-Test Split 83
3.3.2.4 Model Training and Prediction 84
3.3.2.5 Evaluation and Visualization 85
3.4 Software Development 88
3.4.1 Model Loading and Setup 91
3.4.2 GUI Design and Development 92
3.4.3 Data Preprocessing and Majority Voting 93
CHAPTER 4: RESULTS AND DISCUSSIONS 96
4.1 System Design 96
4.2 Data Set Recording 97
4.3 Machine Learning Model Creation 98
4.3.1 Preprocessing 98
4.3.1.1 Combined Normalized Datasets 98

4.3.1.2 Raw vs. Smoothed Signal Visualizations 100
4.3.2 Model Training 102
4.3.2.1 Classification Model 103
4.3.2.2 Confusion Matrix Heatmap 106
4.4 Software Development 110
CHAPTER 5: SUMMARY AND CONCLUSIONS 112
CHAPTER 6: FUTURE SCOPE 114
REFERENCES 115

TABLE OF FIGURES

Figure 2.4.1 Decision Tree 23
Figure 2.4.2 Support Vector Machine 24
Figure 2.4.3 K Nearest Neighbour 26
Figure 2.6.1 Experimental Facility 40
Figure 2.6.2 Types of faults 40
Figure 2.6.3 Channels 41
Figure 2.6.4 Aggregate Sampling 41
Figure 2.6.5 Down-Sampling 42
Figure 2.6.6 Random Sampling 42
Figure 3.1.1 Components Matrix 50
Figure 3.1.2 Casing or Enclosure 52
Figure 3.1.3 Arduino Nano 33IoT 53
Figure 3.1.4 SD Card Module 53
Figure 3.1.5 Quick Connector Male/Female 54
Figure 3.1.6 AC Power Supply Input 54
Figure 3.1.7 Enclosure 56
Figure 3.1.8 Case Lid 57
Figure 3.1.9 6- Axis Accelerometer 58
Figure 3.1.10 Analog Sound Level Meter 59
Figure 3.1.11 DAQ Circuit Diagram 61
Figure 3.1.12 DC Internal Power Brick 65
Figure 3.2.1 Arduino Code - I 76
Figure 3.2.2 Arduino Code - II 77
Figure 3.2.3 Arduino Code - III 78
Figure 3.2.4 Arduino Code - IV 79
Figure 3.2.5 Arduino Code - V 80
Figure 3.3.1 Preprocessing Python Code- I 82
Figure 3.3.2 Preprocessing Python Code - II 82

Figure 3.3.3 Importing Required Libraries 83
Figure 3.3.4 Defining File Patterns and Parameters 83
Figure 3.3.5 Reading and Processing Sensor Data 84
Figure 3.3.6 Smoothing Using Rolling Average 84
Figure 3.3.7 Combining and Normalizing the Data 85
Figure 3.3.8 Model Training Python Code - I 87
Figure 3.3.9 Model Training Python Code - II 87
Figure 3.3.10 Model Training Python Code - III 88
Figure 3.3.11 Data Loading and Labelling 89
Figure 3.3.12 Feature- Label Separation and Encoding 89
Figure 3.3.13 Train-Test Split 90
Figure 3.3.14 Model Training and Prediction 90
Figure 3.3.15 Evaluation and Visualisation 92
Figure 3.4.1 Software Development Python Code- I 95
Figure 3.4.2 Software Development Python Code - II 96
Figure 3.4.3 Software Development Python Code - III 96
Figure 3.4.4 Software Development Python Code - IV 97
Figure 3.4.5 Model Loading and Setup 98
Figure 3.4.6 GUI Design and Development 99
Figure 3.4.7 Data Preprocessing and Majority Voting 100
Figure 3.4.8 Data Preprocessing and Majority Voting 101
Figure 4.1.1 System Design 102
Figure 4.2.1 Acceleration Data Set 103
Figure 4.2.2 Sound Data Set 104
Figure 4.3.1 Combined Normalized Datasets 106
Figure 4.3.2 Sound and Vibrations Data for 0 spring removed 107
Figure 4.3.3 Sound and Vibrations Data for 1 spring removed 107
Figure 4.3.4 Sound and Vibrations Data for 2 springs removed 108
Figure 4.3.5 Sound and Vibrations Data for 3 springs removed 108
Figure 4.3.6 Random Forest Classification Report 109

Figure 4.3.7 Random Forest Accuracy 110
Figure 4.3.8 XGBoost Classification Report 110
Figure 4.3.9 XGBoost Accuracy 110
Figure 4.3.10 Decision Tree Classification Report 110
Figure 4.3.11 Decision Tree Accuracy 111
Figure 4.3.12 SVM Classification Report 111
Figure 4.3.13 SVM Accuracy 111
Figure 4.3.15 Naive Bayes Classification Report 111
Figure 4.3.16 Naive Bayes Accuracy 112
Figure 4.3.17 Random Forest Confusion Matrix 113
Figure 4.3.18 XGBoost Confusion Matrix 113
Figure 4.3.19 Decision Tree Confusion Matrix 114
Figure 4.3.20 SVM Confusion Matrix 115
Figure 4.3.21 Naive Bayes Confusion Matrix 116
Figure 4.4.1 GUI Interface 117

CHAPTER 1: INTRODUCTION

In modern power systems, the operational reliability and efficiency of circuit breakers play a pivotal role in maintaining seamless power distribution and system security. Circuit breakers are programmed to sense abnormal conditions like faults or overloads and to short-circuit the flow of current to avoid damaging the equipment and ensuring grid stability. A malfunction in a Circuit Breaker can lead to substantial downtime, equipment loss, and enhanced operational risks. Therefore, there exists a strong need to implement condition-based monitoring and predictive maintenance methodologies to improve the reliability of systems and reduce unwanted outages.

This work proposes a digital twin approach for measuring the operational fitness of a CB through the application of a multi-sensor array with a concentration on vibration and sound data capture. A digital twin, which is a dynamic virtual replica of a physical system, presents a real-time reflection of the CB's state from live sensor feedback. In this regard, the digital twin continuously tracks the CB's internal mechanical parts, with specific interest in identifying abnormalities in the spring mechanisms, which are vital for the device's actuation and reliability.

The test CB under examination has three springs. Fault conditions included in this project include removal of zero (normal condition), one, two, or three springs, simulating increasing degradation. To adequately record these fault conditions, a specific data acquisition (DAQ) system has been created.

The DAQ setup revolves around the use of an Arduino Nano microcontroller, which was chosen because of its low profile, seamless integration, and ability to integrate with various sensors. The DAQ setup contains a 6-axis accelerometer in order to analyse vibration signatures, as well as an analog sound sensor from Robu.in, which is utilized to record sound emissions that characterize mechanical defects. Data logging is facilitated through two channels: real-time Wi-Fi transmission for real-time access and a microSD card backup to maintain data integrity during network loss. This double-logging strategy provides thorough and accurate data collection for downstream analysis.

Captured sensor data is formatted into CSV files to enable the training of machine learning models via Python-based frameworks. Various classification algorithms, such as K-Nearest

Neighbors (KNN), XGBoost, Naive Bayes, and Decision Trees, are utilized to train predictive models with the capability to precisely classify the presence and severity of faults. These predictive models are encapsulated within a graphical user interface (GUI) that has been built using the Tkinter library, enabling the user to input data selection and visualize model outputs.

While the present implementation of the GUI merely displays individual model predictions without further metrics like confidence scores or performance indices, it lays the groundwork for real-time monitoring and user interaction. Through the integration of a robust data acquisition, analysis using machine learning-based methods, and user-friendly interface, the project provides an extensible and flexible solution to predictive maintenance in circuit breaker systems.

1.1 Objectives

The main aim of this project is to create a holistic digital twin framework for fault detection in circuit breakers with an emphasis on tracking the health of the internal spring mechanisms using vibration and sound analysis.

A key element of this goal is the development and integration of a robust data acquisition (DAQ) system that can effectively record sensor data under changing fault conditions. The Arduino Nano microcontroller acts as the central platform, connecting directly to a 6-axis accelerometer and an analog sound sensor. The system accommodates both real-time Wi-Fi-based data transfer and microSD card recording to provide seamless, high-fidelity data logging under various operating conditions.

After data collection, the collected datasets are classified according to the severity of the fault — relating to cases where zero, one, two, or three springs are removed — and formatted into CSV files. These datasets are used to develop machine learning models, in which classifiers including Support Vector Machine, XGBoost, Naive Bayes, and Decision Trees are trained to classify the various fault states based on the features extracted from the sensors.

Another main goal is to implement a graphical user interface (GUI) based on Tkinter that combines the trained machine learning models to enable intuitive fault detection. The GUI provides users with a means of uploading sensor data and obtaining instantaneous fault classification outputs from the single models. While the present release does not include confidence intervals or classification statistics, the system's modular architecture means that

these could be added in the future, for example, performance analytics and real-time alerts.

In general, the project aims to create a technically robust and practically feasible platform for real-time CB health monitoring, with a focus on modularity, scalability, and preparedness for future extensions towards wider predictive maintenance applications.

1.2 Scope and Limitations

In this project, the scope outlines the major functionalities achieved, while the limitations highlight the boundaries within which the system currently operates.

1.2.1 Scope

1. Digital Twin Development: An operable digital twin of the circuit breaker (CB) is developed based on real-time vibration and sound data monitoring.

2. Spring Fault Detection: The system is particularly designed to detect faults that include the replacement of zero, one, two, or three springs.

3. Multi-Sensor Data Acquisition: Arduino Nano captures information from a 6-axis accelerometer and a sound sensor, with double logging through Wi-Fi and microSD card.

4. Integration of Machine Learning: Sensor data are processed by trained machine learning models (KNN, XGBoost, Naive Bayes, Decision Trees) based on CSV-formatted datasets.

5. User Interface: A Tkinter-based GUI enables users to upload data and see individual model predictions in a user-friendly interface.

Though the project adequately fulfilled its main goals, it works within certain limitations that are worth keeping in mind.

1.2.2 Limitations

1. Limited to Spring Faults Only: The system is only tracking spring faults as of now; mechanical or electrical faults are not covered.

2. Offline Simulation-Base Prediction: Real-time live prediction is based on pre-collected CSV data instead of real-time continuous streaming analysis.

3. Environmental Sensitivity: Sound sensor readings can be influenced by external noise, affecting detection accuracy if not used under test conditions.

4. Hardware Constraints: Arduino Nano's limited processing ability restricts onboard data processing; analysis is dependent on external systems.

1.3 Structure of Report

The report starts by detailing the necessity of maintaining electrical systems in good condition and safe, particularly by using devices known as circuit breakers. These breakers interrupt the flow of electricity if a malfunction occurs. The project entails creating an apparatus that is able to identify internal issues in the breaker, such as damaged springs, prior to causing major problems. To accomplish this, the team developed a system that employs sensors for data gathering and a system to analyze the data and provide meaningful results. The objectives and limitations of the work are well specified so that readers know what has been accomplished.

Then, the report examines other researchers' and engineers' similar work. It discusses various breaker types and how organizations are leveraging recent technologies to verify their condition. The group also investigated intelligent tools such as machine learning, which assist in uncovering hidden machine problems by observing patterns within data. Learning from these past studies helped shape the design and gave the team ideas on how to improve fault detection using both sound and movement signals.

Then the report explains in detail how the project was constructed. It explains the tiny sensors that are used to pick up sound and vibration from the breaker. They are attached to a small computer that captures and stores the data securely. The system was designed to be easy, robust, and easy to use. It can also be expanded or upgraded later if necessary. One took care to ensure that all pieces worked together smoothly, and data was gathered in a valuable manner.

The second half addresses the data used to train software which will be able to identify if the breaker is in good condition or faulty. The software is trained with examples and will be able to predict the fault type from unseen data. Various models were experimented and compared in order to determine which performed better. In order to keep the system simple, the team developed a basic computer interface whereby one can load a data file and immediately see what the models forecast.

Lastly, the report presents the results of testing the entire system. It demonstrates that the system

can successfully indicate the number of faulty springs using the sensor readings. The findings are positive and demonstrate that this technique has application in maintenance. The project concludes by discussing what can be done better, i.e., increasing the types of faults or making the system operate live without uploading files. The team recognizes great potential in applying this idea to real-life situations to ensure machines are safer and more dependable.

CHAPTER 2: LITERATURE REVIEW

2.1 Overview of Air Circuit Breakers and Moulded Case Circuit Breakers[1]

The foundation of any electrical distribution system is its capability to safely and reliably control and protect electrical circuits. At the centre of this are circuit breakers, electromechanical devices intended to break fault currents, protecting equipment from damage, reducing downtime, and guaranteeing personnel safety. Of the numerous circuit breaker types, Air Circuit Breakers (ACBs) and Moulded Case Circuit Breakers (MCCBs) are significant devices in low-voltage and medium-voltage applications. Although their basic function is identical, their design philosophies, operating principles, performance features, and application areas are vastly different. An extensive comprehension of such differences is essential for efficient system design, protection coordination, and predictive maintenance plans, which will be discussed subsequently in this project.

2.1.1 Air Circuit Breakers (ACBs)

Air Circuit Breakers are generally defined by their heavy construction, high current ratings (630 A to 6300 A and higher in specialized use), and the use of air at atmospheric pressure as the main arc quenching medium. They are intended for low-voltage systems, typically below 1000 V AC, but some sophisticated designs are extended to cover medium voltage levels up to about 15 kV. ACBs are made for challenging applications where high fault levels are expected and repeated switching duty may be necessary. Working Principle: Upon the occurrence of an overcurrent or short-circuit fault, the sensing devices inside the ACB's trip unit (either electromechanical or advanced electronic types) notice the fault.

This initiates the release mechanism, which instantaneously releases the latch keeping the main contacts in contact. Powerful springs or pneumatics/hydraulics move the movable contacts away from the fixed contacts and thereby cut the current path. Contact separation in high current results in a substantial electric arc with high heat and ionization of the air in the surrounding vicinity. Successful arc quenching is imperative for the reliable and safe operation of the ACB. The main methods used are:

1. Arc Chutes with Deionizing Plates: The arc is rapidly driven into arc chutes, which are typically constructed from insulating materials like ceramic or glass-reinforced polymers. These chutes contain metallic deionizing plates or splitters. As the arc is

forced to move across these plates, it is elongated and segmented into a series of smaller arcs. This increases the overall resistance of the arc, facilitates cooling by heat dissipation to the plates, and promotes deionization of the conductive path, aiding in its extinction.

2. Magnetic Blowout: Magnetic blowout coils, energized by the fault current itself, generate a strong transverse magnetic field across the arc path.The faster the arc is moved into the quenching chamber, the quicker and more effectively it is extinguished.

3. Contact Design and Materials: The main and arcing contacts are meticulously designed. Arcing contacts, made of materials with high arc resistance (e.g., copper-tungsten alloys), are the first to separate and the last to make contact, bearing the brunt of the arcing. Their shape is often optimized to facilitate arc movement into the quenching devices.

4. Air Cooling: The natural movement of air or specifically designed ventilation paths within the ACB enclosure assist in cooling the contacts and the arc quenching structures after an interruption.

Key Components in ACB:

1. Frame and Enclosure: ACBs feature a robust metallic frame providing structural integrity and support for the internal components. The enclosure offers electrical insulation and protection against environmental factors.

2. Main and Arcing Contacts: As described above, designed for high conductivity and arc withstand capability.

3. Operating Mechanism: Typically a stored-energy mechanism (spring-charged or pneumatic/hydraulic) ensures fast and reliable opening and closing operations, independent of the operator's speed. Motor-operated mechanisms allow for remote operation and automatic reclosing schemes.

4. Tripping Unit: Modern ACBs often incorporate sophisticated microprocessor-based electronic trip units. These units offer precise and adjustable protection characteristics, including long-time delay, short-time delay, instantaneous trip, ground fault protection, and even advanced features like thermal memory and pre-alarm indications. They can

also provide data logging and communication capabilities for integration into smart grids and energy management systems.

5. Auxiliary Contacts: These additional contacts operate in conjunction with the main contacts and are used for signalling the breaker's status (open/closed), interlocking with other devices, or controlling auxiliary circuits.

6. Arc Extinguishing System: The core of the ACB's interrupting capability, comprising arc chutes, deionizing plates, magnetic blowout coils, and potentially other arc control devices.

Applications:

1. Main Incoming Feeders: Protecting the primary distribution boards in large industrial plants, commercial buildings, and hospitals where high fault levels are expected.

2. Substation Primary and Secondary Distribution: Controlling and protecting feeders and busbars in electrical substations.

3. Generator Circuit Breakers: Protecting generators from internal faults and managing their connection to the grid.

4. Critical Power Applications: Ensuring the reliability of power supply to essential services where even momentary interruptions are unacceptable.

5. Applications Requiring Frequent Switching: Their robust mechanical design allows for a higher number of operating cycles compared to some other types of circuit breakers.

2.1.2 Moulded Case Circuit Breakers (MCCBs)

Moulded Case Circuit Breakers are small, universal protection devices used in low-voltage systems, usually up to 1000 V AC/DC, and have a current rating between a few amperes to about 2500 A. The characteristic aspect of an MCCB is that it is self-contained in design wherein all elements such as operating mechanism and tripping elements are housed within a sturdy, moulded insulating casing (usually thermosetting polymers). This building lends mechanical support, electrical insulation, and defence against environmental forces.

Operating Principle:

MCCBs primarily rely on a combination of thermal and magnetic tripping mechanisms for overcurrent protection.[2]

1. Thermal Overload Protection: A bimetallic strip, carrying the load current, heats up proportionally to the current. In a sustained overload condition, the differential expansion of the two metals in the strip causes it to bend. After a certain time delay (inversely proportional to the overload current), the bent strip mechanically actuates the trip bar, causing the breaker to open. This provides protection against prolonged, moderate over currents that could lead to overheating and insulation damage.

2. Magnetic Short-Circuit Protection: An electromagnet (solenoid) with a movable armature is also in series with the circuit current. Under a high fault current condition, such as a short circuit, the rapidly increasing current generates a strong magnetic field. This field attracts the armature with sufficient force to overcome a spring restraint, causing it to strike the trip bar and instantaneously open the breaker. This provides rapid protection against severe fault currents that could cause immediate and significant damage.

High-end MCCBs may utilize electronic trip units with more complex and programmable protection features such as accurate time-current curves, ground fault protection, and communications. These electronic trip units operate with current transformers to detect current and a microprocessor to read it and start tripping if specified limits and time delays are breached.

Arc quenching in MCCBs is typically achieved through:

1. Air Break with Arc Chutes: Similar to ACBs, the arc generated during contact separation is drawn into arc chutes. These chutes often contain metal plates (arc splitters) that help to cool and divide the arc, increasing its impedance and facilitating its extinction. The moulded case itself plays a role in containing and directing the arc.

2. Contact Design: The contacts are designed to minimize arc initiation and withstand the energy of the arc for the duration of the interruption.

Key Components in MCCB:

1. Moulded Case: The robust insulating enclosure that provides mechanical protection, electrical insulation, and containment of arc products.

2. Contacts: Designed for the specific current rating and interrupting capacity of the breaker.

3. Operating Handle: Allows for manual opening and closing, and typically indicates the breaker's status (ON, OFF, TRIPPED).

4. Thermal Trip Element (Bimetallic Strip): Provides time-delayed protection against overloads.

5. Magnetic Trip Element (Electromagnet and Armature): Provides instantaneous protection against short circuits.

6. Trip Bar: A mechanical linkage that, when activated by either the thermal or magnetic tripping elements (or the manual trip button), causes the contacts to open.

7. Arc Extinguishing Chamber: Contains arc chutes and splitters to aid in quenching the arc.

8. Auxiliary Contacts and Accessories: Many MCCBs can be equipped with auxiliary contacts for signalling, shunt trip coils for remote tripping, and undervoltage release coils for tripping on loss of voltage.

Applications:

MCCBs are widely used in a vast range of low-voltage electrical installations, including:

1. Distribution Boards: As main breakers, sub-main breakers, and branch circuit breakers in residential, commercial, and industrial buildings.

2. Motor Control Centres(MCCs): Providing overload and short-circuit protection for electric motors.

3. Individual Equipment Protection: Protecting various types of electrical equipment and machinery.

4. DC Applications: Specialized MCCBs are available for DC circuits in applications like solar power and battery systems.

5. OEM Equipment: Integrated into various original equipment manufacturer products for

circuit protection.

2.2 Predictive Maintenance[3]

Predictive Maintenance(PdM) is a forward-looking asset management tactic based on the ongoing monitoring of equipment condition and use of analytical methods to predict impending failures. Moving away from reactive and time-based maintenance approaches, Predictive Maintenance seeks to maximize maintenance interventions by anticipating the remaining useful life (RUL) of assets, thus allowing maintenance to be undertaken only when dictated by the actual equipment condition. This method provides a theoretical path to reducing operational downtime, lowering maintenance costs, and improving overall system reliability and safety.

The theoretical basis of PdM is rooted in the concept that most equipment deterioration processes have recognizable patterns or precursors that can be detected through scientific data collection and analysis. Through constant measurement of critical operating parameters and using diagnostic technologies, a thorough knowledge of an asset's condition can be established. This condition monitoring is the basis on which predictive models are constructed. These models, utilizing statistical analysis and increasingly advanced machine learning algorithms, attempt to detect anomalies, distinguish trends that signify degradation, and eventually forecast time to failure.

The application of a PdM framework requires a systematic process. First, a theoretical determination of asset criticality directs equipment selection for implementation of PdM. Next, theoretical data acquisition requirements are established based on the parameters of interest that indicate the state of the asset to be monitored, e.g., temperature, vibration, electrical properties, and contact resistance for circuit breakers. Theoretical data flow entails transmission, storage, and preprocessing for data quality and readiness for analysis.

The underlying theoretical contribution of PdM is in its analytical stage. Theoretical models are used to identify patterns within monitored data, anomalies that deviate from normal operation, and trends indicating ongoing deterioration, extrapolating these to predict future equipment performance. More sophisticated theoretical methodologies include machine learning applications in which complex relationships between the data are learned and RUL is projected with growing precision.

The theoretical consequence of such analysis work is to provide actionable findings that can be

used to guide maintenance decision-making. PdM theoretically allows for maintenance activities to be scheduled ahead of time, spare parts inventory to be optimized against anticipated requirements, and maintenance resources to be allocated effectively. The ultimate theoretical objective is a closed-loop process wherein PdM interventions are continually assessed for their efficacy, and the predictive models updated based on actual results.

The theoretical advantages of PdM are noteworthy. It theoretically optimizes for unplanned downtime by avoiding potential failures. It minimizes maintenance expense by reducing unwarranted interventions and maximizing the life expectancy of components. Additionally, proactive detection of deteriorating conditions theoretically improves safety by avoiding catastrophic failure. The theoretical improvement in equipment reliability and the move toward data-driven decision-making corroborate the value proposition of PdM as a key element of contemporary asset management theory.

Yet, theoretical application of PdM also has considerations. Theoretical investment in infrastructure, human expertise, and systems integration creates initial challenges. Theoretical dependence on data quality and the requirement for strong analytical capabilities are also key considerations. In spite of these theoretical assumptions, potential for substantial enhancement of operational effectiveness and asset longevity places PdM as a theoretically robust and progressively critical approach to the maintenance of sophisticated industrial assets. The following discussion of Digital Twins and Machine Learning will further detail the theoretical progress in maximizing the potential of Predictive Maintenance of critical electrical equipment.

2.3 Digital Twins for ACBs and MCCBs

2.3.1 The Concept of a Digital Twin

A Digital Twin is an active virtual copy of a physical asset, process, or system. It goes beyond a basic 3D model by actively integrating a steady stream of real-time operational data, advanced simulation capabilities, and embedded analytical tools. The heart of a digital twin is that it can reflect the behavior and performance of its physical equivalent throughout the entire length of its operational life cycle. This is done by continuously updating the virtual object with streams of data from sensors, Internet of Things (IoT) devices, and other sources that are attached to the corresponding actual ACB or MCCB in the field. This real-time data transfer creates a "living" virtual object that matures alongside the physical asset.

2.3.2 Digital Twin for Air Circuit Breakers

For an Air Circuit Breaker, the digital twin would encapsulate a precise geometric model, alongside a wealth of static information encompassing its technical specifications, design parameters, material properties, manufacturing history, and records of past maintenance activities. Crucially, it would also process a continuous flow of real-time data gleaned from various sensors monitoring critical operational aspects. These parameters could include contact temperature, ambient temperature within the enclosure, vibration levels, current and voltage, operating times, and auxiliary contact status. Furthermore, the digital twin would integrate complex simulation models, either physics-based or data-driven, enabling the virtual recreation of the ACB's thermal behavior, mechanical dynamics, arcing phenomena, and electrical responses. Embedded within the digital twin would be advanced analytics and predictive algorithms, often leveraging machine learning techniques, tasked with real-time anomaly detection, fault diagnosis, remaining useful life prediction, and maintenance forecasting.

2.3.3 Digital Twin for Moulded Case Circuit Breakers

Likewise, a digital twin of a Moulded Case Circuit Breaker would be a comprehensive virtual model, including its physical design and static information listing its ratings, tripping behavior, materials, and manufacturing specifications. Dynamically, it would analyze real-time data from sensors tracking case temperature, terminal temperature, load current, operating cycles, and auxiliary contact status. The virtual model would also include simulation functions, with a focus on thermal response and behavior of thermal and magnetic tripping mechanisms. Combined analytical and predictive algorithms, in many cases utilizing AI and machine learning, would be created for detecting abnormal temperature trends, making estimates of tripping mechanism deterioration, and predicting possible future failure based on historic and real-time conditions.

2.3.4 Benefits of Digital Twins for ACBs and MCCBs

The adoption of digital twins for both ACBs and MCCBs yields a multitude of benefits, including enhanced condition monitoring, predictive maintenance capabilities, improved diagnostics, optimized performance, remote monitoring and expertise, better asset management, enhanced safety, and virtual training opportunities.

2.3.5 Integration with Predictive Maintenance

Essentially, digital twins become a strong enabler for Predictive Maintenance policies. The

seamless flow of live data combined with advanced analytical instruments greatly enhances PdM effectiveness and accuracy levels for ACBs and MCCBs, resulting in better-informed maintenance decisions as well as optimum scheduling of the maintenance process. By bridging the physical and digital worlds, digital twins offer extensive information and analysis capabilities, which open the way to proactive maintenance, increased reliability, and higher efficiency and safety in electrical power systems.

2.4 Machine Learning[4]

Machine Learning (ML) is a branch of Artificial Intelligence (AI) that focuses on building systems capable of learning and improving from experience without being explicitly programmed.

Purpose:

1. Automated decision-making processes.
2. Identify patterns in large datasets.
3. Make predictions or classifications based on input data

Types of Machine Learning Algorithms

1. Supervised Learning
2. Unsupervised Learning
3. Reinforcement Learning

2.4.1 Decision Tree

A decision tree is a supervised machine learning algorithm used for both classification and regression tasks. It resembles a flowchart where each internal node represents a test on an attribute, each branch represents an outcome of the test, and each leaf node represents a class.

How it Works:

1. Root Node: The starting point of the tree, representing the entire dataset.

2. Splitting: The algorithm iteratively splits the dataset into subsets based on specific attributes.
3. Branching: Each split creates a new branch, leading to further divisions of the data.
4. Leaf Nodes: The final nodes in the tree, representing the predicted class label or value.

Key Concepts:

1. Information Gain: A measure used to determine the best attribute for splitting the data.
2. Gini Impurity: Another metric used to evaluate the quality of a split.
3. Pruning: A technique to reduce the complexity of the tree by removing unnecessary branches.

Advantages:

1. Interpretability: Decision trees are easy to understand and visualize.
2. Handles both numerical and categorical data: Versatile for various datasets.
3. Non-parametric: Doesn't assume underlying data distribution.

Disadvantages:

1. Overfitting: Can be sensitive to noise in the data, leading to complex and less accurate models.
2. Instability: Small changes in the data can lead to significant changes in the tree structure.

```
In [1]: from sklearn.tree import DecisionTreeClassifier
        from sklearn.model_selection import train_test_split
        from sklearn.datasets import load_iris
        from sklearn.metrics import accuracy_score

        # Load the Iris dataset
        iris = load_iris()
        X = iris.data
        y = iris.target

        # Split the data into training and testing sets
        X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

        # Create a Decision Tree Classifier
        clf = DecisionTreeClassifier()

        # Train the model
        clf.fit(X_train, y_train)

        # Make predictions on the test set
        y_pred = clf.predict(X_test)

        # Evaluate the model's accuracy
        accuracy = accuracy_score(y_test, y_pred)
        print("Accuracy:", accuracy)
```

Figure 2.4.1 Decision Tree

2.4.2 Support Vector Machine

A Support Vector Machine (SVM) is a supervised learning algorithm for both classification and regression problems. Its fundamental concept is to find a best hyperplane (a line in 2D or a plane in 3D, or its n-dimensional equivalent) that maximizes the margin between classes, in effect, dividing the data. The "support vectors" are the points nearest to this hyperplane since they have a strong impact on its position and thus the boundary of classification.

How it Works:

1. Find the Best Boundary: SVM aims to find the best possible boundary (hyperplane) that separates the two classes (apples and oranges) with maximum margin.

2. Support Vectors: The data points closest to the boundary are called support vectors. These points play a crucial role in determining the position of the boundary.

3. Kernel Trick: In complex scenarios where data isn't linearly separable, SVM uses a technique called the kernel trick. This trick maps the data into a higher-dimensional space where it becomes linearly separable.

Key Concepts:

1. Margin: The distance between the hyperplane and the nearest data points.

2. Kernel Trick: A mathematical function that transforms data into a higher-dimensional

space.

3. Support Vectors: The data points that influence the position of the hyperplane.

Advantages:

1. Effective in high-dimensional spaces: Handles complex datasets with many features.
2. Versatile: Can be used for both classification and regression tasks.
3. Robust to outliers: Less sensitive to noise in the data.

Disadvantages:

1. Sensitive to parameter tuning: Requires careful tuning of hyperparameters.
2. Not as scalable as some other algorithms: Can be computationally expensive for large datasets.

```
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import classification_report, accuracy_score, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

# Load the Iris dataset
iris = datasets.load_iris()
X = iris.data
y = iris.target

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

svm_model = SVC(kernel='rbf', gamma=0.1, C=1.0)
svm_model.fit(X_train, y_train)

# Make predictions on the test set
y_pred = svm_model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))

# Confusion Matrix
conf_matrix = confusion_matrix(y_test, y_pred)
sns.heatmap(conf_matrix, annot=True, cmap='Blues', fmt='d', xticklabels=iris.target_names, yticklabels=iris.target_names)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()
```

Figure 2.4.2 Support Vector Machine

2.4.3 K-Nearest Neighbors Algorithm

K-Nearest Neighbors (KNN) is a simple, yet powerful supervised machine learning algorithm used for both classification and regression tasks. It classifies new data points based on the majority vote of its nearest neighbors.

How it Works:

1. Determine the Distance Metric: A distance metric, such as Euclidean distance or Manhattan distance, is used to calculate the distance between data points.

2. Identify the k-Nearest Neighbors: The algorithm identifies the k closest data points to the new data point based on the chosen distance metric.

3. Classification: For classification tasks, the majority vote of the k-nearest neighbors determines the class label of the new data point.

4. Regression: For regression tasks, the average value of the k-nearest neighbors is assigned as the predicted value.

Key Concepts:

1. K-value: The number of neighbors considered.

2. Distance Metric: The method used to calculate the distance between data points.

3. Majority Vote: For classification, the most frequent class among the neighbors.

4. Average Value: For regression, the average of the target values of the neighbors.

Advantages:

1. Simple to understand and implement: Easy to grasp the underlying concept.

2. Versatile: Can be used for both classification and regression.

3. No training phase: Directly uses the training data for predictions.

Disadvantages:

1. Sensitive to noisy data: Outliers can significantly impact predictions.

2. Computational cost: Can be computationally expensive for large datasets, especially when calculating distances.

3. Curse of dimensionality: Performance can degrade in high-dimensional spaces.

KNN is a powerful algorithm for various tasks, but its performance can be influenced by the choice of the k-value and the distance metric. By carefully selecting these parameters, KNN can be a valuable tool in machine learning.

```python
# Import necessary libraries
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report, accuracy_score, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

# Load the Iris dataset
iris = datasets.load_iris()
X = iris.data  # Features (sepal and petal dimensions)
y = iris.target  # Target (species)

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Initialize and train the KNN model
knn_model = KNeighborsClassifier(n_neighbors=5)  # Set the number of neighbors (k)
knn_model.fit(X_train, y_train)

# Make predictions on the test set
y_pred = knn_model.predict(X_test)

# Evaluate the model
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Figure 2.4.3 K Nearest Neighbour

2.4.4 XGBoost Algorithm[5]

XGBoost is a powerful and efficient ensemble learning algorithm that combines multiple weak models (typically decision trees) to create a strong predictive model. It's an optimized implementation of gradient boosting.

How it works:

1. Initial Model: A base model is trained on the entire dataset.
2. Residual Calculation: The errors or residuals of the base model are calculated.
3. Model Training: A new model is trained to predict these residuals.
4. Model Combination: The predictions of the new model are added to the predictions of the base model.
5. Iteration: Steps 2-4 are repeated iteratively, with each new model focusing on correcting the errors of the previous models.

Key Concepts:

1. Regularization: XGBoost incorporates regularization techniques to prevent overfitting, such as L1 and L2 regularization.

2. Tree Pruning: It automatically prunes trees to avoid overfitting and improve generalization.

3. Parallel Processing: XGBoost can utilize parallel and distributed computing to accelerate training.

4. Handling Missing Values: It has built-in mechanisms to handle missing values.

5. Flexibility: It can handle various types of data, including numerical and categorical data.

2.4.5 Naive Bayes

Naive Bayes is a supervised machine learning algorithm primarily used for classification tasks. It's based on Bayes' theorem and makes a "naive" assumption that the features within a dataset are independent of each other given the class label. Despite this often unrealistic assumption, Naive Bayes performs surprisingly well in many real-world applications, especially text classification.

How it Works:

1. Probability-Based Classification: Naive Bayes calculates the probability of a data point belonging to a particular class based on the probabilities of its features occurring within that class.

2. Bayes' Theorem: It utilizes Bayes' theorem to calculate the posterior probability of a class given the features.

$$\mathrm{P}(Class|Features) = \frac{\mathrm{P}(Features|Class) \times \mathrm{P}(Class)}{\mathrm{P}(Features)}$$

Where:

- P(Class|Features) is the posterior probability of the class given the features.

- ➢ P(Features|Class) is the likelihood of the features given the class.
- ➢ P(Class) is the prior probability of the class.
- ➢ P(Features) is the prior probability of the features.

3. Naive Independence Assumption: The "naive" part comes from assuming that all features are independent of each other within each class. This simplifies the calculation of P(Features|Class) as the product of the probabilities of each individual feature given the class.
4. Classification Decision: The algorithm predicts the class with the highest posterior probability.

Key Concepts:

1. Bayes' Theorem: The fundamental probabilistic theorem used for classification.
2. Conditional Probability: The probability of an event occurring given that another event has already occurred.
3. Prior Probability: The initial probability of a class before observing any features.
4. Likelihood: The probability of observing the features given a particular class.
5. Posterior Probability: The updated probability of a class after observing the features.
6. Feature Independence: The assumption that the features are independent of each other within each class.

Advantages:

1. Simple and Easy to Implement: The underlying concepts are straightforward, and implementation is relatively simple.
2. Computationally Efficient: It's very fast, both in training and prediction, making it suitable for large datasets.
3. Effective with High-Dimensional Data: Performs well with a large number of features, such as in text classification.

4. Works Well with Categorical Data: Naturally handles categorical features.

5. Requires Less Training Data: Can perform well even with limited training data compared to some other algorithms.

Disadvantages:

1. Strong Independence Assumption: The assumption of feature independence is often violated in real-world scenarios, which can affect accuracy.

2. Zero Frequency Problem: If a feature value never occurs with a particular class in the training data, its probability will be zero, potentially leading to a zero posterior probability for that class, regardless of other features. Smoothing techniques (like Laplace smoothing) can help mitigate this.

3. Not Suitable for Complex Relationships: May not perform as well as more complex models when there are intricate dependencies between features.

4. Can be a Bad Estimator: While it can be a good classifier, the probability estimates it provides might not be very accurate.

2.4.6 Random Forest

Random Forest is a supervised learning algorithm for both classification and regression problems. It's an ensemble learning technique that works by building many decision trees at training time and then aggregating the predictions of the individual trees to produce a final prediction. The "randomness" in the name refers to two important features: random selection of data points and random feature selection.

How it Works:

1. Ensemble of Decision Trees: Random Forest builds multiple decision trees on different subsets of the data.

2. Bootstrap Aggregating (Bagging): Each tree is trained on a random subset of the original training data, sampled with replacement. This process is called bootstrapping.

3. Random Feature Selection: When splitting each node in a tree, the algorithm considers

only a random subset of the features. The best split is then chosen from this subset. This introduces diversity among the trees.

4. Prediction Aggregation:

    - ➢ Classification: The final class prediction is determined by the majority vote of the predictions from all the individual trees in the forest.

    - ➢ Regression: The final prediction is the average of the predictions from all the individual trees.

Key Concepts:

1. Ensemble Learning: Combining the predictions of multiple models to improve overall performance.

2. Decision Trees: Tree-like structures that make decisions based on a series of feature-based rules.

3. Bagging (Bootstrap Aggregating): Creating multiple subsets of the training data (with replacement) and training a model on each subset.

4. Random Subspace: Randomly selecting a subset of features to consider at each split in the decision tree.

5. Majority Voting: In classification, the final prediction is the class that receives the most votes from the individual trees.

6. Averaging: In regression, the final prediction is the average of the predictions from the individual trees.

Advantages:

1. High Accuracy: Generally provides high prediction accuracy and is often considered one of the top-performing algorithms.

2. Robust to Overfitting: The combination of bagging and random feature selection reduces the risk of overfitting, making it generalize well to unseen data.

3. Handles High-Dimensional Data: Can effectively handle datasets with a large number of features.

4. Provides Feature Importance: It can estimate the importance of each feature in the prediction process.

5. Handles Missing Values: Can maintain good accuracy even when there are missing values in the data.

6. Works Well with Both Categorical and Numerical Data: Can handle a mix of different data types.

7. Less Sensitive to Outliers: Compared to some other algorithms, it's less affected by outliers in the data.

Disadvantages:

1. Less Interpretable: The large number of trees in the forest makes it more difficult to interpret the model compared to a single decision tree.

2. Can be Computationally Expensive: Training a large number of trees can be computationally intensive, especially for very large datasets.

3. Requires More Resources: Random forests typically require more memory to store the ensemble of trees.

4. Prediction Time Can Be Slower: Making predictions can be slower than with a single decision tree, although still generally fast.

2.5 Graphical User Interfaces (GUI)

A Graphical User Interface (GUI) is a user interface in which users can communicate with electronic devices using visual representation indicators (icons, graphics, menus) instead of text-based commands. The following literature review will discuss the creation, characteristics, and application of GUIs.

2.5.1 Principles of Good GUI Design

The creation of an effective GUI is guided by several key principles aimed at enhancing

usability, efficiency, and user satisfaction. These principles are often iterated upon and refined by experts in the field. Some core concepts include:

1. Visibility of System Status: Users must always know what's going on, with feedback provided within a reasonable time. This includes loading indicators, progress bars, and good status messages.

2. Match Between System and the Real World: The language, concepts, and metaphors used in the interface should be familiar to the user, with the interaction being natural and intuitive. Information should be presented in a reasonable order.

3. User Freedom and Control: Users must be able to easily manipulate and control the system, having easy methods of undoing and redoing operations, and exiting unwanted situations.

4. Consistency and Conventions: Consistency in design components (icons, vocabulary, layout) throughout the interface and following platform conventions decrease the learning curve for the user and enhance predictability.

5. Error Prevention: Making the system design minimize errors from occurring is important. This can include input constraints, good instructions, and warnings prior to critical actions.

6. Recognition Rather Than Recall: Reducing the user's memory load by making options, actions, and information apparent. Users must be able to recognize what to do instead of recalling commands or procedures.

7. Flexibility and Efficiency of Use: The interface must be appropriate for both inexperienced and experienced users. Offering shortcuts, customization possibilities, and accelerators can facilitate efficiency for knowledgeable users without precluding inexperienced ones.

8. Aesthetic and Minimalist Design: Interfaces must be pretty and unencumbered with unnecessary information. Each element should have a function, and nothing should be allowed to clutter to concentrate the attention of the user.

9. Guide Users to Recognize, Diagnose, and Recover from Mistakes: Error messages must be written in simple language, describe the problem clearly, and propose solutions.

10. Help and Documentation: Although the ultimate goal is frequently an intuitive interface, giving users accessible and easily searchable help and documentation can help them out when they need it.

2.5.2 Key Components and Concepts

A GUI is characterized by several fundamental elements that enable direct manipulation and visual interaction:

1. Windows: Discrete areas on the screen that contain applications or documents, allowing users to manage multiple tasks and information simultaneously.

2. Icons: Small graphical representations of applications, files, commands, or devices, providing a visual and easily recognizable way to interact with the system.

3. Menus: Lists of commands or options organized hierarchically, allowing users to access various functionalities of an application.

4. Pointing Devices (e.g., Mouse, Touchscreen): Hardware interfaces that allow users to interact with on-screen elements through actions like pointing, clicking, dragging, and gesturing.

5. Direct Manipulation: A core concept where users interact directly with visual representations of objects and actions, making the interaction feel intuitive and immediate. For example, dragging a file icon to a folder to move it.

6. Widgets (Controls): Interactive elements within windows that allow users to perform specific actions or input data. Common widgets include buttons, text fields, checkboxes, sliders, and dropdown lists.

The combination of these components creates an environment where users can navigate, execute commands, and interact with digital content in a visual and often more intuitive manner compared to command-line interfaces.

2.5.3 User Interaction and Applications

GUIs are now the preeminent mode of computer user interaction and also many other computerized devices owing to their user-friendliness and visual-centric nature. Here is how

they are normally employed:

1. Navigation: Users use pointing devices or touches to relocate a cursor or a finger on the screen and operate visual objects like icons, menus, and links to navigate an application or an operating system.

2. Command Execution: Commands are issued by directly manipulating graphic objects. A click on a button, choice on a menu, or drag-and-drop icon all equate to issuing certain commands to the underlying system.

3. Data Input and Output: GUIs offer different widgets for users to enter data (e.g., entering text into text fields, choosing options from pull-down menus) and show output in a format easy for humans to understand visually (e.g., charts, graphs, text, pictures).

4. Multitasking: Windowing systems enable users to have several programs open at once and switch between them as required.

5. Accessibility: Contemporary GUI design more and more includes features for accessibility for people with disabilities, including keyboard navigation, high-contrast themes, and screen readers.

2.6 Previously Published Work

1) Satish Kumar, Tushar Kolekar, Arunkumar Bongale, Shruti Patil, Atef Zaguia, Chander Prakash

LOW-COST MULTI-SENSOR DATA ACQUISITION SYSTEM FOR FAULT DETECTION IN FUSED DEPOSITION MODELLING[6]

The research paper titled "A Low-Cost Multi-Sensor Data Acquisition System for Fault Detection in Fused Deposition Modelling" presents the development of a cost-effective data acquisition system tailored for fault detection in fused deposition modeling. This work addresses the critical need for effective monitoring and diagnosis of defects arising during the 3D printing process, leveraging multiple sensors to enhance detection accuracy.

Despite its advantages, including rapid prototyping and cost-effectiveness, it suffers from common defects such as warping, cracking, and surface irregularities caused by temperature fluctuations, vibrations, and mechanical faults. Effective fault diagnosis during the printing

process can help mitigate these issues. To tackle these challenges, the authors designed a DAQ system using an Arduino microcontroller. The system integrates vibration, current, and sound sensors to monitor the printing process in real-time. The captured data is processed using advanced machine learning models, including convolutional neural networks (CNNs), to classify normal and faulty conditions with high accuracy.

2) Yannick Wilhelm, Peter Reimann, Wolfgang Gauchel, Bernhard Mitschang[7]

OVERVIEW ON HYBRID APPROACHES TO FAULT DETECTION AND DIAGNOSIS: COMBINING DATA-DRIVEN, PHYSICS-BASED AND KNOWLEDGE-BASED MODELS

The paper emphasizes integrating multiple approaches to address the limitations of standalone techniques in enhancing the accuracy and reliability of fault detection systems. Fault detection 3 and diagnosis consist of three main steps: fault detection, fault diagnosis, and fault correction. The study classifies FDD methods into three categories: data-driven methods, which utilize historical data to detect and diagnose faults using techniques such as Principal Component Analysis, neural networks, and clustering; physics-based methods, which employ mathematical models to represent the system's physical behavior and detect inconsistencies between expected and observed performance; and knowledge-based methods, which incorporate domain expertise using qualitative models like fault trees, expert systems, or ontologies to infer faults and corrective actions.

Hybrid approaches aim to combine the strengths of data-driven, physics-based, and knowledge based methods to improve FDD accuracy. The paper categorizes hybrid strategies into three main types: serial combination, where fault detection and diagnosis methods are applied sequentially, each method's output serving as input for the next stage to improve decision accuracy; parallel combination, where multiple FDD methods operate independently, and their results are fused at the data, feature, or decision level to enhance reliability using techniques such as Bayesian fusion and Dempster-Shafer theory; and mixed combination, which integrates both serial and parallel strategies to enhance system robustness. The paper also introduces an extended workflow for hybrid FDD systems, outlining steps such as data acquisition, feature extraction, fault detection, fault diagnosis, and linking diagnosed faults to appropriate recovery actions through domain knowledge integration. While hybrid FDD approaches hold significant promise, challenges remain, such as ensuring data quality, the labor-intensive development of

accurate physical models, and the computational complexity of knowledge-based systems. The authors call for future research to explore unexplored hybrid strategies, improve the integration of domain knowledge, and enhance automated decision-making systems.

3) Aitor Duo, Tiziana Segreto, Alessandra Caggiano, Rosa Basagoiti, Roberto Teti, Pedro J. Arrazola[10]

DRILLING PROCESS MONITORING: A FRAMEWORK FOR DATA GATHERING AND FEATURE EXTRACTION TECHNIQUES

The paper titled "Drilling Process Monitoring: A Framework for Data Gathering and Feature Extraction Techniques" outlines a comprehensive framework for monitoring drilling processes, 4 emphasizing the role of sensor-based data acquisition and advanced signal processing methods. Drilling is a critical machining operation prone to tool wear and surface roughness issues that impact product quality. The paper discusses the setup for capturing internal signals (e.g., spindle power, tool acceleration) and external signals (e.g., vibrations, acoustic emissions, and cutting forces) to monitor these processes effectively. Sensor data, processed using time domain and frequency-domain analyses, facilitates the extraction of features representing key process parameters such as thrust force, torque, and vibrations. These features enable faster and more accurate decision-making by identifying anomalies and predicting tool wear or roughness profiles. The integration of advanced techniques like wavelet packet transforms (WPT) and principal component analysis (PCA) for feature extraction and dimensionality reduction further enhances the monitoring system's efficiency.

A central focus of the framework is on optimizing feature selection to address the challenges posed by high-dimensional sensor data. Methods like filtering, wrapper techniques, and embedded algorithms are employed to identify the most relevant features while eliminating redundancies, thus simplifying modeling and reducing computational costs. The research highlights the importance of combining internal and external signals for a holistic understanding of the drilling process. Future work aims to improve prediction accuracy by leveraging machine learning algorithms and exploring relationships between sensor signals and surface roughness parameters. The presented framework paves the way for robust monitoring systems capable of ensuring high precision, cost-efficiency, and sustainability in modern machining processes.

4) Giovanni Marco Bisanti, Luca Mainetti, Teodoro Montanaro, Luigi Patrono, Ilaria Sergi[9]

DIGITAL TWINS FOR AIRCRAFT MAINTENANCE AND OPERATION: A SYSTEMATIC LITERATURE REVIEW AND AN IOT-ENABLED MODULAR ARCHITECTURE

This paper emphasizes the potential and applications of Digital Twin (DT) technology in enhancing aircraft maintenance and operation. Building on advancements in the Internet of 5 Things (IoT) and Industry 4.0, it examines how DTs act as virtual replicas of physical assets, leveraging real-time data for diagnostics, prognostics, and predictive maintenance in the aerospace sector. The study systematically reviews existing literature from 2018 to 2023, categorizing approaches as either data-driven, physics-based, or hybrid models, and proposes a modular IoT-enabled architecture. This architecture aims to unify these methodologies, addressing diverse challenges like fault detection, remaining useful life (RUL) estimation, and system optimization for critical components such as engines, wings, and cooling systems.

Moreover, the research highlights gaps in the integration of IoT, data scarcity, real-time decision-making, and the need for explainable AI in predictive maintenance systems. The proposed modular framework combines data acquisition, preprocessing, and machine learning components with visualization tools, offering flexibility for researchers and practitioners. By consolidating insights and suggesting future directions, the paper contributes to advancing DT applications in aviation, fostering collaboration and innovation in maintaining and optimizing aircraft systems.

5) M. Runde, T. Aurud, L.E. Lundgaard, G.E. Ottesen, K. Faugstad[8]

ACOUSTIC DIAGNOSIS OF HIGH VOLTAGE CIRCUIT-BREAKERS

This paper emphasizes the development of a non-invasive diagnostic technique for high voltage circuit breakers, focusing on acoustic signals generated during non-energized switching operations. By analyzing these acoustic "signatures," which reflect the mechanical actions within the breaker, the study demonstrates the ability to detect wear, mechanical faults, and abnormal conditions early. Employing digital signal processing techniques adapted from speech recognition, the research outlines methods for extracting and comparing signatures to identify discrepancies. The work showcases a practical case where a fault in the release mechanism was detected in its early stages, preventing potential operational failures.

The paper further highlights the reliability and robustness of this method, noting its immunity to external noise and its high reproducibility across different operational conditions. By combining acoustic analysis with complementary diagnostic tools, such as dynamic contact resistance measurements, the study presents a comprehensive system for assessing breaker 6 health. While promising, the approach requires more extensive field data to establish its effectiveness fully. The findings suggest significant potential for improving maintenance practices, minimizing downtime, and extending the operational life of circuit breakers.

6) Siyu Shao Stephen McAleer, Ruqiang Yan, Pierre Baldi

HIGHLY ACCURATE MACHINE FAULT DIAGNOSIS USING DEEP TRANSFER LEARNING

Machine fault diagnosis has evolved as a critical field, addressing the need for precise identification and classification of mechanical failures to enhance operational safety and efficiency. Traditional fault diagnosis systems typically involve sensor signal acquisition, manual feature extraction, and subsequent classification using machine learning models. However, these methods often fall short due to their reliance on handcrafted features, which are task-specific and lack generalizability across varying operational conditions. The advent of deep learning has significantly transformed this landscape. By leveraging neural networks with multiple hidden layers, researchers have achieved automatic features learning directly from raw data. For example, convolutional neural networks (CNNs) have demonstrated substantial success in learning hierarchical features from time-frequency images, enabling accurate fault classification without extensive manual intervention.

Transfer learning further amplifies the capabilities of deep learning in this domain by reducing the computational and data demands typically associated with training deep architectures from scratch. By fine-tuning pre-trained models, such as VGG-16, initially trained on large-scale natural image datasets, researchers have managed to adapt these networks effectively for mechanical fault diagnosis tasks. This approach not only expedites the training process but also enhances diagnostic accuracy. Recent studies have shown state-of-the-art results using this strategy, achieving nearly perfect fault detection across diverse mechanical datasets, including bearings, gearboxes, and induction motors. These advancements underscore the potential of combining deep learning and transfer learning to establish robust and scalable fault diagnosis frameworks.

2.6 Dataset Practiced

This repository contains mechanical datasets used in the paper "Highly Accurate Machine Fault Diagnosis Using Deep Transfer Learning". All the data are the original vibration signals acquired by sensors.

Bearing dataset is from Case Western Reserve University bearing data center. For the names of the data files, for example 'B007_0', the first letter represents fault position, the next three numbers represent fault diameters (0.007, 0.014, 0.021 inches) and the last number represents bearing loads (0,1,2,3). There are three kinds of fault positions here, B-bearing rolling element, IR-inner raceway, OR-outer raceway. The data we used is all from the fan end which is marked as 'FE' in the data files.

The Gearbox dataset is from Southeast University, China. This data is collected from Drivetrain Dynamic Simulator. This dataset contains 2 sub datasets, including bearing data and gear data, which are both acquired on Drivetrain Dynamics Simulator (DDS). There are two kinds of working conditions with rotating speed - load configuration set to be 20-0 and 30-2. Within each file, there are 8rows of signals which represent: 1-motor vibration, 2,3,4-vibration of planetary gearbox in three directions: x, y, and z, 5-motor torque, 6,7,8-vibration of parallel gear box in three directions: x, y, and z. Signals of rows 2,3,4 are all effective.

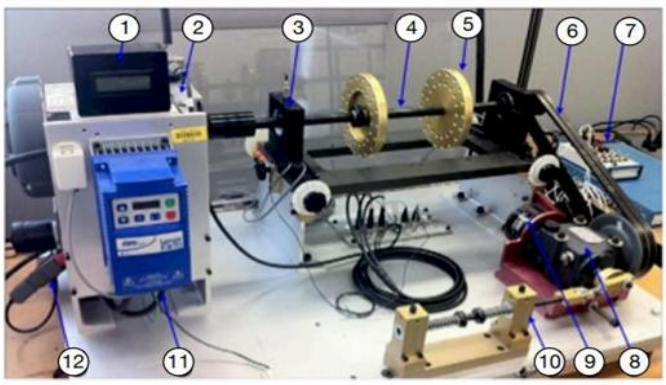


Fig. 4: Experimental facility: (1) Opera meter, (2) Induction Motor, (3) Bearing, (4) Shaft, (5) Loading Disc, (6) Driving Belt, (7) Data Acquisition Board, (8) Bevel gearbox, (9) Magnetic Load, (10) Reciprocating Mechanism, (11) Variable Speed Controller, (12) Current Probe[23]

Figure 2.6.1 Experimental Facility

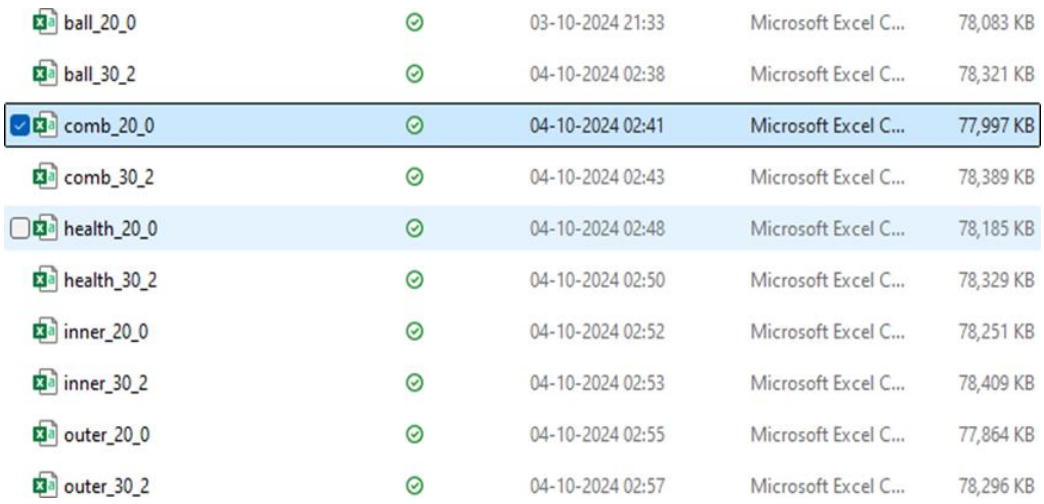


Figure 2.6.2 Types of faults

We have datasets having different types of faults at different positions like rotating speed and load configurations. The dataset includes the reading at regular intervals at 8 channels, where 2-3-4 channels are more effective than others. Every data set consists of 10 lakh datapoints. We tried with a similar dataset but having limited trends in the data set moved to get some trends by making the data in reduced form and having some trends.

Channel1	Channel2	Channel3	Channel4	Channel5	Channel6	Channel7	Channel8
-0.13194	-0.00368	-0.00789	-0.006	-0.00914	0.004551	0.000236	0.000334
-0.13488	-0.00016	0.003791	0.004156	-0.00258	0.006244	-0.00222	-0.00711
-0.13619	-4.6E-05	-0.00213	-0.00061	0.002059	0.000831	0.000814	0.000023
-0.13413	-0.0002	-0.00158	-0.00377	-0.00158	-0.00123	-0.00016	0.004441
-0.13619	0.001631	0.002419	0.004975	-0.00682	-0.00707	-0.00296	0.00192
-0.13544	0.000311	-0.0018	0.000144	-0.00671	-0.00231	0.004143	0.003999
-0.13892	0.001194	-0.00313	0.000919	-0.00223	-0.00263	0.009035	0.004216
-0.13516	0.000719	0.001297	0.004333	0.001569	0.001669	-0.00298	-0.00251
-0.13378	0.001369	-0.00057	-0.00043	0.000844	-0.00046	-0.00219	-0.00264
-0.13165	-0.00135	-0.00359	0.001022	-0.0017	0.000107	-0.00153	0.000498
-0.13746	-0.00076	0.006037	0.002369	-0.00321	0.003592	-0.00117	-0.00183
-0.13651	-0.00185	-0.00295	0.003225	-0.00139	-0.00457	0.001723	0.002866
-0.13773	-0.00024	0.001947	0.001513	0.001032	-0.00563	-0.00041	-0.0006
-0.13313	-9.8E-05	0.001006	0.005018	0.000765	-0.0031	0.005274	-0.00261
-0.13061	-0.00247	-0.00444	0.003659	-0.00158	0.000374	0.001378	0.007385
-0.12772	0.000002	0.005083	0.001148	-0.00296	-0.00562	-0.00408	-0.0007
-0.13097	-0.00094	-0.00098	0.005295	0.000701	-0.00495	-0.00074	-0.00232
-0.13268	-0.00184	0.002713	0.003188	0.005684	0.002811	0.004007	-0.00278

Figure 2.6.3 Channels

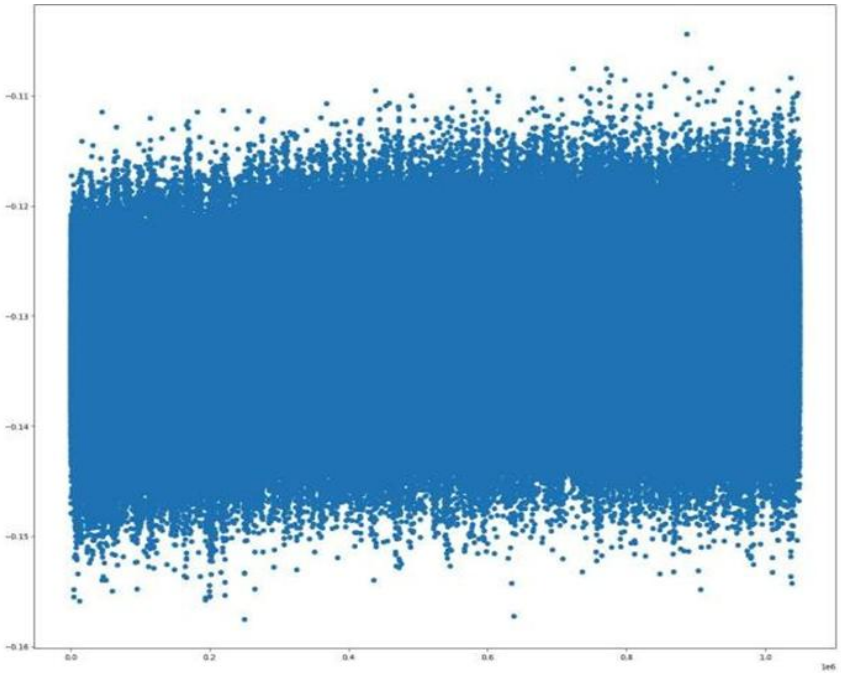


Figure 2.6.4 Aggregate Sampling

	Channel1	Channel2	Channel3	Channel4	Channel5	Channel6	Channel7	Channel8
0	-0,133514	0,000736	0,000541	0,001154	0,000211	-0,000202	0,000042	-0,000318
1	-0,133639	0,000769	0,000526	0,001120	0,000239	-0,000119	0,000034	-0,000327
2	-0,133574	0,000771	0,000583	0,001113	0,000151	-0,000093	0,000007	-0,000331
3	-0,133659	0,000830	0,000642	0,001118	0,000601	-0,000090	0,000038	-0,000364
4	-0,133534	0,000810	0,000647	0,001087	-0,001103	-0,000069	0,000029	-0,000409
...	...	...	...	...	...	...	...	...
995	-0,130567	0,000909	0,000676	0,001179	0,000325	-0,000093	0,000240	-0,000249
996	-0,130431	0,000927	0,000579	0,001164	0,001273	-0,000099	0,000224	-0,000329
997	-0,130513	0,000935	0,000559	0,001172	0,000230	-0,000078	0,000247	-0,000373
998	-0,130461	0,000919	0,000514	0,001157	-0,000085	-0,000073	0,000221	-0,000386
999	-0,130478	0,000928	0,000502	0,001146	0,000110	-0,000142	0,000224	-0,000418

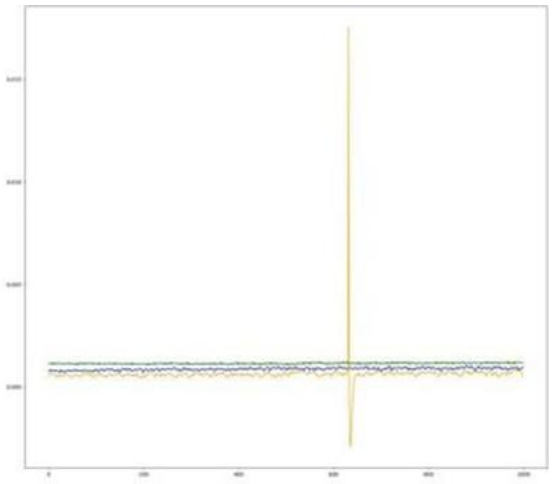


Figure 2.6.5 Down-Sampling

	Channel1	Channel2	Channel3	Channel4	Channel5	Channel6	Channel7	Channel8
0	-0,131940	-0,003675	-0,007886	-0,006003	-0,009137	0,004551	0,000236	0,000334
1048	-0,136319	0,001003	-0,000286	0,000898	-0,001266	-0,005158	0,003089	-0,004870
2096	-0,126244	-0,000573	-0,001557	-0,001227	-0,000763	-0,007302	0,003016	-0,002367
3144	-0,133075	0,003355	0,000867	-0,000536	0,001734	-0,000286	0,000583	-0,005828
4192	-0,124910	0,005053	0,000722	-0,002459	-0,003073	-0,008878	0,006516	0,009226
...	...	...	...	...	...	...	...	...
1043808	-0,127821	-0,004078	-0,004227	-0,003154	0,001473	0,011333	-0,001100	0,009478
1044856	-0,132723	-0,000443	-0,003676	0,000520	0,004350	-0,001841	0,006101	0,001496
1045904	-0,128347	0,003923	0,000492	0,001842	-0,003946	-0,001619	0,001708	-0,003788
1046952	-0,128410	0,001776	0,001936	0,018460	-0,004187	0,001489	0,016226	-0,004568
1048000	-0,130985	0,002214	0,003501	0,001988	-0,006564	-0,001578	0,005418	-0,010881

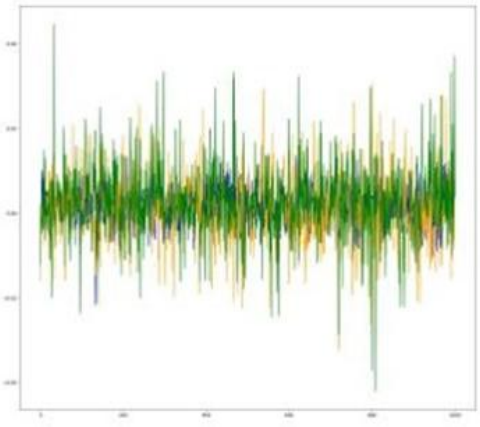


Figure 2.6.6 Random Sampling

2.7 Summary of Literature Review

Literature review offers some of the strategies for fault monitoring and detection in mechanical and industrial systems, stressing the significance of multi-sensor data acquisition and sophisticated analytics. One of the major studies built a low-cost data acquisition system based on vibration, current, and sound sensors for Fused Deposition Modeling (FDM) 3D printing fault detection. This approach employs machine learning algorithms such as convolutional neural networks (CNNs) for real-time classification of faults, which underlines the importance of sensor integration for correct fault diagnosis.

A second significant contribution is the investigation of hybrid fault detection and diagnosis (FDD) systems that integrate data-driven, physics-based, and knowledge-based techniques. Hybrid techniques endeavor to overcome the limitations of every method by improving the accuracy and reliability of fault detection, diagnosis, and correction. The research classifies such hybrid approaches as serial, parallel, and blended hybrids, combining multiple methods to enhance decision-making processes. The article also addresses how sophisticated hybrid systems may deal with intricate faults in industrial settings, advancing the idea that combining these techniques results in more solid and scalable solutions.

Also, the literature presents the significance of data acquisition and feature extraction in monitoring of intricate systems. An example study addresses drilling process monitoring through the utilization of sensors to measure vibrations, acoustic emissions, and cutting forces, which are analyzed for forecasting tool wear and surface roughness. With the optimization of

feature selection and application of machine learning algorithms in analysis, the study enhances monitoring system accuracy and efficiency.

There is also a role of Digital Twin (DT) technology highlighted across a number of studies. DTs, making virtual copies of physical assets, are being adopted across many fields, such as aerospace and production. The union of IoT and machine learning on DT platforms can provide real-time diagnostics, prognostics, and predictive maintenance. This enables better system dependability and better operation efficiency, especially in markets like aviation and energy, wherein downtime and upkeep costs can run high.

Lastly, the principle of acoustic diagnosis, especially in high-voltage circuit breakers, is addressed. Acoustic signatures generated during circuit breaker switching operations can be analyzed to identify early indications of wear or mechanical defects, providing a non-invasive and accurate diagnostic technique. This study emphasizes the possibility of integrating acoustic analysis with other diagnostic equipment to determine the health of key electrical infrastructure.

These studies collectively show the increasing importance of advanced data capture systems, machine learning, and digital twin technology in predictive maintenance and fault detection. Their use across a wide range of disciplines from manufacturing, energy, and aerospace highlights their potential to enhance system dependability and reduce unplanned downtime.

The final part of the literature review links these developments to the introduction of the research paper by reaffirming the point that the combination of machine learning models and sensor-based monitoring can significantly improve fault detection in intricate systems. In particular, it points to the importance of the use of such technologies to Air Circuit Breakers (ACB) monitoring, where patterns in vibration are key fault indicators, especially among the critical spring assemblies. Having the capability to exploit a digital twin of the ACB based on real-time sensor information and machine learning models sits well within the larger pattern of employing predictive maintenance to minimize system downtime as well as enhance its performance. This method not only improves fault detection but also opens the door to more effective maintenance approaches in industrial applications, including those utilized in electrical grids.

CHAPTER 3: MODEL DEVELOPMENT

In this chapter, an extensive description is presented of our design and detail implementation plan for our multi-sensor system specifically designed for smart detection of faults within circuit breakers. The supreme goal of this critical model building phase is to develop a highly resilient, incredibly cost-efficient, and naturally user-friendly platform that not only has the capability to accurately capture a diverse variety of multi-modal sensory data radiating from a circuit breaker during its normal functional cycles but also of effectively pre-processing and dependably logging this precious information. The voluminous data created in this manner will then become the indispensable raw input for the construction, thorough training, and exhaustive validation of a high-level machine learning model. The advanced analytical model will then be charged with the critical task of facilitating the early, precise, and reliable detection of a range of possible fault conditions that can occur during the operation of the circuit breaker.

3.1 System Design

The grand design of our intelligent fault detection system has been masterminded strategically and painstakingly designed as an extremely modular, inherently adaptive, and astoundingly resilient platform. The platform consists of a sequence of sharply delineated but perfectly interconnected phases, with each serving to deliver a decisive component of the overall goals of the project. These critical steps include: the primary and accurate perception of the pertinent physical phenomena displayed by the circuit breaker in actual operation; the correct collection and optimal pre-processing of the uncooked sensory data from the sensors; the safe local logging and secure storage of this vital information for future analysis; and a vital provision and clearly defined route for the eventual incorporation of sophisticated machine learning algorithms specifically engineered for the intelligent processing of the gathered data and the proper prediction of potential fault states. At the very heart of this advanced system is the Arduino Nano 33 IoT microcontroller, which has been optimally chosen to act as the master command, smart control, and high-performance real-time information aggregation center. This microcontroller coordinates the intricate communication between the different networked hardware parts and carefully maintains the non-stop stream of varied sensor data throughout the entire system.

- DFRobot Serial 6-Axis Accelerometer (SEN0386): Clearly showing its TTL Serial (UART) communication interface and the special, high-quality multi-core wire

connection made with the specific UART pins (TX and RX) of the Arduino Nano 33 IoT.

- DFRobot Gravity: Analog Sound Level Meter: Demonstrating clearly its Analog Output signal and the purpose-built, shielded multi-core wire connection carefully routed to a particular Analog Input pin (for example, A0) of the Arduino Nano 33 IoT, highlighting how critical it is to reduce noise in the transmission of the analog signal.

- Arduino Nano 33 IoT: Clearly marking all applicable and used interface pins: the particular Analog Input pin(s) it is wired to from the sound sensor's output, the specific Digital Input/Output pins used for interfacing with the user interface components (push button and LED indicator), the dedicated UART pins (TX and RX) wired to the accelerometer, the Serial Peripheral Interface (SPI) pins (MOSI, MISO, SCK, CS) wired to the Micro SD Card Module, the onboard Wi-Fi module (clearly showing its future-proof potential for remote over-the-air updates and remote data transfer), and the dedicated Power Input pins (VIN and GND) for the external power supply.

- Micro SD Card Module: Clearly demonstrating its SPI communication interface with the corresponding SPI pins of the Arduino Nano 33 IoT, along with the physical Micro SD card itself clearly shown as the long-term and reliable data storage device for the obtained sensor data.

- Power Supply Subsystem: Fully documenting both the external DC Power Supply module (specially identifying its voltage and current ratings: 5V 3A and 9V 2A) plugged into the Arduino Nano 33 IoT VIN and GND pins, and the alternative Micro USB connector, specifically indicating its main usage for initial programming and possible low-power operation in development. If any external voltage regulation circuit is used, it should also be specifically drawn and identified within this subsystem.

- User Interface Components: The Push Button, carefully wired to a particular Digital Input pin of the Arduino Nano 33 IoT and clearly demonstrating the addition of a proper pull-up or pull-down resistor in order to present a solid digital input signal. The LED Indicator, wired to a specific Digital Output pin of the Arduino Nano 33 IoT, with a very important current-limiting resistor added to the circuit to avoid damaging both the LED and the output pin of the microcontroller.

3.1.1 Design

The underlying and well-thought-out reason behind this carefully designed modular structure is to provide a high level of inherent flexibility, smooth scalability for possible future upgrades and expansions, and increased ease of maintenance, effective debugging, and easy troubleshooting. The Arduino Nano 33 IoT has been judiciously chosen as the main processing unit because of its ideal blend of incredibly small physical size, far more than adequate real-time processing power necessary for effectively processing the constant streams of incoming sensor data, and the highly beneficial built-in Wi-Fi connectivity. Although the first operational phase of the project emphasizes heavily on strong and reliable local data logging to the micro SD card, maintaining data integrity and system independence, the onboard Wi-Fi module offers a clear and easily accessible avenue for future seamless enhancements, including the ability to enable real-time remote monitoring of the circuit breaker's operational status from a geographically remote location, enabling convenient over-the-air firmware updates for the embedded data acquisition system, and enabling direct and efficient data transmission to external cloud-based platforms for more advanced, resource-hungry analysis and advanced data visualization methods. The conscious design choice to physically isolate the critical data gathering and local logging functions from the rest, potentially substantially more computationally demanding machine learning-based fault analysis is central to the design of the system. This decoupling of strategy enables the sophisticated machine learning algorithms to be developed independently, thoroughly trained on the data gathered, and finally deployed on a different computing platform with more processing capabilities (e.g., a desktop computer or a cloud server), without subjecting the embedded Arduino microcontroller system to any real-time computational loads or performance constraints, which is mainly optimized for effective data acquisition and logging. In addition, the intrinsic modularity of the design of the system elegantly enables the eventual inclusion of further sensor modalities, e.g., current transformers for measuring the electrical current through the breaker, voltage transducers for sensing the voltage across the contacts, or even temperature sensors for monitoring thermal states, if preliminary thorough analysis of the data gathered by vibration and acoustic measurements shows a strong reason for more exhaustive and multi-faceted awareness of the circuit breaker's complex operational condition and the involved processes of potential fault formation.

3.1.2 Sensor Selection Matrix

Our systematic and rigorous sensor selection procedure entailed a thorough and comparative

analysis of a broad range of commercially available and Arduino-compatible sensors that have the potential to accurately and reliably measure the key physical phenomena directly representative of the operational status and the wide range of possible fault conditions that may occur within a circuit breaker throughout its operational life cycle.

The most important technical parameters that carefully directed our selection of sensors included, but were not restricted to:

1. Measurement Range : The definite maximum and minimum values that can be measured with precision and reliability by the sensor for the object physical quantity. For the accelerometer, this would be the measurable range of acceleration (usually quoted in g-forces or m/s2), and for the sound level meter, it would be the range of measurable sound pressure levels (usually quoted in decibels, dB). The selected range of measurement for every sensor should be wide enough to cover the anticipated fluctuations in these physical parameters not only under normal, healthy circuit breaker operation but also under a range of possible fault conditions, when the amplitudes of vibration and sound signals may drastically differ from their nominal values.

2. Sensitivity : The minimum detectable alteration in the measured physical parameter that the sensor can detect and reliably convert into a corresponding and quantifiable alteration in its output signal. Increased sensitivity is paramount for the detection of slight anomalies in the vibration and sound signatures, which may be the first signs of gradually unfolding or incipient faults in the mechanical or electrical equipment of the circuit breaker. The specification for sensitivity actually determines the resolution at which the sensor can detect minuscule differences in the physical phenomena under observation.

3. Frequency Response : The particular range of frequencies within which the sensor measures the physical quantity to a satisfactory degree of accuracy and reliability without substantial attenuation or distortion of the amplitude and phase of the signal. This parameter is especially important in order to effectively record the transient and frequently high-rate changing signals that are inherently linked with the operations of a circuit breaker, e.g., the high-frequency mechanical vibrations induced during fast contact closure or opening, and the rich frequency content of the acoustic emissions generated due to electrical arcing or mechanical impacts within the breaker mechanism.

A broad enough and proper frequency response guarantees that we do not miss important information contained in the time-varying character of the signals.

4. Interface Compatibility : The exact nature of electrical interface used by the sensor to successfully transfer its sensed readings to the central microcontroller unit (the Arduino Nano 33 IoT in our case). Common interface forms are Analog Voltage Output (in which the value being measured is indicated by an analog voltage level), Digital Interfaces like Inter-Integrated Circuit (I2C) and Serial Peripheral Interface (SPI), and serial communication interfaces like TTL Serial (also referred to as UART). Direct and smooth compatibility with the Arduino Nano 33 IoT's input/output interfaces that are available is a strictly essential prerequisite for easy hardware integration and optimal data collection.

5. Power Requirements : The actual voltage and current levels needed to correctly and reliably drive the sensor. These power requirements need to be carefully planned so that they are within the safe and sustainable power supply ranges of the onboard voltage regulators and output pins of the Arduino Nano 33 IoT. Going over these limits may cause sensor malfunction or even irreparable damage to the microcontroller.

6. Physical Size and Mounting Factors : The actual physical size, overall form factor, and available mounting points of the sensor are significant practical factors for effective physical integration of the sensor into the potentially space-restricted and mechanically complicated environment of the Air Circuit Breaker (ACB) under monitoring. The size and mounting structures of the sensor should permit firm and stable fixation at the desired points of monitoring (e.g., the rotary shaft for vibration, beside the contacts for sound) without causing extraneous vibrations or interfering with the proper functioning of the circuit breaker.

7. Cost-Effectiveness : For the system development and initial prototyping stage, the unit cost of the chosen sensors is an important economic factor. There needs to be a delicate balance between meeting the required levels of performance and being within the project budget. Using very costly sensors may compromise the scalability and wider applicability of the end system.

8. Availability of Libraries and Extensive Documentation : The easy access to good-quality and easy-to-use Arduino libraries tailored specifically for interfacing with the

selected sensors, and extensive and well-explained technical datasheets by the sensor supplier, can greatly ease the software development and hardware integration process. Accurate and detailed documentation saves development time, lowers likely integration faults, and allows easier debugging.

3.1.2.1 Matrix

Feature	DFRobot Serial 6-Axis Accelerometer	Accelerometer 1: ADXL335	Accelerometer 2: MPU6050	DFRobot Gravity: Analog Sound Level Meter	LM393 Sound Sensor
Acceleration Range	±2/4/8/16 g (configurable)	±3 g	±2/4/8/16 g	N/A	N/A
Sensitivity	(Not directly specified, but high precision)	(Not directly specified)	(Not directly specified)	(Linear Analog Output)	(Not directly specified)
Frequency Response	0.1Hz ~ 200Hz	50Hz	(Not directly specified)	31.5Hz ~ 8.5KHz	(Not directly specified)
Interface	I2C	Analog	I2C	Analog	(Not directly specified)
Power	3.3V-5V, <40mA	3-5V	2.3V-3.4V	3.3~5.0V	(Not directly specified)
Size	51.3 x 36 x 10mm	(Small, not precisely specified)	(Small, not precisely specified)	60mm * 43mm	(Not directly specified)
Cost	$25	$10	$15	$40	$10

Figure 3.1.1 Components Matrix

3.1.3 Casing And Sensor Mounting

3.1.3.1 Casing

In order to guarantee the stable performance, physical safeguarding, and electromagnetic shielding of the sensitive electronic components that form our multi-sensor fault detection system, a specially designed and accurately manufactured housing has been developed using advanced 3D printing technology. This new rapid prototyping method has the unique benefit of allowing for the construction of an enclosure that is individually and optimally designed to the particular physical needs and complex internal structure of our system. Polylactic Acid (PLA) filament is the main material used for the 3D printing process. This renewable resource-derived thermoplastic polyester was chosen for a number of important reasons: its native ease of printability with typical 3D printing settings (generally requiring a nozzle temperature range of 190-220°C and a heated bed temperature range of 40-70°C for optimal adhesion and layer

development), its relatively high degree of rigidity and structural integrity, and, most importantly, its native non-conductive electrical properties. The non-conductive character of PLA is an essential safety factor since it reduces the possibility of unforeseen electrical shorts or current leakage from the internal circuitry to the external world, hence dramatically decreasing the possibility of electrical hazards and protecting personnel that handle the system. In addition, a non-conductive enclosure aids in reducing the possibility of electromagnetic interference (EMI) coming from outside the device or internally from within the circuit breaker that could directly influence the sensitive electronics and sensor outputs. Although regular PLA is typically an electrical insulator, careful attention will be given throughout the internal wiring and component installation phases to proper insulation of all electrical connections as well as the physical separation of high-voltage or high-current carrying components (if any are included in subsequent versions).

The general shape and physical form factor of the 3D-printed casing is a small cuboid. This shape was selected in particular to accomplish the essential goal of reducing the overall spatial footprint of the monitoring system, thus increasing its ability to integrate easily within the usually space-limited area normally encountered surrounding Air Circuit Breakers (ACBs) and related electrical equipment. The interior configuration and internal structural makeup of the cuboid casing have been carefully designed to safely hold and rigidly support the following necessary electronic elements in a highly organized and secured manner:

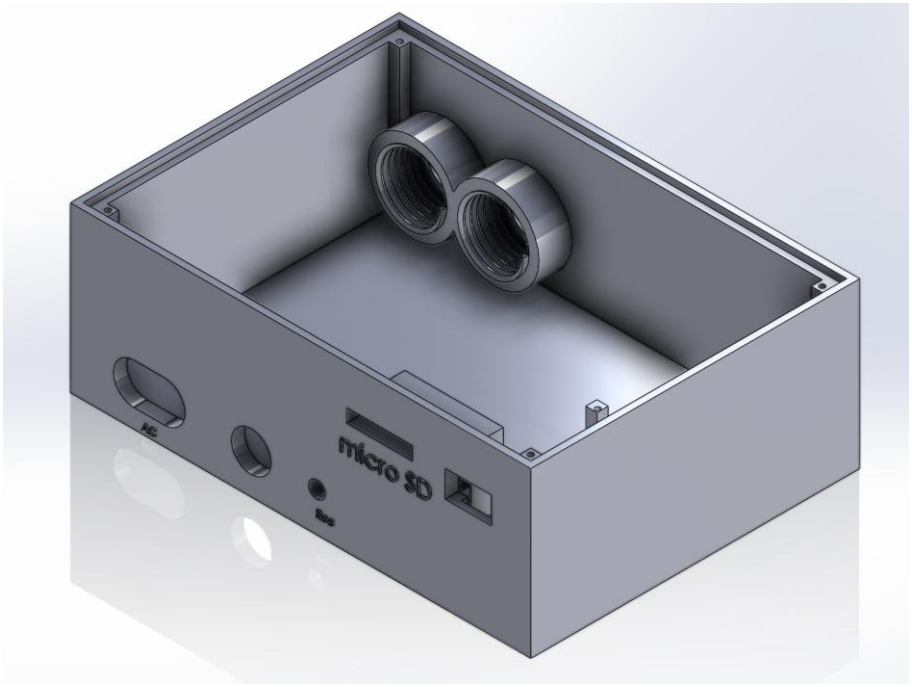


Figure 3.1.2 Casing or Enclosure

- The Arduino Nano 33 IoT microcontroller board: Dedicated internal mounting points, likely in the form of small raised platforms with screw holes or precisely dimensioned slots, will be incorporated into the 3D-printed design to ensure the secure and vibration-resistant attachment of the Arduino board.

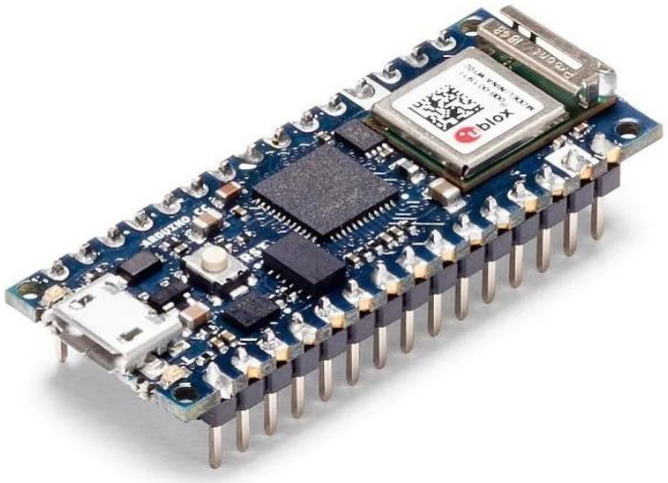


Figure 3.1.3 Arduino Nano 33IoT

- The Power Supply Unit (and associated voltage regulation circuitry, if external): A designated internal compartment or secure mounting area will be allocated for the power supply components, ensuring they are physically isolated from other sensitive electronics to minimize potential thermal or electromagnetic interference.

- The Micro SD Card Module / Data Logger: A dedicated mounting structure, potentially involving small clips or a precisely fitted slot, will be designed to securely hold the Micro SD Card Module in place. Easy access for the insertion and removal of the Micro SD card will be a key design consideration.

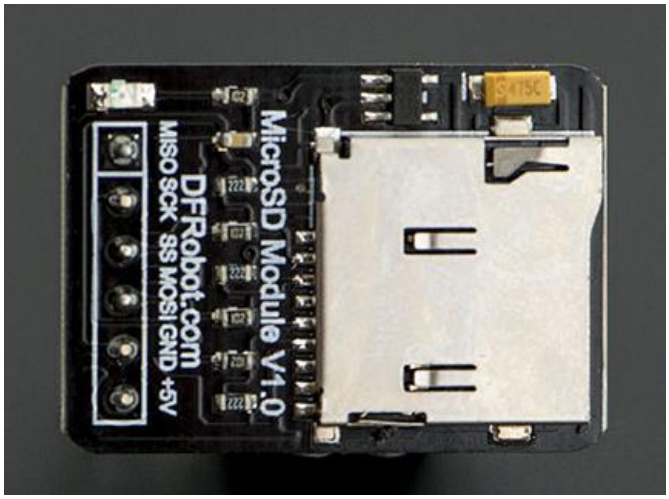


Figure 3.1.4 SD Card Module

- Female Pin-Out Quick Connectors for External Sensor Connections: To facilitate the effortless and reliable connection of the external vibration and sound sensors to the internal electronics of the monitoring system, two robust female pin-out quick connectors will be securely mounted on the exterior surfaces of the 3D-printed casing. These connectors will be affixed using two integrated threaded holes designed into the casing structure, ensuring a strong and vibration-resistant mechanical connection. These quick connectors will be specifically 4-pin (or 4-core compatible) to accommodate the signal and power requirements of the sensors, and they will utilize a standard male-female socket configuration for ease of use.



Figure 3.1.5 Quick Connector Male/Female

- AC Power Supply Input (if applicable in future iterations): If the system is intended to be powered directly from an AC mains source in future versions, a dedicated and appropriately insulated slot or connector will be integrated into the casing design to safely accommodate the AC power input. [Dedicated slot in the 3D model and physical case for AC power input:



Figure 3.1.6 AC Power Supply Input

- Physical Push Button for User Control: A precisely dimensioned hole will be incorporated into an easily accessible location on the casing to accommodate the physical push button. This button will serve as the primary user interface element for initiating and terminating the data recording process.

- LED Indicator for System Status: A small, appropriately sized hole will be designed into the casing, strategically positioned for clear visibility, to house the status LED indicator. This LED will provide immediate visual feedback to the user regarding the current operational state of the monitoring system (e.g., system powered on, data recording in progress, error condition). [Dedicated hole in the 3D model and physical case for the status LED: [Image placeholder for LED hole]]

- Micro SD Card Access Slot: A conveniently located slot will be designed into the casing to allow for easy and unobstructed insertion and removal of the micro SD card without the need to disassemble the entire enclosure.

- The internal structural design of the 3D-printed casing will incorporate four dedicated mounting points (e.g., small standoffs with screw holes) specifically designed to securely affix the Arduino Nano 33 IoT microcontroller board, preventing any unwanted movement or vibration that could potentially lead to electrical shorts or intermittent connection issues. Furthermore, a dedicated mounting structure, potentially involving small clips or a precisely dimensioned cradle, along with a secure mechanical link, will be integrated to ensure the stable and reliable placement of the Micro SD card module. As previously mentioned, two robust threaded holes will be strategically located on the exterior of the casing to facilitate the secure and vibration-resistant mounting of the female pin-out quick connectors, providing a durable and reliable interface for the external sensors. A dedicated slot will be precisely dimensioned to accommodate the AC power supply input (if applicable), ensuring a secure and insulated connection point. Additionally, precisely sized and located holes will be incorporated to accommodate the physical push button and the LED indicator, ensuring their proper mechanical mounting and clear visibility. Finally, a designated access slot will be designed to provide convenient and unobstructed access to the micro SD card for data retrieval and management.

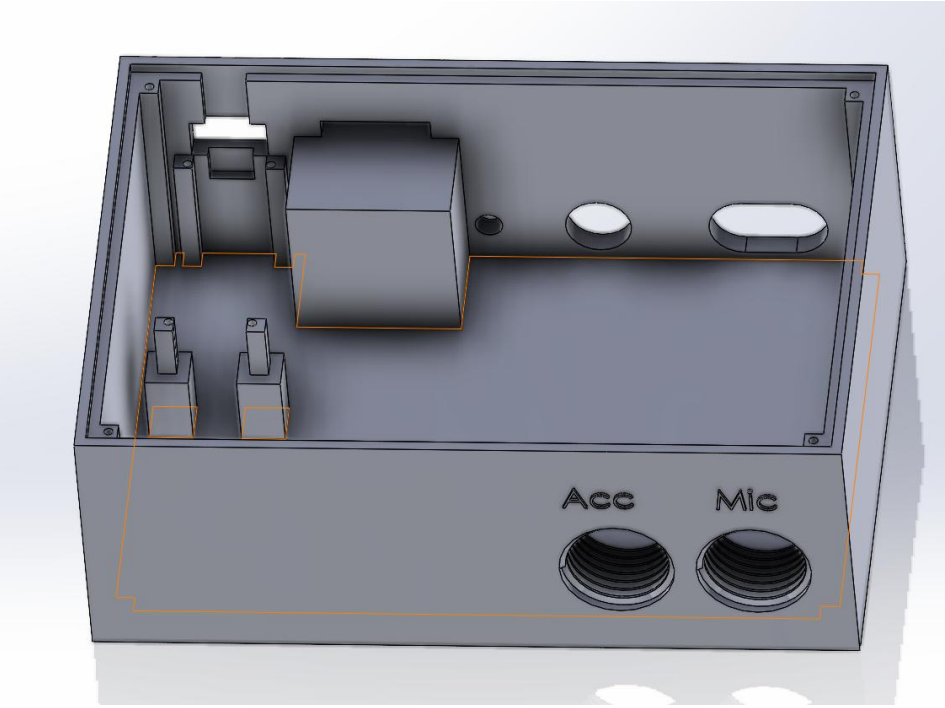


Figure 3.1.7 Enclosure

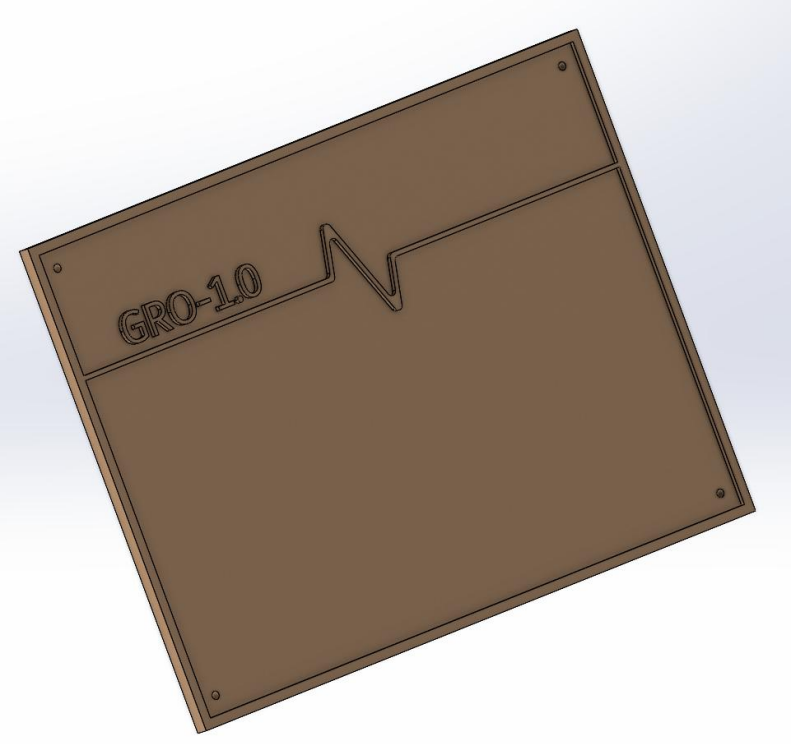


Figure 3.1.8 Case Lid

For enhanced environmental protection and increased system durability, the external quick

connectors will be specified to include an integrated water ring or a similar sealing mechanism [Illustration of a quick connector with a water ring or seal: [Image placeholder for water ring on quick connector]]. This feature is crucial for preventing the ingress of moisture, dust, or other environmental contaminants into the enclosure through the sensor connection points, thereby safeguarding the internal electronic components and ensuring the long-term reliability of the monitoring system, especially in potentially harsh industrial environments.

3.1.3.2 Sensor Mounting

In order to achieve the flexibility required in the placement of the sensors at the best data collection points on the circuit breaker model, the sensors themselves will not be incorporated into the main 3D-printed enclosure. Rather, they will be joined to the main enclosure through properly extended wires. The sensor unit itself, which has been carefully chosen for its appropriateness to the particular mounting location and physical phenomenon being sensed, will occupy one end of each of the wires. The remainder of each sensor wire will have a high-quality male quick connector attached to it. The male connectors themselves will be plugged firmly into identical female quick connectors rigidly mounted on the outside of the 3D-printed enclosure. This connection technique provides both a strong electrical connection for the transmission of power and the transmission of signals as well as a measure of mechanical isolation between the sensors and the main electronics enclosure, which could minimize unwanted vibration or shock transmission to sensitive internal parts. For the accelerometer, a 4-core wire shall be used to transfer the required power supply wires and the Transistor-Transistor Logic (TTL) serial communication signals (Receive and Transmit) between the sensor and Arduino. For the sound level meter, a 3-core wire will be used to transmit the power supply lines and the analog voltage output signal from the sensor to the Analog Input pin of the Arduino, with the shielding being essential for reducing electromagnetic interference and maintaining the integrity of the sensitive analog signal.

Vibration Sensor (DFRobot Serial 6-Axis Accelerometer - SKU:SEN0386) Mounting

1. Exact Point of Mounting: The DFRobot Serial 6-Axis Accelerometer has been ideally positioned to be mounted on top of the ACB's rotary shaft. The rotary shaft represents an important mechanical component of the mechanism of operation of the ACB, and the vibrations so generated are bound to be greatly representative of the mechanical condition in general and also operational dynamics of the breaker in question.

2. Detailed Mounting Technique: The sensor unit itself has two accurately designed slit slots built into its physical casing. These slots are designed to allow for a direct and secure mechanical mounting onto the rotary shaft. Our preferred mounting technique is the use of a specially designed coupler or a similar strong mechanical mounting mechanism (e.g., a small clamp with vibration-dampening material). This coupler will be accurately machined to fit solidly onto the rotary shaft and will have complementary mounting points (e.g., screws or tight-fitting pins) that directly align with the slit slots of the accelerometer housing. The coupler will be made of a material that has good mechanical stiffness and possibly some vibration-dampening characteristics to effectively transmit the vibrations of the breaker to the sensor with as little extraneous noise or resonance influence as possible. The coupler mechanical design and method of attachment will be aimed at having a stiff and direct link to the rotating member in order to have the most accurate and loyal reproduction of the vibrations developed during the critical opening and closing action of the breaker.



Figure 3.1.9 6- Axis Accelerometer

3. Rigorous Reasoning: Placing the accelerometer on the rotary shaft directly is a conscious and thoroughly justified choice considering the fact that the vibrations created by the mechanical motion of the operating mechanism of the circuit breaker during its opening and closing process will be most directly and forcefully expressed at this rotating element. By observing the vibration signatures at this critical point, we seek to acquire dense and very informative data that is immediately related to the mechanical condition and performance of the internal components of the breaker. Any loss of

condition, wear, misalignment, or loosening of important mechanical components in the operating mechanism is extremely likely to appear as measurable changes in the characteristic vibration patterns indicated at the rotary shaft. Hence, this location of mounting is selected optimally to make the sensitivity of our vibration measurement maximum to detect early occurrence and growth of such mechanical defects.

Sound Sensor (DFRobot Gravity: Analog Sound Level Meter) Mounting

1. Accurate Mounting Position: The DFRobot Gravity: Analog Sound Level Meter has been specified to be mounted in close physical proximity to the breaker circuit's electrical contacts. Preferably, the sensor will be installed inside the breaker's enclosure itself if there is enough room and appropriate mounting surfaces. If mounting internally is not possible for reasons of space or accessibility issues, the sensor will be mounted as close as is reasonable to the contact area on the outside of the housing of the breaker. The intent is to make the distance between the source of sound (the contacts) and the microphone of the sensor as small as possible to optimize for picking up the acoustic emissions that occur during operation.

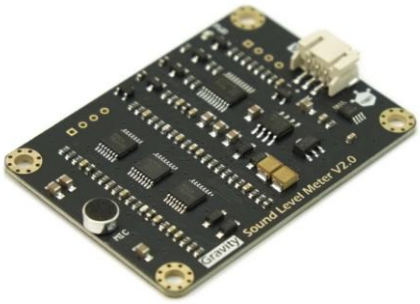


Figure 3.1.10 Analog Sound Level Meter

1. Detailed Mounting Method: The specific method employed for mounting the sound level meter will be contingent upon the available surfaces and the limited space in the vicinity of the contacts. Potential mounting methods include the use of high-strength adhesive pads with good acoustic transparency (to avoid dampening the sound waves), small custom-fabricated brackets that can be securely attached to existing mounting points on the breaker or its enclosure, or even robust cable ties used to firmly secure the sensor to a stable structural element near the contacts. Regardless of the chosen method, the primary objective is to ensure that the sensor remains rigidly fixed in its designated

position and does not introduce any extraneous vibrations or unwanted mechanical noise into the acoustic measurements. The mounting must also ensure that the sensor's microphone is oriented in a direction that optimally captures the sound waves emanating from the contact area.

2. Detailed Rationale: Placing the sound sensor in close proximity to the electrical contacts is absolutely crucial for effectively capturing the acoustic signals that are generated during the circuit breaker's critical operational events, particularly during the rapid opening and closing sequences where electrical arcing across the separating contacts or mechanical impacts between moving parts can produce distinct and characteristic sound patterns. Changes in the overall sound pressure level, alterations in the frequency content of the acoustic emissions, or the emergence of unusual and unexpected noises can serve as valuable indicators of underlying electrical faults, such as the presence of excessive or prolonged arcing due to contact degradation or contamination, as well as mechanical issues within the contact mechanism itself, such as contact bounce, misalignment, or excessive impact forces. By strategically positioning the sound sensor to capture these acoustic signatures, we aim to obtain complementary data that, when analyzed in conjunction with the vibration data, will provide a more comprehensive and insightful assessment of the circuit breaker's overall health and the nature of any developing faults.

3.1.4 Data Acquisition System

The data acquisition system forms the very core of our multi-sensor fault detection platform, serving as the critical interface between the physical world (the circuit breaker and its associated phenomena) and the digital realm where data processing, analysis, and eventual fault diagnosis will occur. The primary responsibility of this subsystem is to accurately, reliably, and synchronously capture the signals emanating from both the vibration sensor (accelerometer) and the sound sensor (sound level meter) and to convert these raw analog and digital signals into a structured digital format that can be efficiently processed, logged, and subsequently analyzed. The central orchestrator of this intricate data acquisition process is the Arduino Nano 33 IoT microcontroller, which leverages its diverse array of integrated peripherals and carefully developed software to manage the interaction with each sensor and the subsequent handling of the acquired data.

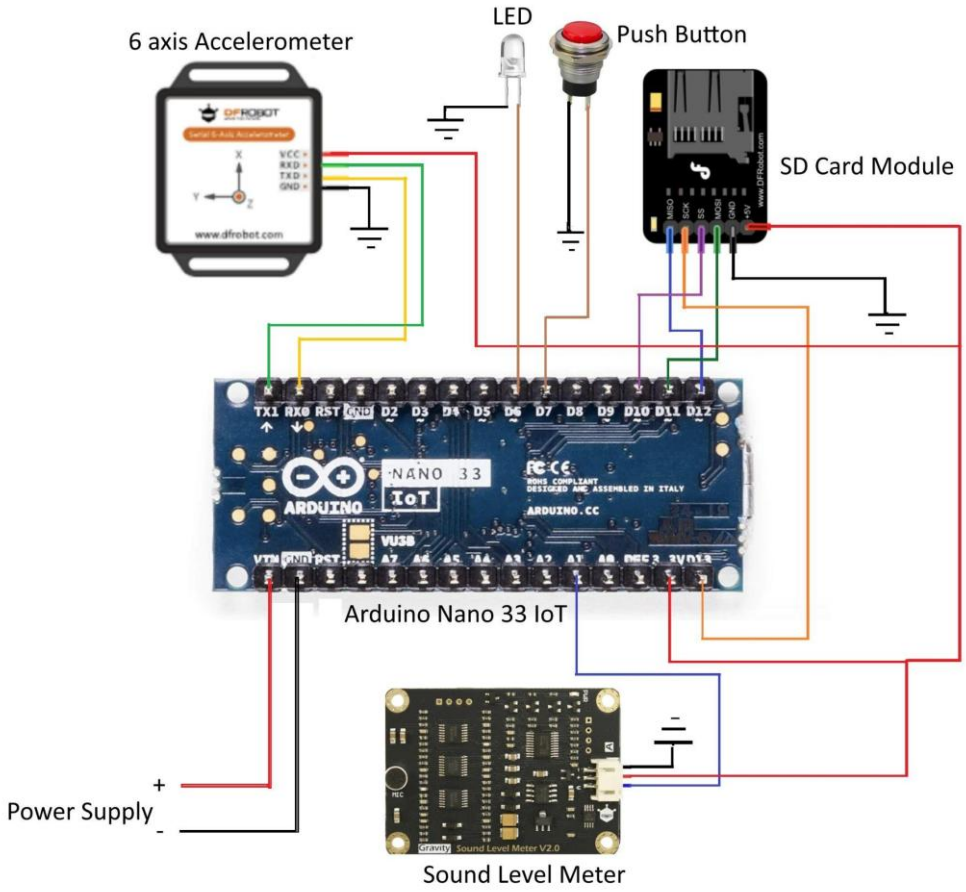


Figure 3.1.11 DAQ Circuit Diagram

3.1.4.1 Acquisition of Sound Data from the DFRobot Gravity

1. Analog-to-Digital Conversion (ADC): The DFRobot Gravity: Analog Sound Level Meter outputs a continuously varying analog voltage signal, where the magnitude of this voltage is directly proportional to the instantaneous sound pressure level (SPL) being detected by the sensor's microphone. To process this analog signal using the digital capabilities of the Arduino Nano 33 IoT, we will utilize the microcontroller's integrated 12-bit Analog-to-Digital Converter (ADC). The Arduino Nano 33 IoT is equipped with a multi-channel ADC that allows it to sample analog voltage levels present at its dedicated analog input pins (typically labeled A0, A1, etc.). We will connect the analog output pin of the sound level meter to one of these analog input pins on the Arduino.

2. ADC Resolution and Range: The 12-bit resolution of the Arduino's ADC means that it

can divide the analog input voltage range (typically 0 to 3.3V on the Arduino Nano 33 IoT) into 2^{12}=4096 discrete digital levels. This provides a reasonably high degree of precision in the conversion process, allowing us to capture subtle variations in the sound pressure level. The analog output range of the DFRobot Gravity: Analog Sound Level Meter will be carefully examined from its datasheet to ensure optimal mapping to the Arduino's ADC input range. If necessary, a simple voltage divider circuit or an operational amplifier-based level shifter might be employed to scale or offset the sound sensor's output voltage to fully utilize the Arduino's ADC input range and maximize the resolution of our measurements.

3. Sampling Rate Determination: The sampling rate, which refers to the number of analog-to-digital conversions performed per second (measured in Hertz, Hz), is a critical parameter that will be carefully determined through experimental testing and consideration of the expected frequency content of the acoustic signals produced by the circuit breaker during its operation. The acoustic events associated with contact closure, arcing (if present), and mechanical impacts are often transient and can contain significant frequency components in the lower to mid-frequency range (as indicated by the sound sensor's frequency response of 31.5Hz - 8.5kHz).

3.1.4.2 Acquisition of Vibration Data from the DFRobot Serial 6-Axis Accelerometer

1. Serial Communication (UART/TTL): The DFRobot Serial 6-Axis Accelerometer communicates its acceleration data for the three orthogonal axes (X, Y, and Z) using a TTL (Transistor-Transistor Logic) level serial communication protocol, also commonly referred to as UART (Universal Asynchronous Receiver/Transmitter). This means that the sensor transmits the acceleration values as a stream of digital data over two dedicated wires: a Transmit (TX) line from the sensor to the Arduino, and a Receive (RX) line from the Arduino to the sensor (although in this data acquisition phase where we are only reading data from the sensor, the RX line might not be actively used).

2. Baud Rate Configuration: Serial communication requires both devices (the Arduino and the accelerometer) to be configured to communicate at the same baud rate, which specifies the number of bits of data transmitted per second. The DFRobot Serial 6-Axis Accelerometer typically supports various baud rates (e.g., 9600, 115200 bits per second). We will need to consult the sensor's datasheet or configuration documentation to determine the default or configurable baud rate and then configure the Arduino's

serial port to match this rate in our software using the Serial.begin(baud_rate) function.

3. Data Packet Structure and Parsing: The accelerometer will transmit the tri-axial acceleration data (and potentially other data like gyroscope readings, although we will primarily focus on acceleration) in a specific data packet format. This format will be detailed in the sensor's communication protocol documentation. It will likely involve a start byte or a preamble, followed by the acceleration values for the X, Y, and Z axes (typically as multi-byte integer values representing the acceleration in g-forces or m/s2), and possibly a checksum or other error detection mechanisms. The Arduino code will need to continuously monitor the serial port for incoming data, identify the start of a valid data packet, read the subsequent bytes corresponding to the acceleration values, and then parse these raw byte sequences into meaningful floating-point or integer representations of the acceleration along each axis. If the DFRobot_WT61PC library is utilized (as previously mentioned), this library will likely provide functions that handle the low-level serial communication and the parsing of the sensor's data packets, significantly simplifying the software development process. If the library is not used, we will need to implement the parsing logic manually based on the sensor's communication protocol.

3.1.4.3 Synchronization of Multi-Modal Data Acquisition

1. Software-Based Synchronization: To enable meaningful correlation and analysis of the simultaneously occurring vibration and acoustic events during the circuit breaker's operation, it is crucial to achieve a degree of temporal synchronization between the data streams acquired from the accelerometer and the sound level meter. In our system, this synchronization will primarily be achieved through software-based techniques within the Arduino code.

2. Triggering Mechanism: The data acquisition process for both sensors will be initiated as closely as possible in time upon the activation of the physical push button by the user. When the Arduino detects a change in the state of the digital input pin connected to the push button, it will immediately begin the process of reading data from both the analog input (sound sensor) and the serial port (accelerometer) within its main program loop.

3.1.4.4 Handling Potential Electrical Interference

Observation of Anomalous Sound Readings: You previously mentioned observing that the

sound level meter was registering readings even when only the accelerometer was connected. This strongly suggests the presence of electrical interference potentially being coupled into the analog signal line of the sound sensor. This interference could be originating from various sources, including the digital switching noise of the Arduino itself, the serial communication signals of the accelerometer, or even external electromagnetic fields.

3.1.5 Power Supply and Circuit

Ensuring a stable, reliable, and appropriately regulated power supply is absolutely critical for the consistent and accurate operation of all the electronic components within our multi-sensor fault detection system. This subsystem is responsible for providing the necessary electrical energy to the Arduino Nano 33 IoT microcontroller, the DFRobot Serial 6-Axis Accelerometer, the DFRobot Gravity: Analog Sound Level Meter, and the Micro SD Card Module. Careful consideration has been given to the voltage and current requirements of each component to design a power distribution network that meets these needs safely and efficiently.

3.1.5.1 Power Source and Input

1. Primary Power Source: Our system is designed to be primarily powered by an external DC (Direct Current) power supply unit (charger brick). This approach offers several advantages, including isolation from the potentially noisy AC mains supply and the provision of a well-regulated DC voltage.

2. Voltage and Current Rating: We have specified a DC power supply unit with dual voltage output options: 5V (Volts) at a current rating of 3A (Amperes) and 9V at a current rating of 2A. This dual option provides flexibility depending on the specific requirements and potential future expansions of the system. The Arduino Nano 33 IoT can accept a wide input voltage range (typically 5V to 12V) through its VIN (Voltage In) pin. Supplying a voltage within this range allows the Arduino's onboard voltage regulator to step down and provide the necessary 3.3V for its core logic and the 3.3V power rail used by some of the connected peripherals. The higher current rating (3A at 5V or 2A at 9V) ensures that the power supply can comfortably provide sufficient current to all connected components without experiencing voltage drops or overheating, even during periods of peak current consumption (e.g., during sensor initialization or SD card writing).

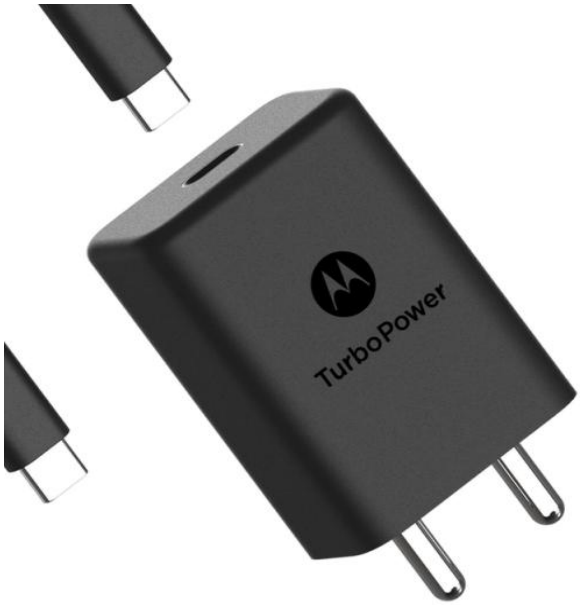


Figure 3.1.12 DC Internal Power Brick

3. Alternative Power Source (for Development): For development purposes, testing, and debugging for the first time, the system can also easily be powered using the Micro USB port of the Arduino Nano 33 IoT. Powered via USB, the Arduino normally runs at 5V and takes power from the USB host (say, a PC). This can be very helpful for bench development and code downloading without the need for the external DC power source. But for independent field operation, the external DC power supply will serve as the primary source of power to provide regular and adequate supply of power.
4. Power Input Connector: The internal DC power source will connect to the system using standard jumper wires that will be incorporated into the 3D-printed housing. These jumper wires will connect directly to the VIN and GND pins on the Arduino Nano 33 IoT, providing a solid and dependable power link. [Sketch of a standard DC barrel jack connector.

3.1.5.2 Voltage Regulation and Distribution

1. Arduino Onboard Regulator: The Arduino Nano 33 IoT has an onboard voltage regulator. When a voltage in the acceptable input voltage range (5V to 12V) is applied to the VIN pin, this regulator effectively reduces the voltage to a steady 3.3V. This 3.3V rail is then utilized to supply power to the core microcontroller of the Arduino and is also provided on specialized 3.3V output pins, which will be utilized to power the DFRobot Serial 6-Axis Accelerometer and the Micro SD Card Module, since these

modules normally run at 3.3V.

2. Current Limiting: Although the power supply unit is adequately rated for current, one must also take into account the current consumption of each component. The Arduino Nano 33 IoT will usually consume a relatively low amount of current (tens of milliamperes). The sensors and the SD card module also have their own current consumption ratings. We will make sure that the sum of the current drawn by all devices connected does not surpass the current sourcing capacity of the Arduino's voltage regulator or the capacity of the external power supply. Where a component may draw more current, we would use external transistors or specialized power distribution ICs to control the flow of current and avoid overloading the Arduino.

3.1.6 Circuit Connections in Detail

The following describes the explicit electrical connections of the Arduino Nano 33 IoT to the range of peripheral devices:

1. DFRobot Serial 6-Axis Accelerometer (SKU:SEN0386):

- VCC (Power): Wired to the 3.3V output pin of the Arduino Nano 33 IoT in order to supply the required working voltage.
- GND (Ground): Wired to one of the GND (Ground) pins of the Arduino Nano 33 IoT to set the common ground reference.
- TXD (Transmit): Wires to the RX (Receive) pin of the Arduino Nano 33 IoT. This transmit line carries the acceleration data that the sensor sends to the Arduino.
- RXD (Receive): Wired to the TX (Transmit) pin of the Arduino Nano 33 IoT. Used if the Arduino must transmit configuration commands or ask for certain data from the sensor (though, during the initial data acquisition stage, this may not be utilized actively).

2. DFRobot Gravity: Analog Sound Level Meter:

- VCC (Power): Linked to either the 3.3V output pin of the Arduino Nano 33 IoT, depending on the particular version of the sound level meter and its suggested operating voltage (to be checked from the datasheet).
- GND (Ground): Linked to one of the GND (Ground) pins of the Arduino Nano 33 IoT.
- OUT (Analog Output): Connected to one of the Analog Input pin (A1) of the Arduino Nano 33 IoT. This pin will receive the analog voltage signal proportional to the sound

pressure level.

3. DFRobot Fermion: MicroSD Card Module for Arduino (Breakout) (SPI Interface):

- VCC (Power): Connected to the 3.3V output pin of the Arduino Nano 33 IoT.
- GND (Ground): Connected to one of the GND (Ground) pins of the Arduino Nano 33 IoT.
- CS (Chip Select): Wired to a Digital Output pin (D10) of Arduino Nano 33 IoT. It's utilized for choosing the SD card module to be communicated.
- MOSI (Master Out Slave In): Wired to the MOSI pin (D11) of the Arduino Nano 33 IoT. This pin is employed for passing data from Arduino to the SD card module.
- MISO (Master In Slave Out): Attached to the MISO pin (D12) of the Arduino Nano 33 IoT. This pin receives data from the SD card module.
- SCK (Serial Clock): Attached to the SCK pin (D13) of the Arduino Nano 33 IoT. This pin supplies the clock signal for SPI communication.

4. Physical Push Button (for Start/Stop Recording):

- One end of the push button connected to a Digital Input pin (D7) of the Arduino Nano 33 IoT.
- The second end of the push button connected to GND (Ground).

5. LED Indicator (for Status while Recording):

- The anode (positive lead) of the LED connected to a Digital Output pin (D6) of the Arduino Nano 33 IoT.
- The cathode (negative lead) of the LED connected to GND (Ground)

3.1.7 All Components

This section offers an extensive and carefully detailed enumeration of all of the critical hardware pieces that have been specially chosen and combined to build our multi-sensor system for the intelligent detection of circuit breaker faults. Each piece has a precise and important function in the final system performance, ranging from sensing the physical phenomena to processing, storage, and giving feedback to the user.

3.1.7.1 Arduino Nano 33 IoT Microcontroller (with Built-in Wi-Fi Module)

a. Description: The Arduino Nano 33 IoT is our system's central processing unit and intelligent control center. It is a highly integrated and powerful development board

based on the Arm Cortex-M0+ 32-bit SAMD21 microcontroller. This microcontroller provides a balance of computing power, minimal power consumption, and an ample set of on-board peripherals such as multiple channels of Analog-to-Digital Converter (ADC) with 12-bit resolution, a Universal Asynchronous Receiver/Transmitter (UART) for serial communication, Serial Peripheral Interface (SPI) and Inter-Integrated Circuit (I2C) communication interfaces for communication with different sensors and modules, and a large number of General Purpose Input/Output (GPIO) pins for the control of LEDs, reading switch presses, and communication with other digital devices.

b. Key Feature: Built-in Wi-Fi Module: One of the most important features of the Arduino Nano 33 IoT that has huge possibilities for future upgrades is its built-in low-power Wi-Fi module (using the ESP32 chip). Although the first part of our project is local data logging to the Micro SD card, with this built-in Wi-Fi feature, there are future applications that can be implemented like real-time remote monitoring of the condition of the circuit breaker through a local network or the internet, OTA firmware updating of the software on the Arduino, and effortless data upload to cloud platforms for more sophisticated analysis, visualization, and long-term storage.

c. Role in the System: Arduino Nano 33 IoT has the following responsibilities:

   I. Configuration and reading of data from the DFRobot Serial 6-Axis Accelerometer through the UART interface.

   II. Sampling the analog voltage output of the DFRobot Gravity: Analog Sound Level Meter through its ADC channels.

   III. Control of the Micro SD Card Module through the SPI interface for writing the sensor data that it has collected to log files.

   IV. Detection of user input from the hardware push button to start and end the data capture process.

   V. Driving the status LED to give light feedback to the user regarding the system state.

   VI. Regulation of the timing and synchronization of sensor data acquisition from both sensors.

3.1.7.2 DFRobot Serial 6-Axis Accelerometer for Arduino (SKU:SEN0386)

a. Description: This is a very powerful and small 6-axis inertial measurement unit (IMU) which combines a 3-axis accelerometer to detect linear acceleration in the X, Y, and Z directions, and a 3-axis gyroscope to detect angular velocity about these axes. In our

first fault detection system, we shall mainly utilize the acceleration data, as vibrations are an important indicator of mechanical faults in circuit breakers. The sensor outputs its readings in digital form through a TTL (Transistor-Transistor Logic) level serial (UART) interface, rendering it directly compatible with the serial communications of the Arduino. It includes a configurable measurement range for the accelerometer (i.e., ±2g, ±4g, ±8g, ±16g) and has a broad frequency response appropriate for capturing the vibration present in circuit breaker operations.

b. Role in the System: The accelerometer is tasked with:
    I. Measuring and detecting the tri-axial accelerations (vibrations) felt at its mounting position on the rotary shaft of the circuit breaker.
    II. Passing these measured acceleration values as digital data packets to the Arduino Nano 33 IoT through the serial interface.

3.1.7.3 DFRobot Gravity: Analog Sound Level Meter

a. Description: It is an SPL-measuring sensor intended to measure the sound pressure level in its local environment. It contains a sensitive microphone that transforms sound waves into a related analog electrical voltage signal. The sensor has a frequency response that is pertinent to human hearing and general acoustic measurement, generally between 31.5Hz and 8.5kHz, which spans the lower and mid-frequency elements commonly present in mechanical sounds and electrical arcing in circuit breakers. The sensor provides an analog voltage level that is proportional to the sound pressure level being measured, with a defined sensitivity (e.g., mV/dB). It is powered by a low DC voltage (usually 3.3V to 5.0V), which does not conflict with the Arduino's supply voltage.
b. Function in the System: The sound level meter is charged with:
    I. Monitoring and sensing the acoustic emissions produced by the circuit breaker as it goes through its operating cycles, specifically those sounds that emanate from the electrical contacts (e.g., arcing, contact bounce) and the operating mechanism.
    II. Sending an analog voltage signal that indicates the instantaneous sound pressure level to the Arduino Nano 33 IoT's analog input pin.

3.1.7.4 DFRobot Fermion: MicroSD Card Module for Arduino

a. Description: This is an easy-to-use and small-sized module that adds the ability for the

Arduino Nano 33 IoT to read data from and write data to an ordinary Micro SD memory card. It usually uses the Serial Peripheral Interface (SPI) protocol, which enables rapid data transfer to communicate with the Arduino. The module has a standard Micro SD card slot and the required level-shifting circuits to facilitate correct communication between the Arduino's 3.3V logic levels and the operating voltage of the SD card.

b. Role in the System: The Micro SD Card Module is tasked with:
    I. Supplying non-volatile storage for the sensor data collected by the Arduino.
    II. Enabling the Arduino to generate and write data into organized log files (e.g., CSV format) on the Micro SD card, with individual files possibly being generated for accelerometer and sound data, and filenames including timestamps for easy retrieval and organization.

3.1.7.5 Custom-Designed and 3D-Printed PLA Cuboidal Case/Enclosure

a. Description: A physical casing specially designed and produced using 3D printing technology with Polylactic Acid (PLA) filament. The cuboidal shape is a small, efficient package size for encapsulating the central electronic components of the system (Arduino, SD card module, power supply circuitry). PLA is a non-conductive material that offers electrical insulation and helps to reduce the risk of electromagnetic interference. The case includes internal mounting provisions for securely retaining the components and external provisions like threaded holes for mounting the quick connectors, and accurately sized slots or holes for the AC power input (if present), the physical push button, the status LED indicator, and easy access to the Micro SD card slot.

b. Role in the System: The 3D-printed case functions to:
    I. Supply protection to the inner electronic parts against dust, moisture, and mishandling.
    II. Provide safe mounting points for the Arduino, SD card module, and possibly power supply elements.
    III. Enable simple and consistent connection of the external sensors through the mounted quick connectors.
    IV. Supply an intuitive interface with the built-in push button and status LED.
    V. Supply some electromagnetic shielding because of the case's closed nature.

3.1.7.6 DC Power Supply Unit

a. Description: This is an external power adapter (charger brick) that transforms AC mains power into a stable DC voltage appropriate for powering the electronic system. The dual voltage output options (5V at 3A and 9V at 2A) offer flexibility in powering the Arduino Nano 33 IoT, which can accept an array of input voltages. The increased current ratings guarantee that the system has enough power for all connected components. The power supply will normally have a common DC barrel jack connector to connect easily to the system.
b. Role in the System: The DC power supply is used as the main source of electricity for the whole multi-sensor fault detection system to supply the required voltage and current to power the Arduino and all the connected sensors and modules.

3.1.7.7 Micro USB Cable

a. Description: A regular Micro USB cable is employed to link the Arduino Nano 33 IoT to a computer. This link has two main functions: first, it is employed to program the Arduino microcontroller with the required software code. Secondly, it can also supply 5V power to the Arduino during the development and testing stage.
b. Role in the System: Mainly for programming the Arduino and as a backup power source during development.

3.1.7.8 Pin Quick Connectors with Water Rings/Seals

a. Description: These are strong and easy-to-use electrical connectors for quick and secure connection and disconnection of wires. The female connectors will be mounted firmly on the outside of the 3D-printed casing to serve as termination points for the internal wiring that is attached to the power and signal pins of the Arduino. The male connectors will be crimped or soldered onto the wires protruding from the vibration and sound sensors. The built-in water rings or seals add an extra level of protection against the environment at connection points, keeping moisture or dust from entering, which is especially critical to maintaining long-term reliability of the system if it is used in industrial settings where contaminants such as these may be present. The 4-pin setup provides both the transmission of power and the required signal lines for each sensor in one, easy-to-use connector.
b. Function in the System: To provide a reliable, safe, and environmentally shielded

connection for the external vibration and sound sensors to the interior electronics of the monitor system contained in the 3D-printed enclosure.

3.1.7.9 Multi-Core Wires

a. Description: Good-quality multi-core (4-core for accelerometer, 3-core for sound level meter) wires with an embedded electromagnetic shield will be utilized to bridge the sensors to the quick connectors. The shielding layer is important in eliminating the pickup of external electromagnetic interference that may contaminate the sensor signals, particularly the sensitive analog signal from the sound level meter. The shielding will be adequately grounded on the Arduino side to ideally channel any induced noise to the ground.
b. Role in the System: To offer a stable and noise-immune path for transferring power and sensor information from the remotely mounted sensors to the main data acquisition system.

3.1.7.10 Jumper Wires

a. Description: A number of short jumper wires (male-to-male and male-to-female) will be employed to make temporary or semi-permanent electrical connections between the Arduino Nano 33 IoT and the Micro SD Card Module, the push button, and the status LED inside the 3D-printed enclosure.
b. Role in the System: To allow for the internal wiring and interconnection of the main electronic components.

3.1.7.11 Physical Push Button

a. Description: Temporary push button switch to be attached to the outside of the 3D-printed case and tied to a digital input pin of the Arduino. The button push will cause the digital input state to change, which the Arduino's software will detect in order to start or stop the recording of data.
b. Function in the System: To offer a basic and straightforward user interface component for managing the start and stop of data acquisition.

3.1.7.12 LED Indicator

a. Description: A tiny Light Emitting Diode (LED) that will be attached to the outside of

the 3D-printed enclosure and wired to a digital output pin of the Arduino using a current-limiting resistor. By modifying the state (on/off, flashing) of this LED in the Arduino program, we can give the user visual feedback regarding the system's current operational status (e.g., system powered up, data logging active, system idle, error condition).

b. Function within the System: To give visual feedback of the system's operating status to the user.

3.1.7.13 Micro SD Card

a. Description: A small, removable flash memory card that will be plugged into the Micro SD Card Module. The size of the SD card will be determined by the projected amount of sensor data to be logged and the desired length of continuous recording.
b. Role within the System: To offer non-volatile storage of the obtained sensor data logs.

This itemized list systematically defines all the hardware elements that form our multi-sensor fault detection system and their respective functionalities and roles in the overall framework. The thoughtful choice and assembly of these elements are essential in meeting the goals of the project for precise multi-modal data acquisition towards intelligent fault detection in circuit breakers.

3.2 Data Recording

This part gives a comprehensive and detailed description of the data recording process that has been adopted in our multi-sensor fault detection system. It includes the specific points where data acquisition is triggered, the length of each recording session, how the recorded data is organized, in what format the data is stored, and the data log file structure. The main objective of this data recording subsystem is to record synchronized and correctly timestamped data from both the sound sensor and vibration sensor during the most critical operational phases of the circuit breaker, giving rich data for further analysis and fault detection using machine learning methods.

3.2.1 Specific Points of Data Recording

Data recording by our system is tailored to capture the dynamic operation of the circuit breaker at its most critical operational transitions, since it is there that characteristic vibration and acoustic signatures due to normal operation as well as possible fault conditions are most likely to be created. The commencement and end of data recording are closely managed to concentrate

on these key time periods, optimizing the appropriateness of gathered data and storage space and processing effectiveness.

3.2.2 Commencement of Data Recording

- User-Initiated Trigger: The main method of starting a data recording session is a manual trigger triggered by the user pressing the physical push button incorporated into the system's 3D-printed casing. The push button is wired to a specific digital input pin on the Arduino Nano 33 IoT.
- Software Button Press Detection: The Arduino's software permanently checks the state of this digital input pin. When the software sees a change from the unpressed state (HIGH because of the pull-up resistor) to the pressed state (LOW when the button is closed, grounding the input pin), it interprets this as a user instruction to start a new data record session.
- Near-Simultaneous Sensor Activation: Upon sensing the press of the button, the Arduino will instantly trigger the process of reading data from both the DFRobot Serial 6-Axis Accelerometer (through serial communication) and the DFRobot Gravity: Analog Sound Level Meter (through analog-to-digital conversion). The software shall be programmed to begin reading from both sensors as close to each other in terms of time so that the resultant multi-modal data is temporally aligned.
- Visual Feedback: Immediately when the process of recording data starts, the status LED light indicator on the casing of the system will be triggered (e.g., lit up with a steady light or began to flash) to serve as a prompt visual feedback to the user that the system is actually recording data.

3.2.3 Generation of New Data Logs for Every Recording Session

- Unique Identification: In every case of data recording (i.e., every time the push button is clicked and a recording session is finished), the Arduino will generate a new set of data log files on the Micro SD card. This way, data from various operational cycles are stored separately for analysis.
- Timestamped Filenames: For easy identification and organization of the recorded data, the filenames of the log files will include a specific timestamp showing the exact date and time when the recording session started. This timestamp will probably be taken from the internal clock of the Arduino (or an external RTC if used). A standard naming

convention will be followed (e.g., "A000.CSV" for accelerometer data and "S000.CSV" for sound data).

- This section will give a detailed description of the software code executed on the Arduino Nano 33 IoT to control the data acquisition and logging processes, and a Description of the structure and format of the resulting Comma Separated Values (CSV) files that will be saved on the Micro SD card.

3.2.4 Arduino Code

```
#include <DFRobot_WT61PC.h>
#include <SD.h>
#include <SPI.h>

// Pin definitions
#define SOUND_SENSOR_PIN A1
#define BUTTON_PIN 7
#define LED_PIN 6
#define SD_CS_PIN 10

// Serial port for WT61PC
#define FPSerial Serial1
DFRobot_WT61PC sensor(&FPSerial);

// Files
File accFile;
File soundFile;

// Recording state
bool wasRecording = false;
int fileIndex = 0;
unsigned long lastBlinkTime = 0;
bool ledState = false;

void setup() {
  Serial.begin(115200);
  FPSerial.begin(9600);

  pinMode(BUTTON_PIN, INPUT_PULLUP);  // Active LOW
  pinMode(LED_PIN, OUTPUT);
  digitalWrite(LED_PIN, LOW);

  // Set accelerometer to ±2g range
  Serial.println("⏳ Configuring accelerometer to ±2g...");
  FPSerial.println("AT+ACCRANGE=0"); delay(200); // ±2g range
```

Figure 3.2.1 Arduino Code - I

```
  FPSerial.println("AT+SAVE"); delay(200);
  Serial.println("✅ Accelerometer configured.");

  // Initialize SD
  Serial.print("📦 Initializing SD card... ");
  if (!SD.begin(SD_CS_PIN)) {
    Serial.println("❌ SD init failed!");
    while (1);
  }
  Serial.println("✅ SD card ready.");

  // Create folders if needed
  if (!SD.exists("/Accel")) {
    if (SD.mkdir("/Accel")) Serial.println("📁 Created '/Accel'");
    else Serial.println("❌ Failed to create '/Accel'");
  }
  if (!SD.exists("/Sound")) {
    if (SD.mkdir("/Sound")) Serial.println("📁 Created '/Sound'");
    else Serial.println("❌ Failed to create '/Sound'");
  }
}

void loop() {
  bool isRecording = digitalRead(BUTTON_PIN) == LOW;

  if (isRecording && !wasRecording) {
    createNewLogFiles();
    Serial.println("▶️ Recording started.");
  }

  if (!isRecording && wasRecording) {
    closeFiles();
    digitalWrite(LED_PIN, LOW);
    Serial.println("⏹️ Recording stopped.");
  }
```

Figure 3.2.2 Arduino Code - II

```
71
72    if (isRecording) {
73      logSensorData();
74      blinkLED(100); // blink every 100ms
75    } else {
76      digitalWrite(LED_PIN, LOW);
77    }
78
79    wasRecording = isRecording;
80  }
81
82  void createNewLogFiles() {
83    char accFilename[32];
84    char sndFilename[32];
85
86    do {
87      sprintf(accFilename, "/Accel/A%03d.CSV", fileIndex);
88      sprintf(sndFilename, "/Sound/S%03d.CSV", fileIndex);
89      fileIndex++;
90    } while (SD.exists(accFilename) || SD.exists(sndFilename));
91
92    accFile = SD.open(accFilename, FILE_WRITE);
93    soundFile = SD.open(sndFilename, FILE_WRITE);
94
95    if (accFile) {
96      accFile.println("AccX,AccY,AccZ,GyroX,GyroY,GyroZ,AngleX,AngleY,AngleZ");
97      Serial.print("✅ Logging to "); Serial.println(accFilename);
98    } else {
99      Serial.println("❌ Failed to create accel file.");
100   }
101
102   if (soundFile) {
103     soundFile.println("dBA");
104     Serial.print("✅ Logging to "); Serial.println(sndFilename);
105   } else {
```

Figure 3.2.3 Arduino Code - III

```
106       Serial.println("❌ Failed to create sound file.");
107     }
108   }
109
110   void closeFiles() {
111     if (accFile) accFile.close();
112     if (soundFile) soundFile.close();
113   }
114
115   void logSensorData() {
116     if (sensor.available()) {
117       float ax = sensor.Acc.X;
118       float ay = sensor.Acc.Y;
119       float az = sensor.Acc.Z;
120       float gx = sensor.Gyro.X;
121       float gy = sensor.Gyro.Y;
122       float gz = sensor.Gyro.Z;
123       float angX = sensor.Angle.X;
124       float angY = sensor.Angle.Y;
125       float angZ = sensor.Angle.Z;
126
127       // Display for Serial Monitor
128       Serial.print("Sound Level (dBA): ");
129       float voltage = analogRead(SOUND_SENSOR_PIN) * (3.3 / 1023.0);
130       float dBA = voltage * 50.0;
131       Serial.println(dBA, 1);
132
133       Serial.print("AccX: "); Serial.print(ax);
134       Serial.print("  AccY: "); Serial.print(ay);
135       Serial.print("  AccZ: "); Serial.println(az);
136
137       // Output for Serial Plotter
138       Serial.print(ax); Serial.print(",");
139       Serial.print(ay); Serial.print(",");
140       Serial.println(az);
```

Figure 3.2.4 Arduino Code - IV

```
141
142       // Save to SD
143       if (accFile) {
144         accFile.print(ax); accFile.print(",");
145         accFile.print(ay); accFile.print(",");
146         accFile.print(az); accFile.print(",");
147         accFile.print(gx); accFile.print(",");
148         accFile.print(gy); accFile.print(",");
149         accFile.print(gz); accFile.print(",");
150         accFile.print(angX); accFile.print(",");
151         accFile.print(angY); accFile.print(",");
152         accFile.println(angZ);
153         accFile.flush();
154       }
155
156       if (soundFile) {
157         soundFile.println(dBA, 1);
158         soundFile.flush();
159       }
160     }
161   }
162
163   void blinkLED(unsigned long interval) {
164     if (millis() - lastBlinkTime >= interval) {
165       ledState = !ledState;
166       digitalWrite(LED_PIN, ledState ? HIGH : LOW);
167       lastBlinkTime = millis();
168     }
169   }
170
```

Figure 3.2.5 Arduino Code - V

3.3 Machine Learning Model Creation

3.3.1 Preprocessing

In every machine learning pipeline, data preprocessing is an important step that has a direct bearing on the accuracy and reliability of the resultant model. Raw data obtained from sensors tends to be noisy, incomplete, or inconsistent in real-time applications of mechanical systems. Hence, preprocessing comprises cleaning, normalizing, combining, and formatting the data so that it is well-suited for feature extraction and training strong machine learning models. In this project, data was collected from a multi-sensor setup on a circuit breaker. The primary goal was to detect faults based on sensor data captured under four mechanical conditions: 0, 1, 2, and 3 springs removed. For each condition, two types of data were recorded—sound levels and acceleration values—using analog sound sensors and a 6-axis accelerometer, respectively.

The error handling in preprocessing lies in importing raw sound and acceleration files for all fault conditions as a first step. These data could be found as CSV files, with each of them having about 15,000 rows of time-series sensor readings. With the pandas library in Python, both files

were imported into DataFrames. An error handling of primary concern was the handling of missing or absent data. Missing data can occur due to delay in transmission or sensor lag. These were treated by forward fill interpolation, applied by 'fillna', which fills in missing values with the last available valid reading. This method guarantees continuity of data without adding spurious jumps or sudden changes in the signal.

Subsequently, the sensor signals were usually marred by slight fluctuations or noise, which would lower the precision of fault classification. To minimize this noise while preserving important event patterns like peaks due to spring faults, the rolling average filter was used. Precisely, a five-sample window was utilized so that the signal could be smoothed while keeping the integrity of fault-related spikes intact. Following noise reduction, the subsequent step involved combining the two sensor streams. Because both data sets were captured at the same time and for the same time period, they were concatenated along the column axis to create a single dataset for each condition. Raw values, however, could not be compared because of variations in sensor range and units. To address this issue, Min-Max Normalization was used to normalize all the values to a common range from 0 to 1. This prevents the sound data and acceleration data from having an undue impact on the learning process during model training. Through conducting thorough preprocessing, we guarantee that the models will be provided with uniform and high-quality data, thus enhancing their efficiency in determining the presence and severity of faults in the circuit breaker system accurately.

```python
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
import matplotlib.pyplot as plt

# Define file name patterns
conditions = ["0_removed", "1_removed", "2_removed", "3_removed"]
window_size = 5  # Rolling average window

for condition in conditions:
    sound_file = f"{condition}_sound.csv"
    accel_file = f"{condition}_acceleration.csv"

    # Load CSVs
    sound_df = pd.read_csv(sound_file)
    accel_df = pd.read_csv(accel_file)

    # Fill missing values (using updated syntax)
    sound_df = sound_df.ffill()
    accel_df = accel_df.ffill()

    # Apply rolling average
    sound_smooth = sound_df.rolling(window=window_size, min_periods=1).mean()
    accel_smooth = accel_df.rolling(window=window_size, min_periods=1).mean()

    # Combine data
    combined_df = pd.concat([sound_smooth, accel_smooth], axis=1)
```

Figure 3.3.1 Preprocessing Python Code- I

```python
    # Normalize
    scaler = MinMaxScaler()
    normalized = scaler.fit_transform(combined_df)
    normalized_df = pd.DataFrame(normalized, columns=combined_df.columns)

    # Save preprocessed CSV
    output_file = f"{condition}_combined_normalized.csv"
    normalized_df.to_csv(output_file, index=False)
    print(f"✅ Saved: {output_file}")

    # Plot sound and acceleration (first column of each)
    plt.figure(figsize=(12, 5))

    plt.subplot(1, 2, 1)
    plt.plot(sound_df.iloc[:, 0], label='Raw Sound', alpha=0.4)
    plt.plot(sound_smooth.iloc[:, 0], label='Smoothed Sound', alpha=0.9)
    plt.title(f"{condition} - Sound Signal")
    plt.legend()

    plt.subplot(1, 2, 2)
    plt.plot(accel_df.iloc[:, 0], label='Raw Acceleration', alpha=0.4)
    plt.plot(accel_smooth.iloc[:, 0], label='Smoothed Acceleration', alpha=0.9)
    plt.title(f"{condition} - Acceleration Signal")
    plt.legend()

    plt.tight_layout()
    plt.show()
```

Figure 3.3.2 Preprocessing Python Code - II

3.3.1.1 Importing Required Libraries

The preprocessing of multi-sensor data begins with the import of essential Python libraries that enable handling, transformation, and visualization of large time-series datasets. The pandas library is used to load and manipulate data from CSV files, as it offers flexible data structures like DataFrames suited for tabular sensor data. To bring all sensor readings to a comparable scale, MinMaxScaler from scikit learn is used for normalization. This prevents bias due to

varying ranges across features. Lastly, matplotlib.pyplot is imported for plotting raw and smoothed sensor signals, helping in the visual inspection of data quality and transformations applied during preprocessing.

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
import matplotlib.pyplot as plt
```

Figure 3.3.3 Importing Required Libraries

3.3.1.2 Defining File Patterns and Parameters

To efficiently manage and preprocess data from multiple experimental conditions, a structured naming pattern for files is used. Four conditions "0_removed", "1_removed", "2_removed", and "3_removed" represent different simulated fault states in the circuit breaker, such as removal or failure of specific components. A rolling window size of 5 is also defined to smooth the sensor signals using a moving average filter. This parameter controls the number of data points considered when calculating the average, allowing for noise reduction while preserving signal trends. Setting such variables at the beginning ensures scalability, consistency, and ease of modification throughout the preprocessing workflow.

```
# Define file name patterns
conditions = ["0_removed", "1_removed", "2_removed", "3_removed"]
window_size = 5  # Rolling average window
```

Figure 3.3.4 Defining File Patterns and Parameters

3.3.1.3 Reading and Processing Sensor Data

For each fault condition, corresponding sound and acceleration readings are loaded from CSV files using pandas. These files store raw time-series data from the experiment. Often, real-time data acquisition systems introduce gaps due to transmission lags or hardware latency, resulting in missing values. To ensure signal continuity and maintain data integrity, a forward fill method is applied. This technique fills missing entries with the most recent valid value in the same column, which is especially effective when missing data is sparse. It ensures no abrupt discontinuities or artificial patterns are introduced, making the dataset cleaner for further analysis.

```
for condition in conditions:
    sound_file = f"{condition}_sound.csv"
    accel_file = f"{condition}_acceleration.csv"

    # Load CSVs
    sound_df = pd.read_csv(sound_file)
    accel_df = pd.read_csv(accel_file)

    # Fill missing values (using updated syntax)
    sound_df = sound_df.ffill()
    accel_df = accel_df.ffill()
```

Figure 3.3.5 Reading and Processing Sensor Data

3.3.1.4 Smoothing Using Rolling Average

Raw sensor signals, especially from mechanical systems like circuit breakers, often contain high-frequency noise due to environmental disturbances or vibration artifacts. To suppress such noise and enhance the interpretability of the data, a rolling average filter is applied to both sound and acceleration signals. Using a window size of 5, the filter computes the mean of each value with its four preceding values, thus smoothing the curve and revealing underlying trends. This technique helps preserve essential signal characteristics while minimizing unwanted fluctuations. Smoothing is crucial in early-stage signal processing to make downstream tasks like classification or clustering more robust.

```
    # Apply rolling average
    sound_smooth = sound_df.rolling(window=window_size, min_periods=1).mean()
    accel_smooth = accel_df.rolling(window=window_size, min_periods=1).mean()

    # Combine data
    combined_df = pd.concat([sound_smooth, accel_smooth], axis=1)
```

Figure 3.3.6 Smoothing Using Rolling Average

3.3.1.5 Combining and Normalizing the Data

Once the rolling average is applied, the treated sound and acceleration data are combined into one DataFrame. By doing so, the combined dataset is able to capture complementary information from two sets of sensors, hence, richer and more robust input for further analysis. Since sound and acceleration signals are measured on different units and scales, normalization is needed to reduce all features to a comparable scale. Min-Max normalization method is used, which converts every value into a [0, 1] range using its minimum and maximum value. This is done so that no sensor dominates because of scale variation and makes the data ready for machine learning algorithms. These preprocessed files are used as a starting point for further analysis, e.g., fault classification or feature extraction.

```
# Normalize
scaler = MinMaxScaler()
normalized = scaler.fit_transform(combined_df)
normalized_df = pd.DataFrame(normalized, columns=combined_df.columns)

# Save preprocessed CSV
output_file = f"{condition}_combined_normalized.csv"
normalized_df.to_csv(output_file, index=False)
print(f"✅ Saved: {output_file}")
```

Figure 3.3.7 Combining and Normalizing the Data

3.3.2 Model Training

The objective of the machine learning code used in this project is to construct, compare, and validate several classification models for the diagnosis of mechanical faults in a circuit breaker system. The process is data-driven and utilizes synthetically generated sensor data to mimic real-world fault conditions. The data is comprised of both sound (in dBA) and acceleration measurements, each file representing a particular level of fault severity—ranging from normal operation through progressive spring removals (0 to 3). Through data aggregation, the training of machine learning classifiers, and the comparison of their accuracy, the code strives to find the best model for the precise classification of fault conditions in circuit breakers.

The code starts with the importing of necessary libraries, such as pandas for data manipulation, scikit-learn for model training and testing, and seaborn/matplotlib for plots. A list of fault conditions—["0_removed", "1_removed", "2_removed", "3_removed"]—is defined, and then a loop is utilized to read and label each respective normalized CSV file. Each dataset is marked with the condition, and all datasets are combined into one dataframe. This format makes the final training dataset complete and easily labeled for supervised learning.

Then, the data frame is split into features and labels. The input features (X) are all columns except "Condition," and the target variable (y) is the "Condition" column. Because the fault conditions are initially saved as strings, a LabelEncoder is used to transform them into numeric class labels—a prerequisite for training machine learning models. The merged dataset is then split into training and test sets based on an 80:20 ratio so that model performance can be tested on unseen data.

The central theme of the script is training and comparing five classification models, namely Random Forest, Decision Tree, Support Vector Machine (SVM), Naive Bayes, and XGBoost. All models are initialized with corresponding hyperparameters and trained on the training set using the fit() function. Predictions on the test set are made after training, and multiple

evaluation metrics are computed. A classification report with precision, recall, and F1-score by class provides a general idea of model performance by fault state. A confusion matrix for class-level accuracy of predictions is shown as a Seaborn heatmap for enhanced understanding.

To allow the models to be reused later without retraining, the trained models are saved as.pkl files. The label encoder is also stored for future uniform label transformations at prediction time. The model and label encoder files are stored using joblib.dump() for both, so they can be loaded and utilized later with negligible overhead. This adds tremendous efficiency and scalability to the fault detection system.

In the loop of evaluation, the script saves the best-performing model by maintaining a record of the best model and determining the best model. The ultimate output aids the user in determining which classifier would be best for deployment. The process also manages exceptions nicely through try-except blocks, so the script can still proceed even in the case of a failure in one model in fitting or prediction.

Overall, the code establishes an end-to-end machine learning pipeline—from data preparation and encoding to training, evaluation, and model selection. It is modular, understandable, and scalable and therefore a good foundation for fault classification in industrial applications like circuit breaker diagnosis. The ability to store and reuse the models and the label encoders is an indication that the system is efficient and can be made reusable in the future.

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import LabelEncoder
import xgboost as xgb
import seaborn as sns
import matplotlib.pyplot as plt

# Define the conditions
conditions = ["0_removed", "1_removed", "2_removed", "3_removed"]

# Load and combine data
data = []
for condition in conditions:
    file = f"{condition}_combined_normalized.csv"
    df = pd.read_csv(file)
    df["Condition"] = condition
    data.append(df)

df_combined = pd.concat(data, axis=0)

# Split into features and target
X = df_combined.drop(columns=["Condition"])
y = df_combined["Condition"]

# Encode the string labels into integers
label_encoder = LabelEncoder()
y_encoded = label_encoder.fit_transform(y)
X_train, X_test, y_train, y_test = train_test_split(X, y_encoded, test_size=0.2, random_state=42)
```

Figure 3.3.8 Model Training Python Code - I

```python
# Initialize models
models = {
    "Random Forest": RandomForestClassifier(n_estimators=100, random_state=42),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "SVM": SVC(kernel='linear', random_state=42),
    "Naive Bayes": GaussianNB(),
    "XGBoost": xgb.XGBClassifier(eval_metric='mlogloss')
}

# Evaluate models
best_model = None
best_score = 0
model_reports = {}

for model_name, model in models.items():
    try:
        model.fit(X_train, y_train)
        y_pred = model.predict(X_test)

        # Inverse transform for human-readable labels if needed
        y_test_labels = label_encoder.inverse_transform(y_test)
        y_pred_labels = label_encoder.inverse_transform(y_pred)

        print(f"\n{model_name} Classification Report:")
        print(classification_report(y_test_labels, y_pred_labels))

        cm = confusion_matrix(y_test_labels, y_pred_labels, labels=label_encoder.classes_)
        print(f"\n{model_name} Confusion Matrix:")
        print(cm)
```

Figure 3.3.9 Model Training Python Code - II

```
        plt.figure(figsize=(6, 5))
        sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                    xticklabels=label_encoder.classes_, yticklabels=label_encoder.classes_)
        plt.title(f"{model_name} - Confusion Matrix Heatmap")
        plt.xlabel("Predicted")
        plt.ylabel("True")
        plt.show()

        accuracy = accuracy_score(y_test, y_pred)
        print(f"\n{model_name} Accuracy: {accuracy:.4f}")

        model_reports[model_name] = {
            "classification_report": classification_report(y_test_labels, y_pred_labels),
            "confusion_matrix": cm,
            "accuracy": accuracy
        }

        if accuracy > best_score:
            best_score = accuracy
            best_model = model

    except Exception as e:
        print(f"\nError with {model_name}: {str(e)}")
print(f"\nThe best model is {best_model.__class__.__name__} with accuracy: {best_score:.4f}")
```

Figure 3.3.10 Model Training Python Code - III

3.3.2.1 Data Loading and Labeling

This code segment initializes the pipeline by loading four distinct normalized datasets of mechanical conditions: 0_removed, 1_removed, 2_removed, and 3_removed. These datasets hold preprocessed sensor readings (acceleration and audio data), one for a particular number of springs removed from the circuit breaker for simulating the severity of the fault. A for-loop is used to import all the CSV files into a pandas DataFrame, and a new column named "Condition" is added to represent the corresponding fault level. The individual DataFrames are joined using pd.concat() into one dataset. This action makes all input samples correctly labeled and ready for supervised machine learning tasks.

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import LabelEncoder
import xgboost as xgb
import seaborn as sns
import matplotlib.pyplot as plt

# Define the conditions
conditions = ["0_removed", "1_removed", "2_removed", "3_removed"]

# Load and combine data
data = []
for condition in conditions:
    file = f"{condition}_combined_normalized.csv"
    df = pd.read_csv(file)
    df["Condition"] = condition
    data.append(df)

df_combined = pd.concat(data, axis=0)
```

Figure 3.3.11 Data Loading and Labelling

3.3.2.2 Feature-Label Separation and Encoding

Once the dataset is joined, the next step is to separate the features (input sensor readings) and the target label (fault class). Everything except "Condition" is dropped as the feature matrix X, and the "Condition" column is the label vector y. Because the labels are originally string-based (e.g., "0_removed"), they are encoded into integer class indices (e.g., 0, 1, 2, 3) by a LabelEncoder. The conversion is necessary because most machine learning algorithms can only handle numeric labels to learn from. This step ensures that the model will be learning to map sensor readings to specific fault classes during training.

```python
# Split into features and target
X = df_combined.drop(columns=["Condition"])
y = df_combined["Condition"]

# Encode the string labels into integers
label_encoder = LabelEncoder()
y_encoded = label_encoder.fit_transform(y)
```

Figure 3.3.12 Feature- Label Separation and Encoding

3.3.2.3 Train-Test Split

After the feature preparation and labeling, the fault condition labels, saved initially as strings, are mapped into numeric class labels using scikit-learn's LabelEncoder. The encoded and fitted version is saved on disk by joblib.dump() so that duplicate label mappings will be used on subsequent predictions.

After encoding, the data is divided into training and test sets using scikit-learn's train_test_split() function. An 80:20 ratio is used, where 80% of the data is reserved for model training and 20% for testing their generalization on unseen data. This is for the purpose of ensuring that the models are tested in real-world scenarios and overfitting is avoided. The division is in a balanced ratio of samples for each class, ensuring fair and consistent measurement of model performance.

```
# Encode the string labels into integers
label_encoder = LabelEncoder()
y_encoded = label_encoder.fit_transform(y)
joblib.dump(label_encoder, "label_encoder.pkl")
print("✅ Saved label encoder")

# Train/test split
X_train, X_test, y_train, y_test = train_test_split(X, y_encoded, test_size=0.2, random_state=42)
```

Figure 3.3.13 Train-Test Split

3.3.2.4 Model Training and Prediction

This is the main section where five different classification models are instantiated and tested: Random Forest, Decision Tree, Support Vector Machine (SVM), Naive Bayes, and XGBoost. All the models are instantiated with a default configuration and trained on the training dataset using the fit() function. After training, predictions are made on the test dataset using the predict() function. A try-except block is used to catch any possible errors elegantly while fitting the models. This module-based design facilitates simple comparison of different algorithms and also offers robustness while testing different classifiers simultaneously.

```
# Initialize models
models = {
    "Random Forest": RandomForestClassifier(n_estimators=100, random_state=42),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "SVM": SVC(kernel='linear', random_state=42),
    "Naive Bayes": GaussianNB(),
    "XGBoost": xgb.XGBClassifier(eval_metric='mlogloss')
}
```

Figure 3.3.14 Model Training and Prediction

3.3.2.5 Evaluation and Visualization

The test phase assesses how well each machine learning model performs on unseen test data. For every classifier, a classification report is printed including precision, recall, F1-score, and support for every fault category. This is helpful in assessing how well every model can detect various fault severities. Apart from this, a confusion matrix is computed and plotted as a heatmap using Seaborn, providing a simple-to-understand view of correct and incorrect predictions for all classes.

The model's overall accuracy is calculated to summarize its performance into a single value. While being evaluated, the best-performing classifier with the highest accuracy is tracked and saved as the most effective. For subsequent use without retraining, every trained model and the label encoder are saved using joblib.dump().

This section thus combines both quantitative and visual evaluation techniques such that the overall picture of the model's strengths and weaknesses is comprehensible. The results obtained, in the form of metrics and visualizations, form the foundation for determining the most precise model for fault detection. A generalized description of these evaluation metrics will be given in the Results and Discussions section of the report.

```
# Evaluate models
best_model = None
best_score = 0
model_reports = {}

for model_name, model in models.items():
    try:
        model.fit(X_train, y_train)
        y_pred = model.predict(X_test)

        # Inverse transform for human-readable labels if needed
        y_test_labels = label_encoder.inverse_transform(y_test)
        y_pred_labels = label_encoder.inverse_transform(y_pred)

        print(f"\n{model_name} Classification Report:")
        print(classification_report(y_test_labels, y_pred_labels))

        cm = confusion_matrix(y_test_labels, y_pred_labels, labels=label_encoder.classes_)
        print(f"\n{model_name} Confusion Matrix:")
        print(cm)

        plt.figure(figsize=(6, 5))
        sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                    xticklabels=label_encoder.classes_, yticklabels=label_encoder.classes_)
        plt.title(f"{model_name} - Confusion Matrix Heatmap")
        plt.xlabel("Predicted")
        plt.ylabel("True")
        plt.show()

        accuracy = accuracy_score(y_test, y_pred)
        print(f"\n{model_name} Accuracy: {accuracy:.4f}")

        model_reports[model_name] = {
            "classification_report": classification_report(y_test_labels, y_pred_labels),
            "confusion_matrix": cm,
            "accuracy": accuracy
        }

        if accuracy > best_score:
            best_score = accuracy
            best_model = model

    except Exception as e:
        print(f"\nError with {model_name}: {str(e)}")
print(f"\nThe best model is {best_model.__class__.__name__} with accuracy: {best_score:.4f}")
```

Figure 3.3.15 Evaluation and Visualisation

1. Classification Report

A classification report is a tool for measuring performance in supervised machine learning classification problems. It gives an overview of the most important metrics of evaluation for every class in a binary or multi-class classification problem. Precision, recall, F1-score, and support are the major elements of a classification report.

Precision calculates the number of correct positive predictions out of all the positive predictions made. That is, it informs us about how many of the items that were classified as a particular class actually belonged to that class. Recall, or sensitivity, or true positive rate calculates the number of actual positives correctly predicted by the model. F1-score is the harmonic mean of recall and precision and gives a single measure that balances both issues. Support is the number of true instances per class in the test set.

Collectively, these measures enable the examination of the strengths and limitations of a classifier. For example, a model that has high precision but low recall will be overly conservative, while a model that has high recall but low precision will be excessively liberal. The classification report is therefore an overall snapshot of how the model is performing per class.

2. Confusion Matrix

Confusion matrix is a commonly employed visualization tool in classification tasks that assist in the assessment of the performance of a machine learning model. It is usually a square matrix of dimensions n x n, with the number of classes in the classification task denoted as n. Every row of the matrix corresponds to the true class, and every column corresponds to the predicted class.

Diagonal entries in the confusion matrix give the correctly classified instances in each class. Misclassifications, i.e., instances given a wrong class by the classifier, are identified through off-diagonal entries. In a task with 4 classes, say, if an entry has a non-zero value, it signifies that various samples of class 2 got wrongly predicted as class 3.

The confusion matrix is particularly useful when dealing with imbalanced datasets, where accuracy alone can be misleading. It allows one to see which particular classes are being confused with which, so that one can better customize the model or use methods such as class weighting or oversampling. In multi-class problems, the matrix turns into an important

diagnostic tool for understanding which classes are difficult for the model to distinguish and informs further efforts to improve the model.

3. Accuracy Score

Accuracy score is a very fundamental yet common performance measure of a classification model. It is the proportion of the number of predictions that were predicted correctly by the model to the total number of predictions. In other words, accuracy indicates how frequently the model makes a correct prediction.

Mathematically, accuracy is computed as:

$$Accuracy = \frac{Number\ of\ Correct\ Predictions}{Total\ Predictions\ Made}$$

This is an intuitive and easy-to-interpret measure, hence a popular place to start when evaluating model performance. It is best used, however, when the dataset is balanced and all classes are roughly equally represented. When there is a severe imbalance between class sizes, a high accuracy score can be uninformative. For example, if 95% of the data is in one class, a model always predicting that class will have 95% accuracy—but would be worthless in practice for detecting the minority class.

While accuracy gives a broad sense of performance, it is frequently employed together with other measures such as precision, recall, and the confusion matrix to gain a better and more reliable understanding of how a model performs under different conditions.

3.4 Software Development

The given Python code is a full graphical user interface (GUI)-based system for identifying circuit breaker faults based on machine learning models. The process starts with the loading of pre-trained models and label encoders stored using the joblib library. Five classifiers—Random Forest, Decision Tree, Support Vector Machine (SVM), Naive Bayes, and XGBoost—are loaded into memory to make predictions. A label encoder is employed to convert numeric prediction values back into their respective class labels for easier interpretation.

The user interface is built using the tkinter library, which offers an easy and lightweight

framework for developing desktop applications. After the program is opened, a window with the name "Circuit Breaker Fault Detection" is shown, with three main interactive buttons that allow users to upload a Sound CSV file, an Accelerometer CSV file, and a third to run the prediction process. A huge scrolled text area is part of the interface for showing detailed model prediction outputs. The user interface is intuitive, with minimal training for the users to use the tool.

To improve data quality and eliminate noise, data is smoothed through the use of a rolling mean of window 5. The smoothing process removes sharp spikes or fluctuations that might hamper model accuracy. Data is then normalized between 0 and 1 through the MinMaxScaler to ensure all the values of the features lie between the same range and any single feature does not dominate the prediction.

Post preprocessing, both sound and accelerometer data are horizontally concatenated, taking both sources of data and presenting them as one common dataset. All features are averaged to create one input vector to capture the global behavior of the signals. This feature vector is reshaped and fed in sequence to every machine learning model that has been loaded. Each model outputs the most likely class label for the input data. If the model accepts probability outputs (Random Forest, Naive Bayes, XGBoost), then the highest probability score is also fetched as a measure of confidence.

All the model predictions are gathered, and Python's Counter class from the collections module is used to determine the most frequently predicted class. The ensemble technique diminishes the possibilities of a mistake from one particular model and increases overall prediction stability. The last majority-voted prediction as well as separate model outputs is efficiently presented in the GUI output box for quick review by users.

Error handling is used throughout the code to handle missing file uploads or unforeseen exceptions during prediction. If the user tries to predict without uploading both necessary files, a message box informs them of the oversight. Likewise, in case of any runtime errors, they are trapped and shown politely without crashing the application. In total, the code combines the deployment of machine learning models with a workable user interface, presenting the full solution for non-technical users to implement AI-based fault detection on real-world circuit breaker testing.

```python
import tkinter as tk
from tkinter import filedialog, messagebox, scrolledtext
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler
import joblib
from collections import Counter
from sklearn.metrics import classification_report
import os

# Load models and encoder
model_names = ["RandomForest", "DecisionTree", "SVM", "NaiveBayes", "XGBoost"]
models = {}
for name in model_names:
    models[name] = joblib.load(f"{name}_model.pkl")

label_encoder = joblib.load("label_encoder.pkl")
```

Figure 3.4.1 Software Development Python Code- I

```python
# GUI App
class FaultDetectorApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Circuit Breaker Fault Detection")
        self.root.geometry("700x600")

        self.sound_path = ""
        self.accel_path = ""

        # UI Components
        tk.Label(root, text="Circuit Breaker Fault Detector", font=("Arial", 16, "bold")).pack(pady=10)

        tk.Button(root, text="Upload Sound CSV", command=self.upload_sound).pack(pady=5)
        tk.Button(root, text="Upload Accelerometer CSV", command=self.upload_accel).pack(pady=5)
        tk.Button(root, text="Predict Fault", command=self.predict_fault).pack(pady=10)

        self.output_box = scrolledtext.ScrolledText(root, wrap=tk.WORD, width=80, height=25)
        self.output_box.pack(padx=10, pady=10)

    def upload_sound(self):
        self.sound_path = filedialog.askopenfilename(filetypes=[("CSV files", "*.csv")])
        if self.sound_path:
            messagebox.showinfo("File Selected", f"Sound file: {os.path.basename(self.sound_path)}")

    def upload_accel(self):
        self.accel_path = filedialog.askopenfilename(filetypes=[("CSV files", "*.csv")])
        if self.accel_path:
            messagebox.showinfo("File Selected", f"Accelerometer file: {os.path.basename(self.accel_path)}")

    def preprocess(self, df):
        df = df.rolling(window=5).mean().dropna()
        scaler = MinMaxScaler()
        df_scaled = scaler.fit_transform(df)
        return pd.DataFrame(df_scaled, columns=df.columns)
```

Figure 3.4.2 Software Development Python Code - II

```
    def majority_vote(self, predictions):
        count = Counter(predictions)
        return count.most_common(1)[0][0]

    def predict_fault(self):
        if not self.sound_path or not self.accel_path:
            messagebox.showerror("Error", "Please upload both CSV files.")
            return

        try:
            df_sound = pd.read_csv(self.sound_path)
            df_accel = pd.read_csv(self.accel_path)

            df_sound = self.preprocess(df_sound)
            df_accel = self.preprocess(df_accel)

            df_combined = pd.concat([df_sound.reset_index(drop=True), df_accel.reset_index(drop=True)], axis=1)

            X_input_array = df_combined.mean().values.reshape(1, -1)
            X_input = pd.DataFrame(X_input_array, columns=df_combined.columns)

            predictions = {}
            probs = {}

            for name, model in models.items():
                pred = model.predict(X_input)
                label = label_encoder.inverse_transform(pred)[0]
                predictions[name] = label
```

Figure 3.4.3 Software Development Python Code - III

```
                try:
                    if hasattr(model, "predict_proba"):
                        prob = model.predict_proba(X_input).max()
                    else:
                        prob = 1.0  # for models like SVM if prob disabled
                except:
                    prob = 1.0
                probs[name] = prob

            final_prediction = self.majority_vote(list(predictions.values()))

            # Display
            self.output_box.delete(1.0, tk.END)
            self.output_box.insert(tk.END, "Model Predictions:\n\n")
            for name in model_names:
                self.output_box.insert(tk.END, (f"{name}: {predictions[name]} \n"))

            self.output_box.insert(tk.END, f"\n Majority Vote Prediction: {final_prediction}\n")


        except Exception as e:
            messagebox.showerror("Error", str(e))


# Launch
if __name__ == "__main__":
    root = tk.Tk()
    app = FaultDetectorApp(root)
    root.mainloop()
```

Figure 3.4.4 Software Development Python Code - IV

3.4.1 Model Loading and Setup

The first step of the code is to set up the environment by importing necessary machine learning models and preprocessing functions. The script starts with importing basic libraries like joblib, a commonly used library for serializing and deserializing Python objects, particularly trained

machine learning models.

A list of model names—["RandomForest", "DecisionTree", "SVM", "NaiveBayes", "XGBoost"]—is prepared to process the loading procedure in a systematic way. An empty dictionary 'models' is set up to save each model once loaded. The code goes through the list of model names by using a basic 'for 'loop. For every model, the respective.pkl file (previously stored after training) is loaded by joblib.load() and is stored in the dictionary with the name of the model as the key. This modular design makes it simple to add or drop models in the future without changing the overall design.

Along with the classifiers, a LabelEncoder object is also imported from a stored.pkl". The LabelEncoder is important since, when model training was being done, class labels were presumably transformed from string labels (i.e., "No Fault," "1 Spring Removed") into numeric values (i.e., 0, 1, 2, 3) for model compatibility. When it comes to prediction, the label encoder will decode these numeric predictions back into human-readable fault categories.

This initialization step is crucial to reduce prediction latency and improve user experience since models do not have to be loaded dynamically at runtime. It also creates a neat and tidy architecture for handling multiple machine learning models in the same application.

```
import tkinter as tk
from tkinter import filedialog, messagebox, scrolledtext
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler
import joblib
from collections import Counter
from sklearn.metrics import classification_report
import os

# Load models and encoder
model_names = ["RandomForest", "DecisionTree", "SVM", "NaiveBayes", "XGBoost"]
models = {}
for name in model_names:
    models[name] = joblib.load(f"{name}_model.pkl")

label_encoder = joblib.load("label_encoder.pkl")
```

Figure 3.4.5 Model Loading and Setup

3.4.2 GUI Design and Development

The graphical user interface for the fault detection program is created with Python's tkinter library. The interface is clean and functional, allowing users to engage with the machine learning models implemented beneath without programming knowledge. The window of the GUI is set up with a definite title and size using the root window configuration. This helps the

application window to have an even size, allowing for a simple and tidy workspace for users.

On launch, the user interface has a top header that reads "Circuit Breaker Fault Detector" to inform the users of the application's functionality. The header is below which three major buttons are placed vertically. The first button is for uploading a CSV file of sound level, while the second is for uploading a CSV file of accelerometer data. Clicking on either button prompts the opening of a file dialog window where users can navigate their local directories and choose the right file. After successful selection, a success message appears, showing the name of the selected file.

The third button begins the fault prediction process after both necessary files have been uploaded. Below these buttons, a scrolled text box is positioned to display the output predictions of each machine learning model as well as the final majority-voted conclusion. The employment of a scrolled text area enables the application to provide longer output results without flooding the main window space.

The design makes it so that the user flow is simple: upload the sound file, upload the accelerometer file, and then click predict. Error checking is built into the design so that users cannot continue without both files, thus improving reliability. The GUI overall is designed to be highly user-friendly, with sparse controls and a well-defined workflow that allows for rapid and efficient interaction with the underlying machine learning system.

```
# GUI App
class FaultDetectorApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Circuit Breaker Fault Detection")
        self.root.geometry("700x600")

        self.sound_path = ""
        self.accel_path = ""

        # UI Components
        tk.Label(root, text="Circuit Breaker Fault Detector", font=("Arial", 16, "bold")).pack(pady=10)

        tk.Button(root, text="Upload Sound CSV", command=self.upload_sound).pack(pady=5)
        tk.Button(root, text="Upload Accelerometer CSV", command=self.upload_accel).pack(pady=5)
        tk.Button(root, text="Predict Fault", command=self.predict_fault).pack(pady=10)

        self.output_box = scrolledtext.ScrolledText(root, wrap=tk.WORD, width=80, height=25)
        self.output_box.pack(padx=10, pady=10)

    def upload_sound(self):
        self.sound_path = filedialog.askopenfilename(filetypes=[("CSV files", "*.csv")])
        if self.sound_path:
            messagebox.showinfo("File Selected", f"Sound file: {os.path.basename(self.sound_path)}")

    def upload_accel(self):
        self.accel_path = filedialog.askopenfilename(filetypes=[("CSV files", "*.csv")])
        if self.accel_path:
            messagebox.showinfo("File Selected", f"Accelerometer file: {os.path.basename(self.accel_path)}")
```

Figure 3.4.6 GUI Design and Development

3.4.3 Data Preprocessing and Majority Voting

After the user uploads the required Sound and Accelerometer CSV files, the system starts preprocessing the raw data to make it ready for machine learning prediction. Both uploaded files are imported into pandas DataFrames. The initial preprocessing step performs a rolling mean operation with a window size of five. This method smoothes the data by averaging every five consecutive samples, which assists in eliminating random fluctuations and noise present in real-world sensor measurements.

After the smoothing process, the system will then normalize the data through the application of the MinMaxScaler from the scikit-learn library. Normalization will convert all values of the features to be between zero and one. This process is important because machine learning algorithms are sensitive to feature scales, and having uniform scaling will prevent any feature from being able to predict outcomes on its own due to magnitude.

Following preprocessing, the sound and acceleration data are horizontally concatenated, merging the two varied sensor inputs into a unified feature set. For dimensionality reduction of the high-dimensional data into a ready-to-use real-time prediction format, the mean value is calculated across each feature column. This gives a concise single-row feature vector that describes the overall input behavior.

This input vector that has been processed is then passed to each of the loaded machine learning models separately. Each model makes a prediction of a fault category based on the provided data. In order to make a final strong and stable output, a majority voting process is implemented. All the predictions made by all the models are gathered, and the most commonly predicted fault label is chosen as the final output. This ensemble voting technique enhances the resistance of the system to errors made by individual models so that the final output approximates the most likely fault state based on a consensus and not on the output of an individual model.

```
    def preprocess(self, df):
        df = df.rolling(window=5).mean().dropna()
        scaler = MinMaxScaler()
        df_scaled = scaler.fit_transform(df)
        return pd.DataFrame(df_scaled, columns=df.columns)

    def majority_vote(self, predictions):
        count = Counter(predictions)
        return count.most_common(1)[0][0]

    def predict_fault(self):
        if not self.sound_path or not self.accel_path:
            messagebox.showerror("Error", "Please upload both CSV files.")
            return

        try:
            df_sound = pd.read_csv(self.sound_path)
            df_accel = pd.read_csv(self.accel_path)

            df_sound = self.preprocess(df_sound)
            df_accel = self.preprocess(df_accel)

            df_combined = pd.concat([df_sound.reset_index(drop=True), df_accel.reset_index(drop=True)], axis=1)

            X_input_array = df_combined.mean().values.reshape(1, -1)
            X_input = pd.DataFrame(X_input_array, columns=df_combined.columns)

            predictions = {}
            probs = {}
```

Figure 3.4.7 Data Preprocessing and Majority Voting

```
            for name, model in models.items():
                pred = model.predict(X_input)
                label = label_encoder.inverse_transform(pred)[0]
                predictions[name] = label

                try:
                    if hasattr(model, "predict_proba"):
                        prob = model.predict_proba(X_input).max()
                    else:
                        prob = 1.0  # for models like SVM if prob disabled
                except:
                    prob = 1.0
                probs[name] = prob

            final_prediction = self.majority_vote(list(predictions.values()))

            # Display
            self.output_box.delete(1.0, tk.END)
            self.output_box.insert(tk.END, "Model Predictions:\n\n")
            for name in model_names:
                self.output_box.insert(tk.END, (f"{name}: {predictions[name]} \n"))

            self.output_box.insert(tk.END, f"\n Majority Vote Prediction: {final_prediction}\n")


        except Exception as e:
            messagebox.showerror("Error", str(e))


# Launch
if __name__ == "__main__":
    root = tk.Tk()
    app = FaultDetectorApp(root)
    root.mainloop()
```

Figure 3.4.8 Data Preprocessing and Majority Voting

CHAPTER 4: RESULTS AND DISCUSSIONS

4.1 System Design

The data acquisition system is based on an Arduino Nano 33 IoT, which elegantly links our mechanical test rig and the world of the digital where fault diagnosis and signal processing take place. Vibration signals from a DFRobot Serial 6-Axis Accelerometer are transmitted over UART at a corresponding baud rate, split into tri-axial acceleration values, and time-stamped in real time. In parallel, the DFRobot Gravity analog sound level meter supplies its continuously fluctuating voltage output to the Arduino 12-bit ADC, measuring acoustic pressure fluctuations to 8.5 kHz with precision. Both streams are triggered by a single push-button, providing software-based synchronization tightness and allowing mechanical impacts, arcing phenomena, and acoustic signatures to be correlated directly.

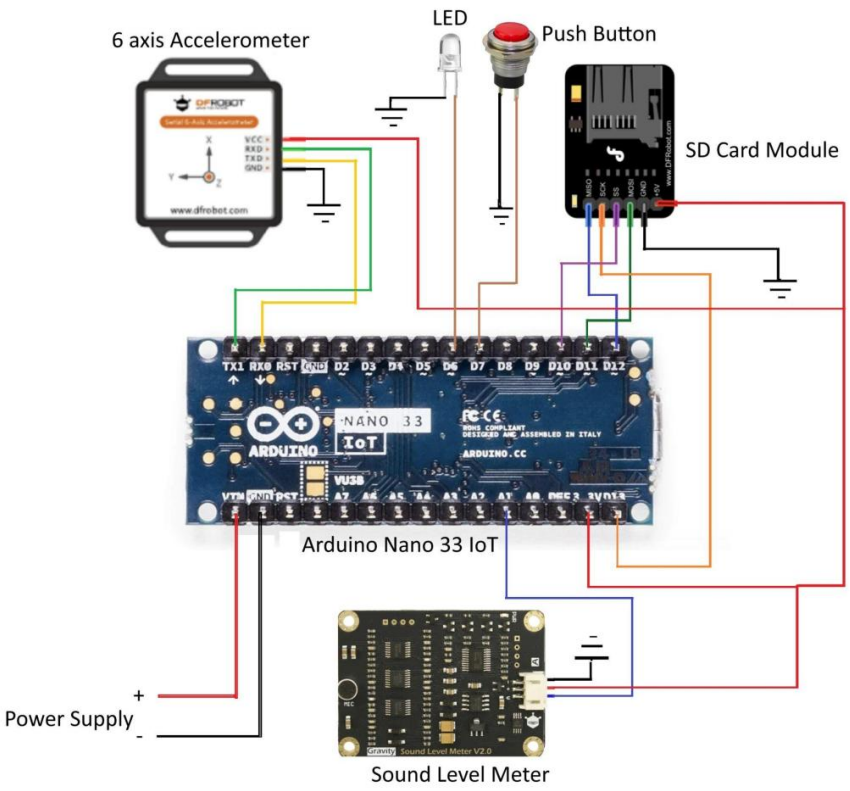


Figure 4.1.1 System Design

Meticulous attention to sampling rates, based on initial spectral analysis of breaker operation,

guarantees that transient effects are accurately captured without aliasing. In order to prevent observed cross-talk and noise coupling into the analog channel, shielding was applied on sensor lines and simple RC filtering was added at the ADC input. Lastly, each set of samples is recorded to independent, time-stamped CSV files on a micro-SD card, ensuring consistent storage of high-resolution sound and vibration data for subsequent preprocessing, feature extraction, and fault classification using machine learning.

4.2 Data Set Recording

The sensor data employed for fault classification in the Air Circuit Breaker (ACB) system was obtained through a Data Acquisition (DAQ) system. Two different sets of sensor inputs were recorded and stored as CSV files—one for acoustic signatures (dBA) from a sound sensor, and another for motion dynamics comprising acceleration, gyroscope data, and orientation angles. All these measurements were obtained in real-time during ACB operation to identify the changes in response to spring removal.

The sound sensor data (sound.csv) records dBA levels that are usually from 45 to 55 dBA, which is low to moderate noise typical of normal ACB activity. The motion data (acceleration.csv) contains columns like AccX, AccY, GyroX, GyroY, and constant orientation angles. There is a one-to-one correspondence for each row in both files at the same point of time, allowing proper alignment for time-series analysis.

These files generated by DAQ are used as input for preprocessing and prediction models based on machine learning, and their organized and synchronized format is best suited for processing in model training and automated diagnostic systems.

AccX	AccY	AccZ	GyroX	GyroY	GyroZ	AngleX	AngleY	AngleZ
0.000261	-0.02232	0.002677	0.018501	-0.01914	-0.03288	0	0	0
0.008317	0.04603	0.123003	0.017178	-0.00354	0.002105	0	0	0
-0.05544	-0.04508	-0.01729	-0.00422	-0.00765	-0.02173	0	0	0
0.011663	-0.01889	0.062577	0.005346	-0.03456	-0.01489	0	0	0
-0.07084	0.050495	-0.0588	0.037577	-0.01899	-0.02053	0	0	0
0.09286	0.032033	-0.06772	-0.03496	-0.00652	0.023977	0	0	0
0.026764	0.04549	0.026332	0.032591	0.027609	-0.04192	0	0	0
-0.01935	-0.00448	0.024403	0.028175	-0.02322	4.10E-05	0	0	0
-0.10992	-0.00618	0.035855	-0.00171	0.03704	-0.01607	0	0	0
-0.00102	-0.01818	0.002558	-0.0035	-0.00707	0.030055	0	0	0
-0.0008	0.000886	-0.08959	0.01138	0.00814	0.000381	0	0	0
-0.058	-0.054	0.011642	-0.01373	-0.02124	0.022929	0	0	0
-0.03504	0.018428	0.09113	-0.00746	-0.02304	0.008072	0	0	0
-0.00803	-0.04019	-0.00484	-0.00194	-0.02093	0.021922	0	0	0
0.078705	0.035853	0.096477	0.014531	0.002062	-0.00477	0	0	0
0.036051	-0.05076	-0.01792	0.007688	0.019208	0.021681	0	0	0
-0.06198	0.01027	-0.00344	-0.01996	0.007405	0.015617	0	0	0
0.016113	-0.0403	0.06175	0.003417	0.032255	-0.0005	0	0	0

Figure 4.2.1 Acceleration Data Set

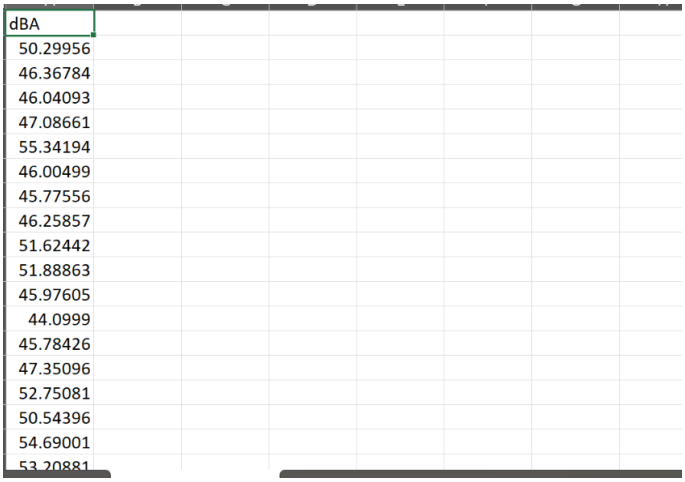


Figure 4.2.2 Sound Data Set

4.3 Machine Learning Model Creation

4.3.1 Preprocessing

After the model development phase was completed, involving data preprocessing and feature preparation, this section reports the results obtained from that pipeline. The goal of preprocessing was not only to clean and normalize the data but also to prepare it for machine learning classification tasks by minimizing noise, completeness, and signal characteristics pertinent to fault detection.

Two main types of output were produced from the preprocessing step: (1) a collection of merged and normalized datasets stored as structured CSV files, and (2) visualizations in the form of line plots that illustrate the impact of smoothing on raw sensor signals. These outputs indicate the integrity and transformation of the data before model training and give insight into the quality of features extracted from the synthetic circuit breaker conditions.

4.3.1.1 Combined Normalized Datasets

The jointly normalized datasets are the final preprocessed structured output for direct input into the machine learning pipeline. The four output CSV files—associated with 0_removed, 1_removed, 2_removed, and 3_removed fault conditions—each have smoothed and normalized sensor data for both sound and acceleration features.

The main function of these files is to transform raw sensor data into clean, consistent, numerically stable format. Applying a rolling average prior to normalization reduces high-

frequency noise and retains the signal trend, increasing the signal-to-noise ratio. This is particularly useful in synthetic fault data where signal variance captures mechanical deterioration. The combined structure guarantees temporal alignment of acoustic and vibration data and provides a complete feature set for every instance in the data stream.

Normalizing all columns with MinMaxScaler guarantees evenness of scale over features. This is required by distance-based or tree-based classifiers to avoid having one feature type (for example, raw dBA) monopolizing learning simply because its numerical range is larger. This outcome is a well-balanced training dataset where every feature participates fairly in model building.

What is so valuable about these outputs is not just their cleanliness but also their label alignment. The structured format guarantees that each row of the dataset maps to a consistent fault condition, making it easier to integrate with supervised learning models. Moreover, the naming convention of each file inserts the fault type directly into the filename, making automated workflows for data loading and classification easier.

The datasets also preserve adequate feature richness to distinguish between classes that are closely spaced. For example, although 2_removed and 3_removed datasets might have very slight differences in signal magnitude or frequency, these differences are preserved by smoothing instead of being suppressed, allowing the classifier to learn fine-grained differences.

In total, the CSV outputs constitute the building blocks of data in the project. They represent an effective conversion of noisy, unstructured sensor data into accurate, labeled, and machine-interpretable form. They have a structure that supports reproducible experiments, predictable results, and high classification accuracy, as later attested by model performance metrics. They are, therefore, an important deliverable in the preprocessing pipeline and a direct facilitator of the project's fault detection objectives.

dBA	AccX	AccY	AccZ	GyroX	GyroY	GyroZ	AngleX	AngleY	AngleZ
0.367238	0.123184	0.105215	0.121255	0.130494	0.033924	0.002757	0.549246	0.786619	0.4908
0.343411	0.129086	0.151526	0.202577	0.128756	0.053691	0.044522	0.549246	0.786619	0.4908
0.334148	0.09991	0.125807	0.166474	0.109436	0.056806	0.039474	0.549246	0.786619	0.4908
0.332685	0.109906	0.121823	0.175411	0.106058	0.041323	0.041032	0.549246	0.786619	0.4908
0.351819	0.091722	0.138236	0.147961	0.120972	0.039919	0.039274	0.549246	0.786619	0.4908
0.341408	0.118862	0.152967	0.128929	0.092872	0.046314	0.066425	0.549246	0.786619	0.4908
0.339973	0.124268	0.15282	0.102795	0.100973	0.062097	0.045403	0.549246	0.786619	0.4908
0.3405	0.134845	0.163825	0.114065	0.117997	0.054211	0.055799	0.549246	0.786619	0.4908
0.3515	0.099209	0.167268	0.106841	0.114287	0.090491	0.055238	0.549246	0.786619	0.4908
0.343129	0.119671	0.148655	0.123428	0.092699	0.096531	0.079394	0.549246	0.786619	0.4908
0.343059	0.092221	0.140214	0.117518	0.117057	0.10396	0.068126	0.549246	0.786619	0.4908
0.338997	0.067376	0.113252	0.113547	0.092713	0.079206	0.099093	0.549246	0.786619	0.4908
0.337847	0.062779	0.11946	0.131585	0.073982	0.079297	0.102928	0.549246	0.786619	0.4908
0.327488	0.092644	0.110243	0.120585	0.073865	0.049922	0.121068	0.549246	0.786619	0.4908
0.329578	0.116011	0.124887	0.145975	0.08334	0.054551	0.104436	0.549246	0.786619	0.4908
0.340651	0.126812	0.11089	0.165349	0.081399	0.060159	0.114607	0.549246	0.786619	0.4908
0.366322	0.125648	0.128308	0.161273	0.078122	0.074676	0.111116	0.549246	0.786619	0.4908
0.38432	0.14064	0.112391	0.15333	0.083839	0.102693	0.107023	0.549246	0.786619	0.4908
0.38579	0.175588	0.142369	0.133029	0.090934	0.137834	0.109218	0.549246	0.786619	0.4908

Figure 4.3.1 Combined Normalized Datasets

4.3.1.2 Raw vs. Smoothed Signal Visualizations

The second key result of the preprocessing step is the series of visualizations of raw sensor signals versus their smoothed versions for both sound and acceleration data. These plots play a dual role: visual qualitative verification of the applied smoothing method, and visual representation of the effect of noise removal on signal quality—especially pertinent for the scenario of mechanical fault detection.

For every condition (from 0_removed to 3_removed), two subplots are plotted. The left subplot plots the original and smoothed sound signals, whereas the right subplot plots the same for acceleration data. Each plot is concerned with the first feature column of the corresponding dataset, usually dBA levels for sound and AccX for acceleration. The smoothed signals are calculated using a rolling average with a fixed window size, overlaid on top of the original signal to allow direct comparison.

These graphs show evidently the efficiency of the smoothing operation in eliminating random noise and sudden oscillations without deforming the general pattern of the signal. In the unsmoothed traces, particularly for acceleration, spikes and irregularities frequently occur—artefacts which are not necessarily due to mechanical faults. The smoothed form eliminates these anomalies but retains meaningful trends like increasing vibration levels or changes in amplitude over time, which are needed for correct fault classification.

The visualization also verifies that smoothing does not over-flatten the signal. For example, in the 3_removed condition, both smoothed sound and acceleration signals still exhibit characteristic peaks, which presumably indicate large mechanical impacts or oscillations. This is important because over-smoothing can cause loss of fault-indicative features.

In conclusion, raw vs. smoothed signal plots confirm data transformation visually, affirming the success of the smoothing approach. They illustrate that useful structure has been maintained at the expense of unwanted noise suppression. This guarantees that the ultimate feature fed into the classifier is both clean and information-rich—perfect for downstream model training and fault detection.

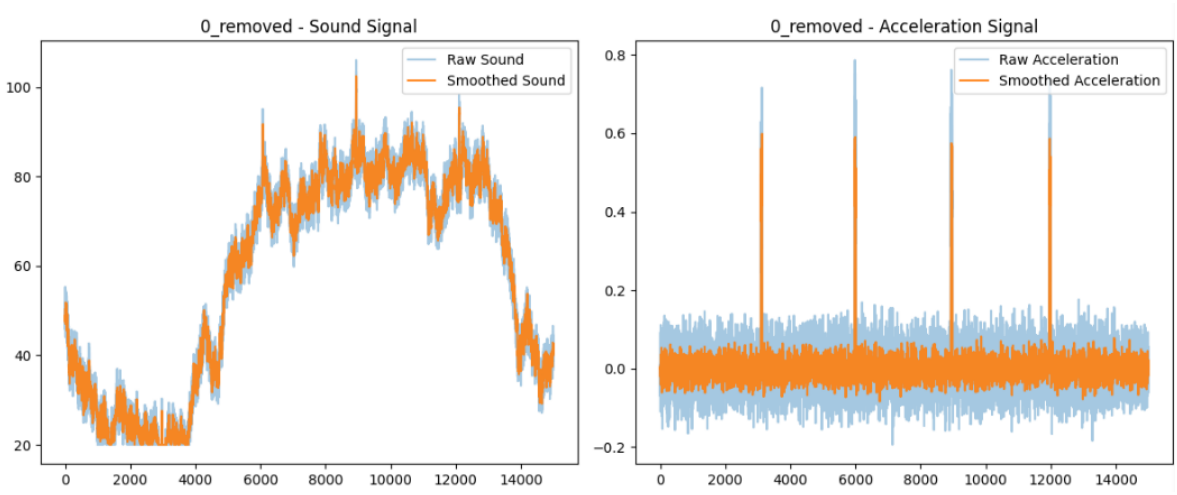


Figure 4.3.2 Sound and Vibrations Data for 0 spring removed

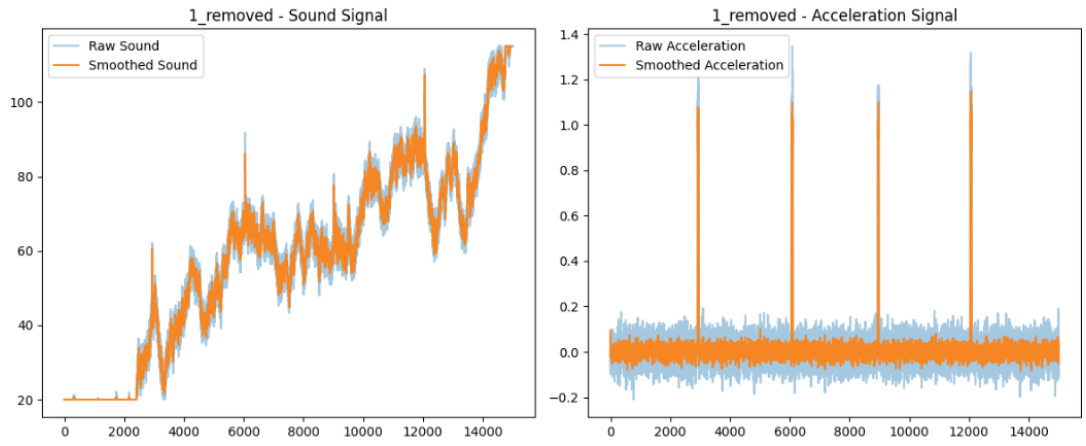


Figure 4.3.3 Sound and Vibrations Data for 1 spring removed

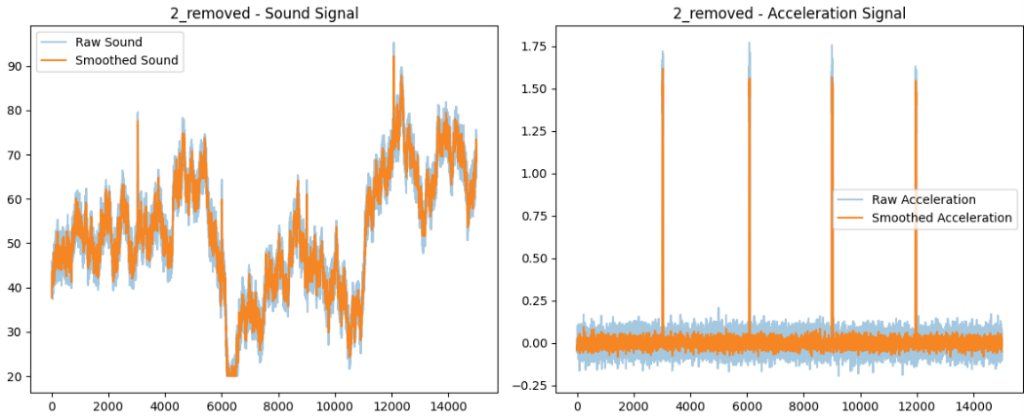


Figure 4.3.4 Sound and Vibrations Data for 2 springs removed

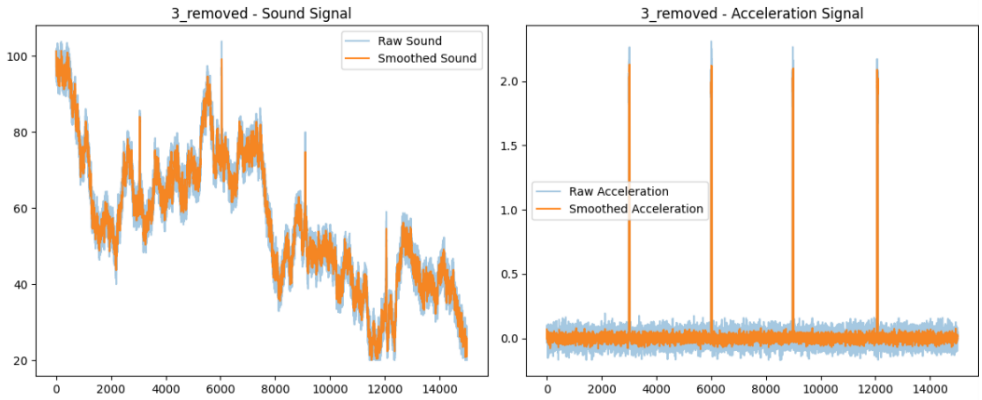


Figure 4.3.5 Sound and Vibrations Data for 3 springs removed

4.3.2 Model Training

This chapter discusses and explains the results from the training of classification models to identify mechanical failures in circuit breakers based on synthetically created sensor data. The main intention of this stage was to compare the performance of several machine learning models in discriminating various fault states, each with its unique spring removal context. The classification problem consisted of four classes based on the number of springs removed: 0, 1, 2, and 3. These classes were represented by combined sensor features based on both sound and acceleration signals, preprocessed and normalized to provide consistency and reliability across samples.

All five classifiers—Random Forest, XGBoost, Decision Tree, Support Vector Machine (SVM), and Naive Bayes—were compared and tested based on standard evaluation measures such as accuracy, precision, recall, F1-score, and confusion matrices. All models were trained

and validated based on a similarly prepared dataset for equal evaluation.

The outcomes are examined in two ways: first, the performance measures derived from each classifier are considered; second, the confusion matrices are investigated to learn more about the nature of mistakes committed by each model. The dual analysis yields a complete understanding of model performance and assists in deciding on the most trustworthy classifier for implementation in circuit breaker fault detection systems.

4.3.2.1 Classification Model

The results of classification derived from the preprocessed and concatenated normalized datasets also affirm the good separability between the four fault conditions that have been simulated on the circuit breaker. All the five machine learning classifiers, i.e., Random Forest, Decision Tree, SVM, Naive Bayes, and XGBoost, were trained and tested on the same dataset with an 80:20 train-test split, and their performances were compared using typical metrics: precision, recall, F1-score, and accuracy.

Of all models, the best-performing model was the Random Forest Classifier at an overall accuracy of 99.98%. Its classification report is perfect or nearly perfect with all classes as each fault condition (from 0_removed up to 3_removed) possess precision, recall, and F1-score value equal to or very close to 1. The confusion matrix of this model reveals only two misclassified items out of 12,000 samples, so it has the very good generalization and minimum overlap between the class boundaries.

Random Forest Classification Report:

	precision	recall	f1-score	support
0_removed	1.00	1.00	1.00	2984
1_removed	1.00	1.00	1.00	2925
2_removed	1.00	1.00	1.00	3077
3_removed	1.00	1.00	1.00	3014
accuracy			1.00	12000
macro avg	1.00	1.00	1.00	12000
weighted avg	1.00	1.00	1.00	12000

Figure 4.3.6 Random Forest Classification Report

```
Random Forest Accuracy: 0.9998
```

Figure 4.3.7 Random Forest Accuracy

The XGBoost classifier produced comparable results, also achieving 99.98% accuracy with almost identical metrics. The Decision Tree classifier followed closely, with a slightly lower accuracy of 99.95%.

```
XGBoost Classification Report:
              precision    recall  f1-score   support

   0_removed       1.00      1.00      1.00      2984
   1_removed       1.00      1.00      1.00      2925
   2_removed       1.00      1.00      1.00      3077
   3_removed       1.00      1.00      1.00      3014

    accuracy                           1.00     12000
   macro avg       1.00      1.00      1.00     12000
weighted avg       1.00      1.00      1.00     12000
```

Figure 4.3.8 XGBoost Classification Report

```
XGBoost Accuracy: 0.9998
```

Figure 4.3.9 XGBoost Accuracy

```
Decision Tree Classification Report:
              precision    recall  f1-score   support

   0_removed       1.00      1.00      1.00      2984
   1_removed       1.00      1.00      1.00      2925
   2_removed       1.00      1.00      1.00      3077
   3_removed       1.00      1.00      1.00      3014

    accuracy                           1.00     12000
   macro avg       1.00      1.00      1.00     12000
weighted avg       1.00      1.00      1.00     12000
```

Figure 4.3.10 Decision Tree Classification Report

```
Decision Tree Accuracy: 0.9995
```

Figure 4.3.11 Decision Tree Accuracy

Support Vector Machine (SVM) and Naive Bayes performed reasonably well but showed comparatively higher misclassification rates. SVM achieved an accuracy of 99.22%, with minor confusion especially between adjacent fault levels (e.g., some samples of 0_removed misclassified as 1_removed, and vice versa). Naive Bayes scored 99.08%.

SVM Classification Report:

	precision	recall	f1-score	support
0_removed	0.99	0.99	0.99	2984
1_removed	0.98	1.00	0.99	2925
2_removed	1.00	0.99	0.99	3077
3_removed	0.99	0.99	0.99	3014
accuracy			0.99	12000
macro avg	0.99	0.99	0.99	12000
weighted avg	0.99	0.99	0.99	12000

Figure 4.3.12 SVM Classification Report

```
SVM Accuracy: 0.9922
```

Figure 4.3.13 SVM Accuracy

Naive Bayes Classification Report:

	precision	recall	f1-score	support
0_removed	0.99	0.99	0.99	2984
1_removed	0.99	0.99	0.99	2925
2_removed	0.99	0.99	0.99	3077
3_removed	0.99	0.99	0.99	3014
accuracy			0.99	12000
macro avg	0.99	0.99	0.99	12000
weighted avg	0.99	0.99	0.99	12000

Figure 4.3.15 Naive Bayes Classification Report

```
Naive Bayes Accuracy: 0.9908
```

Figure 4.3.16 Naive Bayes Accuracy

In general, these findings highly support the discriminative capability of the normalized synthetic datasets. The classifiers were successful in learning meaningful signal patterns with respect to different fault levels. Ensemble-based models such as Random Forest and XGBoost exhibited the most consistent performance and thus are the prime candidates for integration into any subsequent fault detection system.

4.3.2.2 Confusion Matrix Heatmap

The confusion matrix results provide further detail of each model's classification behaviors and exhibit distinct patterns in misclassification. In Random Forest, the matrix is practically diagonal with merely two samples incorrectly classified out of all four classes—reflecting its potential for very precise prediction from a heterogeneous array of sensor signals. The large diagonals retain close to perfect values (e.g., 2984/2984 for 0_removed and 3014/3014 for 3_removed), reaffirming the consistency of the model in all fault scenarios.

XGBoost has almost the same matrix as Random Forest except for a single or double off-diagonal entry and otherwise perfect class separation. The matrix for Decision Tree also performs well with some off-diagonal entries, for instance, two 2_removed misclassified and one 3_removed sample weakly misclassified, resulting in an accuracy of 99.95%. These figures indicate that tree-based ensemble models are able to nicely extract complex patterns from high-dimensional synthetic data.

```
Random Forest Confusion Matrix:
[[2984    0    0    0]
 [   0 2925    0    0]
 [   0    2 3075    0]
 [   0    0    0 3014]]
```

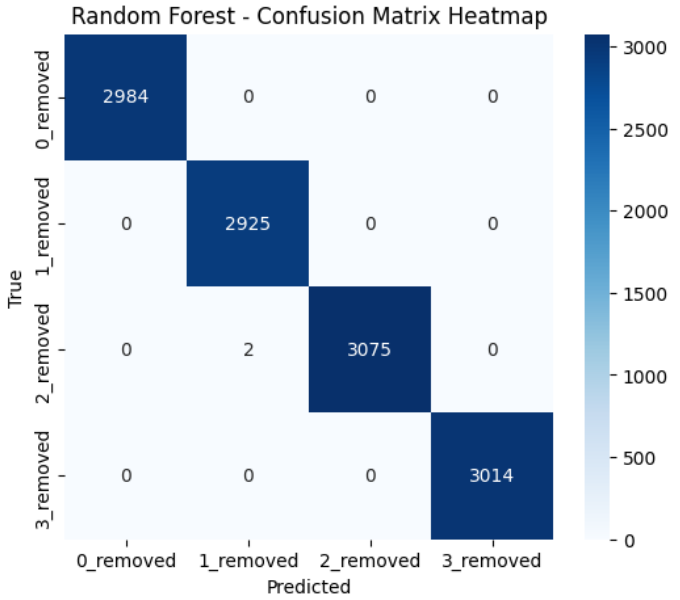


Figure 4.3.17 Random Forest Confusion Matrix

```
XGBoost Confusion Matrix:
[[2983    0    1    0]
 [   0 2925    0    0]
 [   0    2 3075    0]
 [   0    0    0 3014]]
```

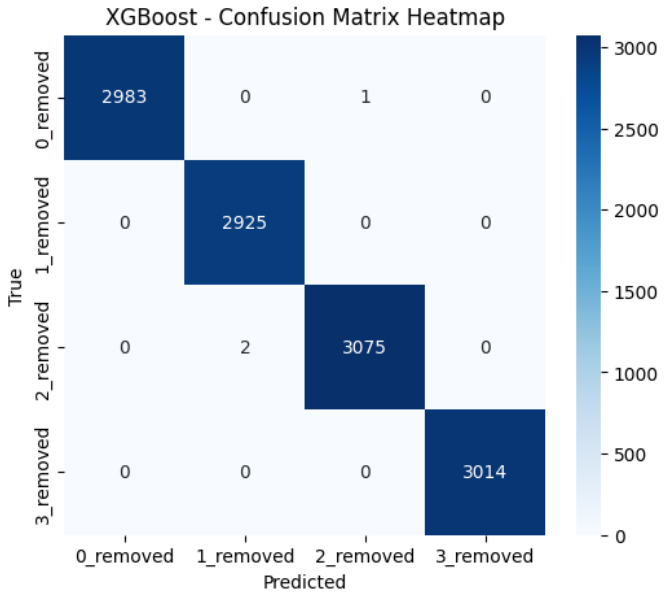


Figure 4.3.18 XGBoost Confusion Matrix

```
Decision Tree Confusion Matrix:
[[2983    0    1    0]
 [   0 2925    0    0]
 [   0    2 3073    2]
 [   0    0    1 3013]]
```

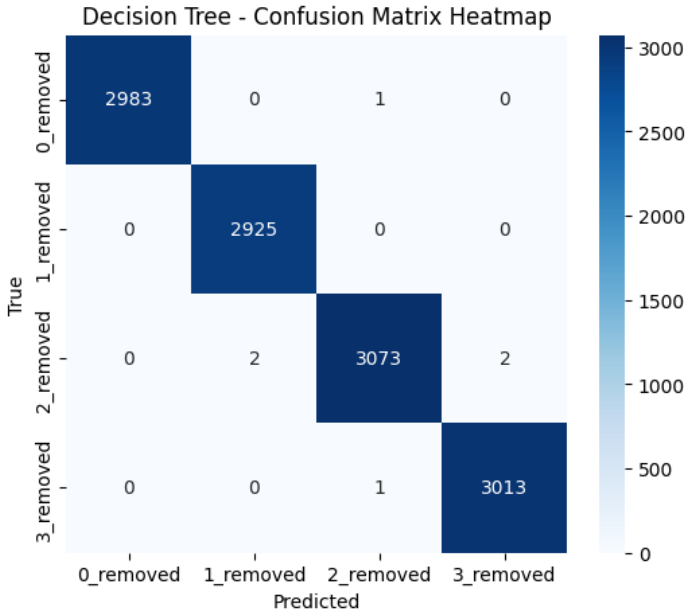


Figure 4.3.19 Decision Tree Confusion Matrix

Conversely, the SVM confusion matrix shows more dispersed misclassifications. For example, 20 samples of 0_removed were classified as 1_removed, and 18 samples of 3_removed were mixed up with neighboring classes. This confirms that SVM, while good, has some difficulties with fault conditions that share overlapping feature characteristics.

```
SVM Confusion Matrix:
[[2956   20    0    8]
 [   8 2917    0    0]
 [   8   18 3042    9]
 [   5   18    0 2991]]
```

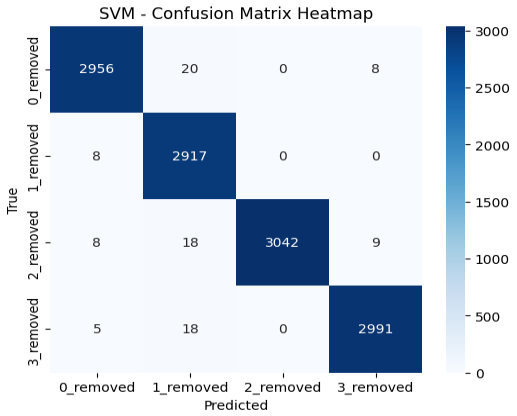


Figure 4.3.20 SVM Confusion Matrix

The Naive Bayes matrix also reflects moderate misclassification between neighboring fault levels. They include 22 cases of 0_removed incorrectly labeled as 3_removed and 18 misclassifications between 2_removed and 3_removed. These trends highlight the difficulty in dealing with subtle differences between classes through probabilistic models, especially where there is feature correlation.

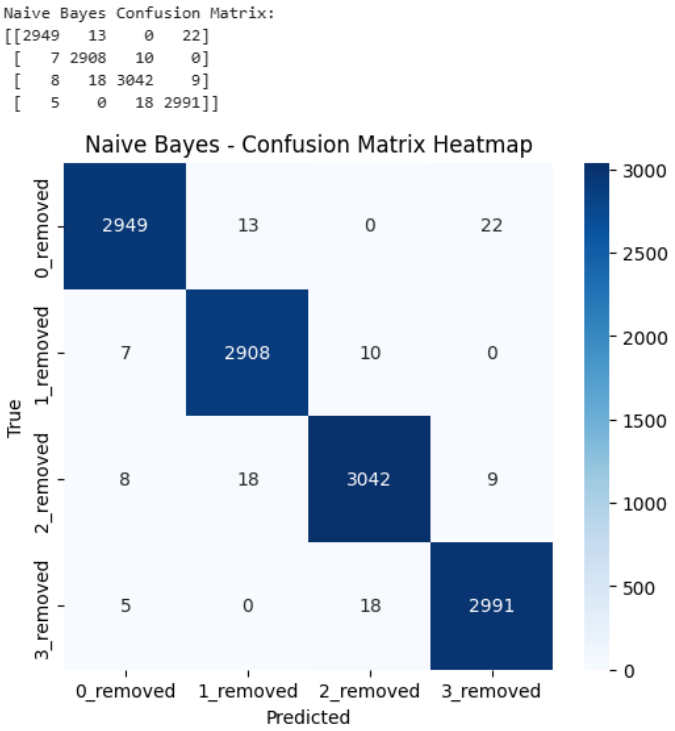


Figure 4.3.21 Naive Bayes Confusion Matrix

In combination, the confusion matrices confirm that tree-based classifiers not only had high overall accuracy but also provided class-specific consistency, which makes them highly appropriate for real-world fault detection applications where reliability is paramount. They also provide visual confirmation that fine differences between fault conditions are being successfully represented in the normalized feature space.

4.4 Software Development

The GUI application developed for fault detection in circuit breakers allows the uploading of sound and accelerometer sensor data in CSV format and makes fault predictions using five pre-trained machine learning models: Random Forest, Decision Tree, Support Vector Machine (SVM), Naive Bayes, and XGBoost. The data is preprocessed upon file upload, including smoothing through a rolling mean filter and normalization through the Min-Max scaling

method. Next, the accelerometer and sound datasets are combined, and their mean is calculated to create a uniformed input to be fed into the models.

All five machine learning models make an individual prediction for the fault condition ("0_removed", "1_removed", "2_removed", or "3_removed"). To reach a final conclusion, a majority voting approach is utilized. The technique combines all the predictions from the five models and picks the class label that gets the maximum number of votes as the final result.

In situations where the model predictions are split evenly, for example, a 2-2-1 voting pattern (i.e., two models vote for one class, two vote for another, and one votes for a third class), the system breaks the tie by applying a deterministic solution. That is, it applies the Counter.most_common() function, which chooses the first appearing label of those tied based on their order of appearance in the predictions. There is no randomization or further tie-breaking criteria being used, ensuring repeated runs under the same input are consistent.

Overall, the combination of several models and the implementation of majority voting significantly improve the robustness of the predictions. In addition, the GUI presents individual model results and the ultimate aggregated result, thus providing users with transparency and a full overview of the decision-making process of the system.

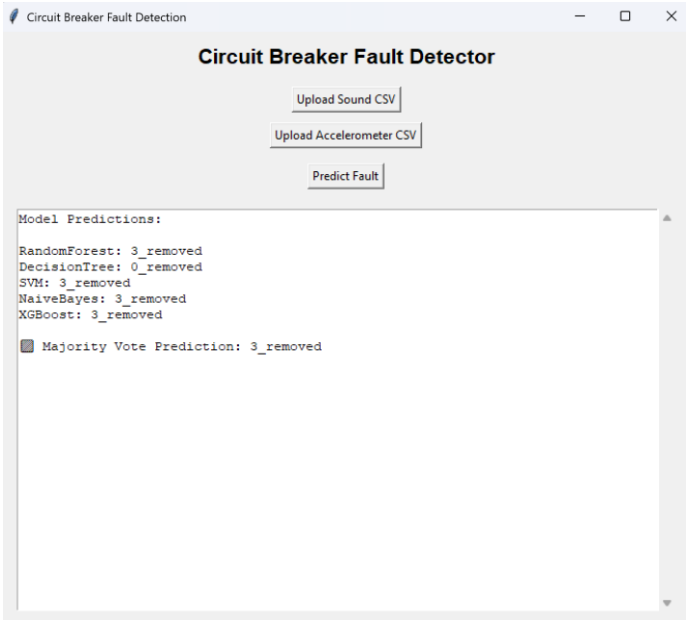


Figure 4.4.1 GUI Interface

CHAPTER 5: SUMMARY AND CONCLUSIONS

The project "Multi-Sensor Setup for Fault Detection in Circuit Breaker" offers a new, feasible, and scalable solution for predictive maintenance within the field of electrical protection systems. The key concept of this work is to utilize digital twin technology combined with vibration and acoustic signal monitoring for the detection of internal mechanical faults in circuit breakers, i.e., for anomalies in their spring mechanisms. Circuit breakers are a vital part of electrical safety, and circuit breaker failures can cause extensive downtime, operating risk, and equipment destruction. Thus, a trustworthy early fault detection system is necessary to avoid disastrous failures, enhance reliability, and decrease maintenance expenses.

The genesis of this project is the combination of low-cost, miniaturized, and highly efficient hardware components into a sound and consistent data acquisition system. Arduino Nano 33 IoT microcontroller was chosen for its wireless connectivity features and small form factor, making it a suitable choice for embedded monitoring purposes. The controller was interfaced with a 6-axis accelerometer and an analog sound level meter so that the system can capture vibration and sound signals continuously upon circuit breaker operation. The captured data, for four fault states—normal (0 springs unbolted), and progressive degradation cases (1, 2, and 3 springs unbolted)—were logged at the same time to a microSD card and streamed over Wi-Fi, to provide data redundancy and security in the event of communication loss.

Following successful data capture, an extensive preprocessing pipeline was executed with Python. This pipeline included smoothing signals through rolling averages, normalization of sensor measurements, and construction of formatted CSV data sets. These data sets constituted the training area for various supervised machine learning models, such as Random Forest, Support Vector Machine (SVM), XGBoost, Decision Tree, and Naive Bayes classifiers. Each of these algorithms was assessed using test accuracy, confusion matrices, and classification reports. The models that were trained performed well, with the Random Forest and XGBoost classifiers performing especially well in fault classification. The modular nature of the project made it simple to integrate these models into the subsequent stage: user interaction.

For ease of access and use, a Graphical User Interface (GUI) was created using the Python Tkinter library. This user-friendly GUI allows users to upload new sensor data sets (CSV format), initiate the prediction process with all five trained models, and get immediate classification outputs showing the circuit breaker's health status. The GUI-based system not

only facilitates improved user experience but also is an effective visualization and diagnostic tool, enabling scalable deployment in industrial settings. The final architecture, therefore, embodies a harmonious fusion of hardware, software, signal processing, and artificial intelligence towards effective and real-time monitoring of circuit breakers.

Overall, this project proves to be effective in establishing the viability and efficiency of a sensor-based, digital twin-supported fault detection architecture for circuit breakers. Utilizing multi-modal sensor information and employing state-of-the-art machine learning algorithms, the system ensures high accuracy in detecting internal spring-related faults. The classification models learned during this project were able to differentiate between four fault severity levels with great accuracy, and the strong relationship between mechanical spring degradation and vibration and sound signature changes was underscored.

One of the noteworthy accomplishments of this project is its comprehensive design—from real-time data sensing through IoT-enabled microcontrollers, to developing machine learning models, and lastly to implementing a GUI that makes the whole system interpretable and usable to users. The two-mode data logging strategy, which offers real-time Wi-Fi streaming backed by secure microSD storage, offers a solid basis for implementing this configuration in actual electrical substations and industrial control panels. In addition, the low-cost implementation using off-the-shelf components guarantees that this solution is not only affordable but also scalable across various applications such as smart grids, solar and wind-based renewable energy systems, and heavy industry machines.

Although the present iteration of the system is restricted to fault detection of spring removal, it provides the foundation for future development to incorporate a broad array of mechanical and electrical faults. The foundation established in this project can be applied to extend into real-time streaming analysis, edge computation, SCADA integration, and cloud-based predictive modeling. With continued advancements in signal filtering, diversity of datasets, and real-time deployability, this project can become a full-fledged, industry-standard predictive maintenance application. To some extent, the combination of smart analytics, embedded systems, and user-friendly design in this project represents an innovative strategy toward electrical safety and operational reliability in contemporary power systems.

CHAPTER 6: FUTURE SCOPE

1. Real-Time Monitoring and Logging: The current system can be extended to support real-time monitoring of circuit breaker health using live sensor data acquisition. This would enable immediate fault detection and proactive maintenance.

2. Optimized Data Logging with Peak Value Capture: Instead of continuous high-frequency data recording, a smarter approach could be adopted where only peak vibration or sound events are logged. This would significantly reduce storage requirements and eliminate unnecessary data, optimizing both system performance and memory usage.

3. Wireless Communication Integration: Future enhancements could include integrating Wi-Fi or Bluetooth modules for seamless wireless data transmission. This would eliminate the need for physical connections and make the system easier to deploy in complex electrical setups.

4. Detection of Additional Fault Types: The system can be expanded to detect a wider range of mechanical and electrical faults such as bearing wear, contact erosion, coil damage, or arc faults, providing a more comprehensive fault diagnosis platform.

5. Cloud Connectivity and IoT Integration: Linking the setup with Internet of Things (IoT) platforms could allow remote monitoring, automated reporting, predictive maintenance alerts, and centralized data analytics across multiple circuit breaker units.

6. Advanced Machine Learning and Deep Learning Techniques: Future development could focus on employing advanced algorithms such as deep neural networks, reinforcement learning, or ensemble models to improve fault classification accuracy and adapt to evolving circuit breaker conditions over time.

REFERENCES

[1] Ramakrishna S. & Badrinarayanan, V., “Power System Protection and Switchgear”, McGraw-Hill Education, 2nd Edition, 2011

[2] Hughes, E., Drury, J., & Smith, I., “Electrical and Electronic Technology”, Pearson Education, 11th Edition, 2012

[3] Stephen B. & David S., “Predictive Maintenance in Dynamic Systems”, Springer-Verlag, 2018

[4] Russell, S.J. & Norvig, P., “Artificial Intelligence: A Modern Approach”, Pearson, 4th Edition, 2021

[5] Raschka, S., & Mirjalili, V., “Python Machine Learning”, Packt Publishing, 3rd Edition, 2019

[6] Satish Kumar, Tushar Kolekar, Arunkumar Bongale, Shruti Patil, Atef Zaguia, Chander Prakash, “Low Cost Multi-Sensor Data Acquisition System for fault detection in fused deposition modelling”

[7] Yannick Wilhelm, Peter Reimann, Wolfgang Gauchel, Bernhard Mitschang, “Overview on hybrid approaches to fault detection and diagnosis: Combining data-driven, physics-based and knowledge-based models”, 14th CIRP Conference on Intelligent Computation in Manufacturing Engineering, CIRP ICME ‘20

[8] M. Runde T.Aurud L.E. Lundgaard G.E. Ottesen, K. Faugstad “ACOUSTIC DIAGNOSIS OF HIGH VOLTAGE CIRCUIT-BREAKERS”, IEEE Transactions on Power Delivery, vol. 7, NO. 3, jdY 1992

[9] Giovanni Marco Bisanti, Luca Mainetti, Teodoro Montanaro, Luigi Patrono∗, Ilaria Sergi, “Digital twins for aircraft maintenance and operation: A systematic literature review and an IoT-enabled modular architecture”, Elsevier 2023

[10] Aitor Duo, Tiziana Segreto, Alessandra Caggiano, Rosa Basagoiti, Roberto Teti, Pedro J. Arrazola, “Drilling process monitoring: a framework for data gathering and feature extraction techniques”, Elsevier 2020